Machine Learning, Deep Learning and Online Learning

Part I: Artificial Neural Networks and Deep Learning

Lorenzo Vergata

November 12, 2026

Contents

I Artificial Neural Networks and Deep Learning **9**

1 Introduction: from Machine Learning to Deep Learning **11**
1.1 What is Machine Learning? . . . 11
1.1.1 Supervised, unsupervised and reinforcement learning . . . 12
1.1.2 Classification and regression . . . 13
1.1.3 Unsupervised learning: clustering . . . 13
1.2 What is Deep Learning? . . . 13
1.2.1 The classical pipeline: the Iris dataset . . . 13
1.2.2 The real problem: features . . . 14
1.2.3 Learning the features . . . 14
1.2.4 Data as the real enabler . . . 15
1.2.5 2012: the ImageNet challenge . . . 15
1.2.6 Beyond classification . . . 16

2 The Perceptron and the Origins of Neural Networks **17**
2.1 AI, Machine Learning, Deep Learning and Neural Networks . . . 17
2.1.1 The birth of Artificial Intelligence . . . 17
2.1.2 Why neural networks? The limits of early computers . . . 18
2.1.3 The biological inspiration . . . 18
2.2 The artificial neuron . . . 18
2.2.1 How a biological neuron works . . . 18
2.2.2 The mathematical model . . . 19
2.2.3 Historical milestones . . . 20
2.3 What can a perceptron compute? . . . 20
2.4 Hebbian learning . . . 20
2.4.1 Hebb's postulate . . . 20
2.4.2 The Hebbian learning rule . . . 21
2.4.3 A worked example: learning the OR function . . . 22
2.4.4 Epochs, iterations and steps . . . 23
2.4.5 Does it always converge? To the same solution? . . . 23
2.5 The perceptron is a linear classifier . . . 23
2.5.1 The XOR problem and the first AI winter . . . 24
2.5.2 Multiple layers, and why Hebbian learning fails . . . 24

3 Feedforward Neural Networks and Backpropagation **27**
3.1 The feedforward neural network . . . 27
3.1.1 Topology . . . 27
3.1.2 Connectivity and number of parameters . . . 28
3.1.3 Activation functions . . . 29
3.1.4 Which activation where? . . . 29
3.2 Universal approximation . . . 31
3.3 Learning as optimization . . . 31

3.3.1 Gradient descent . . . 32
3.3.2 Backpropagation is gradient descent . . . 33
3.3.3 An explicit gradient computation . . . 33
3.3.4 Batch, stochastic and mini-batch gradient descent . . . 33
3.4 Automatic differentiation and the forward–backward pass . . . 34
3.4.1 The chain rule . . . 34
3.4.2 Forward pass and backward pass . . . 35
3.5 Laboratory: a first feedforward network in PyTorch . . . 35
3.5.1 Setup . . . 36
3.5.2 Data exploration . . . 36
3.5.3 Train, validation and test splits . . . 36
3.5.4 Normalization, tensors and data loaders . . . 37
3.5.5 The model . . . 37
3.5.6 Training hyperparameters . . . 38
3.5.7 The training loop . . . 38
3.5.8 Results . . . 39

4 Loss Functions and Maximum Likelihood Estimation 41
4.1 Maximum likelihood estimation . . . 41
4.1.1 An intuitive example . . . 41
4.1.2 The recipe . . . 41
4.1.3 MLE of the mean of a Gaussian . . . 42
4.2 Regression: the sum of squared errors . . . 42
4.2.1 Why the derivation matters . . . 43
4.3 Classification: the cross-entropy . . . 44
4.3.1 Binary classification . . . 44
4.3.2 Multi-class classification . . . 45
4.4 The loss defines the task . . . 45

5 Generalization, Overfitting and Regularization 47
5.1 Generalization and overfitting . . . 47
5.1.1 The inductive hypothesis . . . 47
5.1.2 Measuring generalization: the test set . . . 48
5.1.3 Training, validation and test sets . . . 48
5.1.4 The bias–variance trade-off . . . 49
5.2 Cross-validation . . . 49
5.3 Early stopping . . . 50
5.3.1 Hyperparameter tuning with early stopping . . . 51
5.4 Weight decay . . . 51
5.4.1 Maximum a posteriori estimation . . . 51
5.4.2 Derivation . . . 52
5.4.3 L1 versus L2 . . . 52
5.4.4 Choosing γ . . . 53
5.5 Dropout . . . 53
5.6 Laboratory: overfitting and regularization in practice . . . 54
5.6.1 The Glass dataset and unbalanced classification . . . 54
5.6.2 Baseline: overfitting . . . 55
5.6.3 Early stopping . . . 55
5.6.4 Dropout . . . 56
5.6.5 L1, L2 and elastic net . . . 56
5.6.6 AdamW: decoupled weight decay . . . 57
5.6.7 Final test . . . 57
5.7 Laboratory: evaluation strategies and hyperparameter tuning . . . 57
5.7.1 Four evaluation strategies . . . 57

5.7.2 Implementation details . . . 58
5.7.3 The main hyperparameters . . . 59
5.7.4 Search strategies . . . 59

6 Training Deep Networks: Activations, Initialization, Normalization and Optimizers 61
6.1 The vanishing gradient problem . . . 61
6.2 Modern activation functions . . . 62
6.2.1 Rectified Linear Unit (ReLU) . . . 62
6.2.2 Leaky ReLU and ELU . . . 62
6.2.3 Swish and the zoo of activations . . . 63
6.3 Weight initialization . . . 63
6.3.1 Xavier initialization . . . 64
6.3.2 Glorot and He initialization . . . 64
6.4 Batch normalization . . . 65
6.5 Beyond plain gradient descent . . . 65

7 Sequence Modeling: Recurrent Neural Networks and LSTM 67
7.1 From static to dynamic models . . . 67
7.1.1 Memoryless models . . . 67
7.1.2 Models with memory . . . 68
7.2 Recurrent neural networks . . . 68
7.2.1 Equations . . . 68
7.2.2 Backpropagation through time . . . 69
7.3 Long-term dependencies and the vanishing gradient . . . 69
7.3.1 Analysis on a single recurrent neuron . . . 70
7.4 Long Short-Term Memory . . . 70
7.4.1 The modern LSTM cell . . . 70
7.4.2 Gated Recurrent Unit . . . 71
7.5 Architectures and tricks . . . 72
7.6 Sequence-to-sequence problems . . . 72
7.7 Lab: time-series classification with recurrent networks . . . 73
7.7.1 The dataset and its preparation . . . 73
7.7.2 A configurable recurrent classifier . . . 74
7.7.3 Experiments and lessons learned . . . 75
7.8 Lab: time-series forecasting . . . 75
7.8.1 Data and sequences . . . 75
7.8.2 Direct forecasting . . . 76
7.8.3 Autoregressive forecasting . . . 77
7.8.4 Sequence-to-sequence with attention . . . 77

8 Autoencoders and Word Embeddings 79
8.1 Unsupervised learning, revisited . . . 79
8.2 Autoencoders . . . 79
8.3 The problem of representing words . . . 80
8.3.1 Distributed representations . . . 81
8.4 Neural language models . . . 81
8.4.1 The Neural Network Language Model . . . 82
8.5 word2vec . . . 82
8.5.1 Regularities of the embedding space . . . 83
8.5.2 Using embeddings . . . 84
8.5.3 GloVe . . . 84
8.6 A dictionary of layers . . . 85

9 Digital Images, Linear Filtering and Image Classification **87**
9.1 Digital images . . . 87
9.2 Local operations and linear filtering . . . 88
9.3 Image classification with a neural network . . . 89
9.3.1 The network is a linear classifier . . . 90
9.3.2 Two interpretations of a linear classifier . . . 90
9.4 Why image classification is challenging . . . 91

10 Convolutional Neural Networks **93**
10.1 From hand-crafted to data-driven features . . . 93
10.2 The architecture of a CNN . . . 94
10.3 Convolutional layers . . . 94
10.4 Activations, pooling and the dense part . . . 96
10.4.1 Feature space as a meaningful embedding . . . 97
10.5 Lab: images in Python and a first CNN . . . 97
10.5.1 Handling images . . . 97
10.5.2 Convolutions by hand . . . 98
10.5.3 Why convolutions instead of dense layers . . . 98
10.5.4 A CNN for CIFAR-10 . . . 99

11 Understanding and Training CNNs: Inductive Bias, Augmentation and Transfer Learning **101**
11.1 A convolutional layer is a constrained dense layer . . . 101
11.1.1 Inductive bias . . . 102
11.2 The receptive field . . . 102
11.3 Training a CNN . . . 103
11.3.1 The ImageNet breakthrough . . . 103
11.4 Data augmentation . . . 104
11.5 Transfer learning and fine-tuning . . . 105
11.6 Lab: augmentation, retrieval and transfer learning . . . 105
11.6.1 Augmentation on a small dataset . . . 105
11.6.2 Image retrieval . . . 107
11.6.3 Transfer learning and fine-tuning on Oxford-IIIT Pets . . . 107

12 Famous CNN Architectures and What CNNs Learn **109**
12.1 LeNet (1998) . . . 109
12.2 AlexNet (2012) . . . 110
12.3 VGG (2014) . . . 111
12.4 Visualizing what CNNs learn . . . 111
12.5 Network in Network and Global Average Pooling . . . 112
12.6 MobileNet and separable convolutions . . . 113

13 Normalization, Explainability and Modern CNN Architectures **115**
13.1 Data preprocessing . . . 115
13.2 Batch normalization in CNNs . . . 115
13.3 Multi-label classification and localization . . . 116
13.4 Class activation maps . . . 116
13.5 Inception (GoogLeNet, 2014) . . . 118
13.6 ResNet (2015) . . . 119
13.7 Lab: modern blocks, localization and class activation maps . . . 120
13.7.1 Building blocks . . . 120
13.7.2 Bounding-box regression . . . 121
13.7.3 Classification, CAM and Grad-CAM . . . 121

14 Semantic Segmentation **123**
14.1 The segmentation problem . . . 123
14.2 From classification to dense prediction . . . 124
14.3 Upsampling . . . 124
14.4 U-Net . . . 125
14.5 From a CNN classifier to a fully convolutional network . . . 126
14.6 Lab: fully convolutional networks and U-Net . . . 127
14.6.1 From a patch classifier to an FCN (sea lions) . . . 127
14.6.2 U-Net on Cityscapes . . . 128

15 Object Detection and Instance Segmentation **131**
15.1 Localization and pose estimation . . . 131
15.2 Multitask learning . . . 131
15.3 Object detection . . . 132
15.3.1 R-CNN (2014) . . . 132
15.3.2 Fast R-CNN (2015) . . . 133
15.3.3 Faster R-CNN (2015) . . . 133
15.3.4 YOLO . . . 134
15.4 Instance segmentation . . . 134
15.5 Lab: multitask learning and two-stage detectors . . . 135
15.5.1 Classification and localization with two heads . . . 135
15.5.2 Faster R-CNN and Mask R-CNN on Penn-Fudan . . . 136

Part I

Artificial Neural Networks and Deep Learning

Chapter 1

Introduction: from Machine Learning to Deep Learning

This first part of the book provides both the theoretical background and the practical skills needed to understand and use artificial neural networks, and to become familiar with deep learning architectures for complex engineering problems, with a particular emphasis on computer vision. The path we follow is the following:

- we start from the basics of neural networks, moving from the *perceptron* to *feedforward neural networks*, and we study how they are trained (backpropagation, loss functions, regularization, activation functions, initialization, normalization and optimizers);
- we then study *sequence models*: recurrent neural networks and Long Short-Term Memories, together with autoencoders and word embeddings;
- finally we dive into *visual recognition* tasks: image classification, convolutional neural networks (CNNs), the best practices to train them, famous architectures, and CNNs for advanced vision tasks such as semantic segmentation and object detection.

Transformers, large language models, multimodal models and graph neural networks are deliberately *not* covered: they build on top of everything presented here and belong to more advanced treatments. Similarly, generative text-to-image models are only mentioned. Alongside the theory, several chapters contain *laboratory sections* in which the models are implemented in PyTorch.

The goal of this chapter is to set the vocabulary: what machine learning is, which kinds of learning problems exist, and what distinguishes *deep* learning from classical machine learning. The take-home message of the chapter can be anticipated already: *it is all about features*.

1.1 What is Machine Learning?

Many definitions of machine learning can be found in the literature. A convenient one is the following.

Definition 1.1 (Machine Learning) *Machine learning is a category of research and algorithms focused on finding* patterns in data *and using those patterns to* make predictions*.*

Machine learning falls within the umbrella of Artificial Intelligence (AI), but it intersects many other disciplines. A popular diagram (originally drawn by SAS in 2014 and later modified by others) depicts machine learning as the intersection of:

- **Artificial Intelligence**;
- **Data Mining**;

- **Pattern Recognition**;
- **Statistics**.

In particular, it is impossible to think about machine learning without taking statistics into account, and throughout this book we will often refer to statistical concepts (e.g., maximum likelihood estimation in Chapter 4).

The reason why the field is complex to describe is historical. Machine learning is, at the end of the day, a *set of techniques*, and these techniques have been discovered and rediscovered over the years by different communities: people from AI, from pattern recognition, from statistics, from data mining. Very often the same technique has different names depending on the community that found it. Within machine learning we find, among many others: classification, regression, supervised learning, linear discriminant analysis, polynomial regression, ridge regression, model assessment, validation and cross-validation, clustering, and of course neural networks and deep learning. In this part of the book we will cover only the last two of these, but many of the other concepts (e.g., classification versus regression, validation) will be used extensively.

Remark 1.1 *The term "machine learning" became widespread with the 1997 book* Machine Learning *by Tom Mitchell, which gave a famous (and rather convoluted) definition in terms of a task T, an experience E and a performance measure P. Rather than discussing that definition, we focus on the distinction that will actually be used throughout the book: the one between supervised and unsupervised learning.*

1.1.1 Supervised, unsupervised and reinforcement learning

Some of the techniques developed in this book fall into the supervised class, others into the unsupervised one, so the distinction is important.

Notation. Throughout the book, every input $\mathbf{x}$ is a *vector*, not a number: one input $\mathbf{x}_n$ is one record of a dataset, i.e., a vector of features.

Definition 1.2 (Supervised learning) *Given a training set of N input–target pairs*

$$\mathcal{D} = \{(\mathbf{x}_1, t_1), (\mathbf{x}_2, t_2), \ldots, (\mathbf{x}_N, t_N)\}, \tag{1.1}$$

where each $\mathbf{x}_n$ is a vector of features and t_n is the desired output *(target) associated to it, the goal of supervised learning is to learn a model able to predict the correct target when a* new *input is given.*

Definition 1.3 (Unsupervised learning) *Given a dataset of inputs $\{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_N\}$* without *targets, the goal of unsupervised learning is to exploit the regularities in the data to build an abstract representation of them, which can later be used for reasoning or prediction.*

What makes the difference between the two is whether *someone* has given us the targets. If nobody needs to provide them, the problem is unsupervised. Note that this does not mean that unsupervised methods do not optimize anything or do not have errors: many of them minimize the very same kind of error functions used in supervised learning; what is different is the goal (finding regularities) and the absence of externally provided labels.

Definition 1.4 (Reinforcement learning) *In reinforcement learning the learner does not receive the desired output but a* reward*, which tells whether an action was good or bad. The reward may be delayed: an action taken today (e.g., investing in a stock) may reveal itself as good only after some time (e.g., when the stock doubles its value a month later). The goal is the long-term maximization of the return.*

Reinforcement learning is not treated in this part of the book. We will focus mostly on supervised learning, with some incursions into unsupervised learning.

1.1.2 Classification and regression

Within supervised learning a further distinction is fundamental, because it has a direct impact on the *architecture* of the model, on the *loss function* and on the *training* procedure.

Example 1.1 (Cars and motorcycles) *Consider a set of pictures, each labelled either as* car *or as* motorcycle*. The pictures are the data, the labels are the targets. In the classical machine learning pipeline, from each picture we extract some* hand-crafted features *(e.g., the size of the vehicle, its colour, the number of wheels) and we give them to a machine learning object, the* classifier*, which has to choose which of the two labels should be associated to the picture.*

Definition 1.5 (Classification) *In a classification problem the output is* one, and only one*, out of a finite set of possible outcomes (classes). If there are two classes, the problem is called* binary classification*; if the classes are more than two, it is a* multi-class classification *problem.*

Machine learning, in this setting, is about finding the classifier. The classifier can have many shapes: a decision tree, a logistic regressor, a support vector machine... In our case, it will be a neural network.

Definition 1.6 (Regression) *In a regression problem the output is a* number*, i.e., it ranges over the real values (or a subset of them, e.g., the integers or an approximation thereof).*

For instance, instead of labelling the picture of a car, we may want to associate to it the *value* (price) of the object. We no longer pick an option from a closed set: we have to output a number, so the output must be able to span the whole set of real values. The machine learned to do so is called a *regressor*.

Summarizing: classification and regression are both supervised; in classification the codomain is a (non-numerical) finite set of items, in regression the output is a real number. We will see that this difference changes the architecture of the network (in particular its output layer) and the loss function used to train it.

1.1.3 Unsupervised learning: clustering

Example 1.2 (Grouping vehicles without labels) *Suppose we are given a set of images of bicycles, motorbikes and cars, without any label. By looking at them for a while, we notice that some images look similar to each other, other images look similar among themselves but different from the first ones, and so on; maybe there is also an object that does not belong to any of the groups. No one supervised this task: we found the regularities by ourselves.*

This task is called *clustering*. Clustering is not studied in this book, but we will face related problems: for example, *word embeddings* (Chapter 8) find regularities in text data in an unsupervised way, using neural networks.

1.2 What is Deep Learning?

The picture painted so far (data, hand-crafted features, classifier) is the one of *classical* machine learning. Around 2012 the world of machine learning changed. In its 2013 list of breakthrough technologies, the *MIT Technology Review* placed deep learning at the top, stating that "with massive amounts of computational power, machines can now recognize objects and translate speech in real time; artificial intelligence is finally getting smart". All the big technology companies jumped on the topic, and countless start-ups were founded (the list changes every week, which is why it is pointless to report it here). To understand what changed, we start from a classic example.

1.2.1 The classical pipeline: the Iris dataset

The *Iris* dataset dates back to 1936 and was used by Ronald Fisher (the same Fisher of the Fisher information matrix). It is the "hello world" of machine learning. Each flower is described by four features: sepal length, sepal width, petal length and petal width, and the goal is to classify the flower into one of three species.

If we place all the examples in the space defined by the features (the *vector space model*), the points form a pattern, and the goal of the classifier is to find the *decision boundary* in this space. More or less every classification problem can be described like this:

1. we have an object to classify (an image, a text, a record of a database);
2. we extract some **hand-crafted features** from it;
3. we give the features to a **classifier**, whose job is to find the separation boundary between the examples of the different classes.

Sometimes the boundary is linear; sometimes the points are arranged in a more complex way and a non-linear classifier is needed. For instance, in a two-dimensional dataset where the points of one class lie inside a ring of points of the other class, a nearest-neighbour classifier works, a linear support vector machine (SVM) fails, an SVM with radial basis function (RBF) kernel works very well, and Naive Bayes works reasonably. In classical machine learning *this is the job of the practitioner*: understand the data, find the best classifier for that kind of data, and find the right features for the problem.

1.2.2 The real problem: features

What if the data look hopelessly mixed in feature space, so that no classifier seems able to separate the classes? Two explanations are possible:

1. the items are really indistinguishable (think of trying to tell apart two identical twins);
2. much more likely, the items are indistinguishable *using these features*.

Example 1.3 (The Italian fiscal code) *The Italian* codice fiscale *(the analogue of a social security number) encodes the gender of a person: the two digits encoding the day of birth are increased by 40 for women. Hence, the first of these two digits is at most 3 for men and at least 4 for women. If we try to predict the gender by looking at the wrong character, e.g., the final check character, the problem looks impossible; if we look at the right position, it becomes trivial.*

The activity of transforming raw data into features that make the problem easy, based on knowledge of the problem, is called **feature engineering**. With images this has historically been very hard: many attempts have been made to design features, but they were not general enough. Features that work for flowers do not work for cars, nor for faces, nor for anything else. Finding good, general features and a proper representation of an image is a difficult task.

Classical pipelines looked as follows:

- **Speech recognition:** from the sound, compute the *Mel-Frequency Cepstral Coefficients* (MFCC; loosely speaking, a Fourier transform of the log-spectrum, i.e., "the Fourier transform of the Fourier transform"); then possibly cluster them with a mixture of Gaussians, and use the clusters as features for a classifier.
- **Image recognition:** extract image descriptors such as SIFT or HOG (think of corners and the patch around them), then cluster and pool them, and train a classifier on the result.

These are *hand-engineered low-level features*, followed by mid-level features (clustering, pooling) and a classifier. The question is: who guarantees that MFCCs, SIFT or HOG are the right features? What if better features exist? One can build and optimize any processing chain on top of low-level features, but if those features hide the important part of the data, the problem will never be solved, or will be solved sub-optimally.

1.2.3 Learning the features

This is the idea of deep learning.

Definition 1.7 (Deep learning) *Deep learning is the task of* learning the features *(the representation) of the data such that the downstream task (classification, regression, ...) becomes easy. Deep learning learns, at the same time,* both the representation and the solution of the task.

Instead of spending time on the classifier, we learn features that make the classes (ideally) linearly separable. The name comes from the fact that learning is pushed "down, deep" towards the raw data. Two synonyms are commonly used:

- **end-to-end learning**: everything is trained together, from the raw data at one end to the class at the other end;
- **representation learning**: the key point is not the final classifier (which could be done with classical machine learning), but *how to learn a representation that makes the learning task easy.*

Very often the representation is learned in a *hierarchical* way: a sequence of feature extractors is applied to the raw data; the first ones extract low-level descriptors (edges, colour patches, colours), which are aggregated into mid-level features, which are in turn aggregated into high-level features, so that some features end up describing, e.g., the neck of a bird or parts of a car. None of these features is hand-coded: everything is learned end-to-end from data.

The representation can be so important that it is sometimes the main contribution of a model:

- *word embeddings* are an unsupervised way to learn a representation of words; from the embedding alone one can answer questions or classify via nearest neighbours;
- *BERT*-like encoders map a text into a vector, and information retrieval can be performed simply by nearest-neighbour search in the encoded space;
- visual image search (find images similar to a query) and face identification (the representation of a face must be close to previous pictures of the same person and far from other people) become easy once the right representation is available.

The better the representation, the easier the task.

1.2.4 Data as the real enabler

The 2012 *MIT Technology Review* attributed the breakthrough to computational power. However, the huge amount of data generated in the last twenty years, i.e., the *World Wide Web*, is the real enabler of deep learning. Most of the companies that first jumped on deep learning (search engines, social networks) are not computing companies (with the notable exception of GPU manufacturers such as NVIDIA): they are *data* companies. Computing power can be bought; large amounts of data are much harder to obtain. Today, large language models make this point even more evident.

1.2.5 2012: the ImageNet challenge

Why is 2012 considered the year of change? Because of the **ImageNet** Large Scale Visual Recognition Challenge: about 1.5 million images classified into 1000 categories, on which different algorithms competed. The evaluation metric is the *top-5 error*: for each image, the algorithm provides five guesses; if the true class is among them the prediction is considered correct, otherwise it is wrong. Typical outputs of the winning network are instructive:

- a *mite* is correctly recognized (other guesses: black widow, cockroach, tick, starfish);
- a *starfish*-like guess appears when a red object lies on a sandy background (the network associates sand with the sea);
- a mushroom is classified as *agaric*, which is a special kind of mushroom, so the error is not unreasonable;

- a picture of *cherries* is classified as *dalmatian* (the picture indeed contains a dalmatian dog in the background), with grape, elderberry and currant as subsequent guesses: this is an intrinsically ambiguous image, which leads to the notion of *multi-label classification* discussed later in the book.

All of this was achieved with a *convolutional neural network* (AlexNet). From the diagram of such a network one should be able, by the end of this part of the book, to write the code that implements it and to compute exactly its number of parameters.

1.2.6 Beyond classification

The same technology solves many other tasks, all covered in this book (except where stated):

- **semantic segmentation**: assigning a class to each pixel of an image (e.g., detecting buildings in aerial images, a binary version of the problem);
- **instance segmentation**: not only finding where persons are, but distinguishing the different persons in the picture;
- **pose estimation**: finding a person and the skeleton of that person;
- **style transfer**: rendering a picture with the style of another picture (e.g., of a painting);
- **super-resolution**: increasing the resolution of an image, rather easily done with a CNN;
- **text-to-image generation** (not covered here): early models had evident flaws (e.g., birds with the wrong number of legs), while modern multimodal models produce far more accurate results.

Take-home messages

1. It is all about **features**: deep learning means learning the representation together with the task.
2. To learn representations and tasks end-to-end we will use **neural networks**, which is why the next chapter starts from the very first artificial neuron.

Chapter 2

The Perceptron and the Origins of Neural Networks

This chapter has a historical flavour: it goes from the 1940s–1950s to the mid-1980s and shows how the first artificial neuron, the *perceptron*, was conceived, how it was trained with *Hebbian learning*, and why its limitations caused the first "winter" of neural network research. The perceptron is interesting not only historically: it is simple enough to be trained by hand on a blackboard, and in doing so we will already meet many concepts (learning rate, online learning, epochs, iterations, decision boundaries) that are central to modern feedforward neural networks, studied from Chapter 3 on.

2.1 AI, Machine Learning, Deep Learning and Neural Networks

As seen in Chapter 1, deep learning is machine learning in which both features and model (classifier, regressor, ...) are learned: one enters with the raw data and exits with the answer, without the classical sequence of pre-processing, feature engineering, feature selection and classification/regression. Deep learning is therefore a special subset of machine learning, which is in turn a subset of Artificial Intelligence. Nowadays, when people say "AI", they usually think about the results obtained with deep learning (generative AI, natural language processing, image recognition). However AI is much broader: it includes multi-agent systems, logic, planning and much more, and it was born in the 1950s.

Where do neural networks belong in this picture? Perhaps surprisingly, *neural networks were born together with AI.* They became extremely popular with deep learning because they are the natural tool for end-to-end learning, thanks to *backpropagation* (Chapter 3), which allows to propagate the error and therefore to train deep models from the raw features to the final decision. But neural networks were already one of the tools of machine learning in the 1980s, and people were working on them already in the 1950s. Roughly:

- **1940s–1969**: the perceptron and its learning rule (this chapter);
- **1985–2000s**: backpropagation and modern feedforward neural networks (next chapters);
- **∼2012 onwards**: deep learning, driven by image processing and convolutional neural networks (Chapters 10 and following).

2.1.1 The birth of Artificial Intelligence

AI was founded in 1956 at the Dartmouth workshop, proposed by four people: John McCarthy (Dartmouth College), Marvin Minsky (Harvard University), Nathaniel Rochester (IBM) and Claude Shannon (Bell Telephone Laboratories) — the same Shannon who introduced the notion of entropy in information theory. The proposal stated that the study was to proceed

> *on the basis of the conjecture that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it.*

The original idea was therefore not to build "intelligent machines" or to replicate the human brain, but to *model some aspects of human intelligence, including learning, so that a machine could reproduce the same results*. Among the topics listed in the proposal there were *automatic computers* (at the time, the idea of writing code in a high-level language and having a machine produce the binary code, i.e., a compiler, was pure fantasy — today we describe a program in natural language and a large language model writes the code) and, precisely, *neural nets*.

2.1.2 Why neural networks? The limits of early computers

In the 1940s and 1950s computers were bunches of cables plugged together: programming meant re-wiring the machine; a few years later, programs were written on punched cards. These machines were already appreciated: they were faster and more precise than humans at doing math (before them, "computers" were people doing calculations by hand). However, they were:

- **brittle**: with noisy data they could not provide a proper answer;
- **purely serial**, not parallel;
- **not adaptive** and **not fault tolerant**.

The researchers' conclusion was that, to obtain better performance, *new computing architectures* — literally new computers — were needed. If the aim was to simulate the human brain, they argued, one needed hardware similar to the human brain: massively parallel, fault tolerant, adaptive, and interacting directly with the environment (an early form of the end-to-end idea). History has arguably disproved the hypothesis that brain-like hardware is necessary, but this was the very beginning of artificial neural networks.

2.1.3 The biological inspiration

A simplified view of the human brain is the following:

- about 10^{11} (one hundred billion) neurons, i.e., cells;
- each neuron is connected on average to a few thousand other neurons ($\approx 10^4$), giving on the order of 10^{14}–10^{15} connections (synapses).

Taking inspiration from the brain, therefore:

- computation is *distributed* among a huge number of simple *non-linear* units;
- there is *redundancy*: there are more units than strictly needed (cells die and are renewed; a three-year-old child has about twice as many synapses as an adult);
- execution is *intrinsically parallel*: there is no von Neumann fetch–execute cycle; all neurons "fire" (perform their non-linear computation) together.

2.2 The artificial neuron

2.2.1 How a biological neuron works

The following is the textbook description of the neuron as understood in the 1940s–50s (much simpler than what neuroscience knows today, but sufficient for our purposes). A neuron is a cell with a body and filaments called *dendrites*. Electro-chemical charge (variations of positive and negative ions, not electricity as in a battery) arrives to the cell body through the dendrites and accumulates there, until it exceeds a threshold, the *membrane threshold*. When the threshold is exceeded, the charge is released entirely through another filament, the *axon*, which reaches the dendrites of other neurons, charging them in turn, and so on in a cascade.

The point where the axon of one neuron touches the dendrite of another is the *synapse*. At the synapse an electro-chemical reaction takes place, which, depending on the neuroreceptors and neurotransmitters involved, can either *increase* the charge of the receiving cell (*excitatory* synapse, positive contribution) or *decrease* it (*inhibitory* synapse, negative contribution).

2.2.2 The mathematical model

The computation above can be modelled in two steps:

1. **accumulation**: a weighted sum of the inputs $x_1, \ldots, x_I$ coming from other neurons, weighted by the *synaptic weights* $w_1, \ldots, w_I$ (positive weights are excitatory, negative ones inhibitory);
2. **thresholding**: the sum is compared with a threshold, called the *bias* b; if the sum exceeds the bias the output is "high", otherwise "low".

Definition 2.1 (Perceptron) *The output of the perceptron (neuron j) is*

$$h_j(\mathbf{x} \mid \mathbf{w}, b) = h\left(\sum_{i=1}^{I} w_i x_i - b\right), \tag{2.1}$$

where $h(\cdot)$ is a threshold activation function, either the sign *function*

$$\text{sign}(a) = \begin{cases} +1 & a > 0 \\ -1 & a < 0 \end{cases} \tag{2.2}$$

or the Heaviside step *function, which outputs* 1 *for positive and* 0 *for negative arguments.*

The two choices are equivalent: one is obtained from the other by a linear transformation (scaling and translation), so they are both just thresholding mechanisms. We will mostly use the symmetric one (± 1), as it makes the computations simpler.

The bias as a weight. By renaming $b = -w_0$ and assuming an additional fictitious input $x_0 = 1$ that is always equal to one, the sum and the bias can be merged:

$$\sum_{i=1}^{I} w_i x_i - b = \sum_{i=0}^{I} w_i x_i = \mathbf{w}^\top \mathbf{x}, \tag{2.3}$$

so that

$$h_j(\mathbf{x} \mid \mathbf{w}) = h\left(\sum_{i=0}^{I} w_i x_i\right) = h\left(\mathbf{w}^\top \mathbf{x}\right). \tag{2.4}$$

The bias is just a number, i.e., a parameter, so this renaming changes nothing; it simplifies notation and, as we will see, it allows the bias to be learned exactly like the other weights. (Both the summation and the vector notation are used in this book: they are just two ways of writing the same thing.)

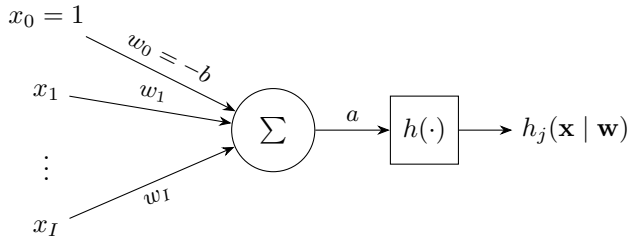


Figure 2.1: The perceptron: a non-linear (threshold) function applied to the weighted sum of the inputs, where the bias is treated as the weight of an input always equal to one.

Remark 2.1 *Real neurons are much more complex. Their behaviour is* time dependent*: after firing they have a refractory period during which they cannot fire again; summation is not instantaneous (the chemical reactions take time, giving an auto-regressive behaviour), and so on. We will not go beyond the simple model above, which dates back to the mid 1940s. In diagrams, a neuron will be drawn as a single node meaning "a non-linear function applied to the weighted sum of the inputs, one of which is always equal to one".*

2.2.3 Historical milestones

- **1943 – McCulloch and Pitts**: a neurophysiologist (Warren McCulloch) and a logician/mathematician (Walter Pitts) proposed the *Threshold Logic Unit*: a Heaviside step applied to a linear combination of the inputs. It was a model on paper.
- **1957–58 – Rosenblatt's Mark I Perceptron**: the first working hardware prototype, developed by Frank Rosenblatt. The inputs were voltages; each weight was a *potentiometer* (a variable resistor), so that the product weight×input was obtained physically; the contributions were added up, compared with a threshold, and a non-linear component (a comparator) produced the output. Since learning means changing the weights, the potentiometers were turned by *electric motors*: the large number of cables visible in pictures of the machine are mostly the cables of those motors. The idea was to show an example to the perceptron and let the motors modify the weights so that the machine *learns*.
- **1960 – Widrow and Hoff's ADALINE**: Bernard Widrow (with Ted Hoff) proposed to treat the bias, which was then either fixed or randomly set, as just *another input* whose weight is learned, exactly as in the formulas above. The perceptron we study is the McCulloch–Pitts neuron with this "patch".

2.3 What can a perceptron compute?

Since the perceptron was meant as an alternative to classical hardware, it should at least be able to do what Boolean circuits do. Consider Boolean inputs $x_1, x_2 \in \{0, 1\}$, the Heaviside activation, and $x_0 = 1$.

Example 2.1 (OR and AND) *With weights* $(w_0, w_1, w_2) = (-\frac{1}{2}, 1, 1)$ *we get*

x_1	x_2	$w_0 + w_1x_1 + w_2x_2$	h
0	0	$-1/2$	0
0	1	$1/2$	1
1	0	$1/2$	1
1	1	$3/2$	1

which is the ***OR*** *function. With weights* $(w_0, w_1, w_2) = (-\frac{3}{2}, 1, 1)$ *we get*

x_1	x_2	$w_0 + w_1x_1 + w_2x_2$	h
0	0	$-3/2$	0
0	1	$-1/2$	0
1	0	$-1/2$	0
1	1	$1/2$	1

i.e., the output is 1 *only if all inputs are* 1*: this is the* ***AND*** *function.*

The same piece of hardware, with different parameters, is *re-programmed* to behave as a different logic gate — a sort of ancestor of field-programmable gate arrays. Two advantages follow:

1. the circuit is always the same; only the potentiometers change;
2. most importantly, the weights can be *learned* simply by showing examples and modifying the weights according to a simple rule. With classical logic networks one has to *design* the circuit to implement the desired function; with the perceptron one shows examples of the truth table and the weights adapt.

The key element is therefore the *rule used to update the weights*.

2.4 Hebbian learning

2.4.1 Hebb's postulate

In 1949 the psychologist Donald Hebb, convinced (over-ambitiously) that understanding how neurons work would explain human behaviour, studied the behaviour of a few connected cells. His observation is known as Hebb's postulate:

The strength of a synapse increases according to the simultaneous activation of the relative input and the desired target.

In other words, if an input is high when the output is high, the next time it will be easier for the output to be high when the input is high: the connection becomes stronger. Hebbian learning is learning that happens through *repeated synchronous patterns*.

Example 2.2 (Shifting gears) *Think of learning to drive a car with a manual gearbox. To shift gear one has to press the clutch, disengage the gear, engage the new gear and release the clutch while slowly pressing the accelerator. In old cars without synchronized gears (such as the Fiat 500 of the 1960s–70s) one even had to perform a "double clutch" (*doppietta*): press the clutch, disengage, release the clutch, press the accelerator to synchronize the gears, press the clutch again, engage, release. At first each step requires conscious control. After many repetitions, the sequence triggers as a single automatic move: we do not think about the individual actions, we just "shift gear". This automatism can even fool us: a driver used to a manual car, driving an automatic one for the first time, may react to the brake lights of the car ahead by pushing the leftmost pedal (which in a manual car is the clutch) with full force — but in an automatic car the leftmost pedal is the brake. A "path" in the brain links the stimulus to the sequence of actions. This is not literally caused by Hebbian learning, but it illustrates how Hebbian learning was expected to work: simultaneous activations, repeated many times, create stronger and stronger paths.*

2.4.2 The Hebbian learning rule

Definition 2.2 (Hebbian learning rule) *Starting from (random) weights, at step k each weight is updated as*

$$w_i^{(k+1)} = w_i^{(k)} + \Delta w_i, \qquad \Delta w_i = \eta\, x_i\, t, \tag{2.5}$$

where x_i is the i-th input, t the desired *output (target) and $\eta > 0$ the* learning rate. *The update is applied* only when the perceptron makes a mistake *on the current example; if the example is correctly classified the weights are left unchanged.*

With the ± 1 coding the rule has an immediate interpretation:

- if input and desired output *agree* ($+1 \cdot +1$ or $-1 \cdot -1$), the product is positive and the synapse is strengthened ("turn the potentiometer knob up");
- if they *disagree* ($+1 \cdot -1$ or $-1 \cdot +1$), the product is negative and the synapse is weakened, possibly becoming negative (inhibitory).

The product $x_i t$ tells *whether* to turn the knob up or down; the learning rate η tells *how much*, i.e., how fast we learn, or how quickly we are convinced by the examples we see.

Remark 2.2 *For the perceptron, the value of $\eta > 0$ does not affect whether the algorithm succeeds: with $\eta = 0.1, 0.5, 1, 2, \ldots$ the procedure converges anyway (possibly to a different solution). This is why many perceptron implementations simply use $\eta = 1$. We keep η explicitly because the same notion will be central in modern neural networks.*

The algorithm is:

1. initialize the weights (randomly);
2. show one sample: if the perceptron predicts it correctly, do nothing; if it is wrong, correct the weights with (2.5);
3. move to the next sample; after the last sample, restart from the first one;
4. stop when *all* the samples are correctly classified.

2.4.3 A worked example: learning the OR function

We want the perceptron to learn the OR function with ± 1 coding (output -1 only when both inputs are -1). We start from $\mathbf{w}^{(0)} = (0, 0, 0)$ (we will see in Chapter 6 that all-zero initialization is the worst possible choice for modern networks, but here it is convenient), and use $\eta = 0.5$ (also chosen for convenience; any other value would work). The training set is

record	x_0	x_1	x_2	t
1	1	-1	-1	-1
2	1	-1	1	1
3	1	1	-1	1
4	1	1	1	1

The output is $y = \text{sign}(w_0 x_0 + w_1 x_1 + w_2 x_2)$.

Epoch 1.

- *Record 1*: $y = \text{sign}(0 \cdot 1 + 0 \cdot (-1) + 0 \cdot (-1)) = \text{sign}(0)$. The sign of zero is not defined (neither $+1$ nor -1): we treat it as 0, i.e., as an error. Update:

$$\Delta \mathbf{w} = \eta\, t\, \mathbf{x} = \tfrac{1}{2} \cdot (-1) \cdot (1, -1, -1) = (-\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2}), \qquad \mathbf{w}^{(1)} = (-\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2}).$$

- *Record 2*: $y = \text{sign}(-\tfrac{1}{2} \cdot 1 + \tfrac{1}{2} \cdot (-1) + \tfrac{1}{2} \cdot 1) = \text{sign}(-\tfrac{1}{2}) = -1 \neq 1$. Update:

$$\Delta \mathbf{w} = \tfrac{1}{2} \cdot 1 \cdot (1, -1, 1) = (\tfrac{1}{2}, -\tfrac{1}{2}, \tfrac{1}{2}), \qquad \mathbf{w}^{(2)} = (0, 0, 1).$$

  Notice that record 2 has been processed using the weights updated after record 1. Updating the weights after each sample and immediately using them is called **online learning**.

- *Record 3*: $y = \text{sign}(0 + 0 + 1 \cdot (-1)) = -1 \neq 1$. Update:

$$\Delta \mathbf{w} = \tfrac{1}{2} \cdot 1 \cdot (1, 1, -1) = (\tfrac{1}{2}, \tfrac{1}{2}, -\tfrac{1}{2}), \qquad \mathbf{w}^{(3)} = (\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2}).$$

- *Record 4*: $y = \text{sign}(\tfrac{1}{2} + \tfrac{1}{2} + \tfrac{1}{2}) = \text{sign}(\tfrac{3}{2}) = 1$. Correct, no update.

Are we done? *No*: by fixing the weights for the later records, we might have broken the prediction on the earlier ones. We have performed one pass through the data.

Epoch 2. With $\mathbf{w} = (\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2})$:

- record 1: $\text{sign}(\tfrac{1}{2} - \tfrac{1}{2} - \tfrac{1}{2}) = \text{sign}(-\tfrac{1}{2}) = -1$: correct;
- record 2: $\text{sign}(\tfrac{1}{2} - \tfrac{1}{2} + \tfrac{1}{2}) = 1$: correct;
- record 3: $\text{sign}(\tfrac{1}{2} + \tfrac{1}{2} - \tfrac{1}{2}) = 1$: correct;
- record 4: $\text{sign}(\tfrac{3}{2}) = 1$: correct.

All records are correctly classified: training stops with $\mathbf{w} = (\tfrac{1}{2}, \tfrac{1}{2}, \tfrac{1}{2})$. Had an error occurred, say, on record 3 of the second epoch, we would have corrected the weights, completed the epoch, and started a new one, stopping only when the whole dataset is correctly predicted. As an exercise, the same procedure can be repeated with $\{0, 1\}$ coding and the Heaviside function: different weights are obtained, but the procedure works as well.

2.4.4 Epochs, iterations and steps

Definition 2.3 (Epoch, iteration, step) *An **epoch** is one pass through the whole training set. An **iteration** (or **step**) is one update of the parameters.*

In the example above, the first epoch consisted of four iterations (in online learning, one iteration per sample). Training a network "for 100 epochs" means making 100 passes through the data. The term *episode* should not be used in this context, since it has a different meaning (e.g., in reinforcement learning and meta-learning). We will see in Chapter 3 that, depending on how many samples are used for each update, the relation between the number of iterations and the number of epochs changes.

2.4.5 Does it always converge? To the same solution?

Two natural questions arise.

1. *Does Hebbian learning always converge, for any dataset and initialization?* If the function *can* be learned by the perceptron, the algorithm always converges; if it cannot be learned, it never converges. Which functions can be learned is discussed below.

2. *Does it always converge to the same weights?* No. The solution depends on the initialization and on the learning rate: starting from $(1, 0, 0)$, or using $\eta = 0.1$, may lead to a different set of weights, which still solves the problem.

2.5 The perceptron is a linear classifier

The perceptron outputs $+1$ if the weighted sum of the inputs is positive and -1 if it is negative. The *decision boundary* is where the weighted sum is zero:

$$\sum_{i=0}^{I} w_i x_i = \mathbf{w}^\top \mathbf{x} = 0, \tag{2.6}$$

which is a *hyperplane* in the I-dimensional input space: a plane in three dimensions, a line in two. With two inputs,

$$w_0 + w_1 x_1 + w_2 x_2 = 0 \quad \Longrightarrow \quad x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2} x_1, \tag{2.7}$$

which is a line in the (x_1, x_2) plane with intercept $-w_0/w_2$ and slope $-w_1/w_2$. Whatever lies on one side of the line is classified as $+1$, whatever lies on the other as -1.

Remark 2.3 *Although the perceptron is a* non-linear *function (a threshold) of a* linear *combination of the inputs, its decision boundary is linear. Hence the perceptron is a **linear classifier**, exactly like linear discriminant analysis or logistic regression; it is simply another linear classifier, trained with Hebbian learning.*

It follows that a perceptron can learn exactly the **linearly separable** functions, i.e., those for which the inputs of the two classes can be separated by a hyperplane.

For the OR function, the point $(-1, -1)$ (red) can be separated from the other three (green) by a line; for the AND function, the point $(1, 1)$ can be separated from the others. Hence both are learnable and Hebbian learning converges. Not always to the same weights, though: there are infinitely many lines separating the points.

Remark 2.4 *Precisely the fact that infinitely many separating hyperplanes exist motivated Vapnik and colleagues to look for the* best *one among all possible perceptrons: the hyperplane with* maximum margin. *This is the origin of the* support vector machine.

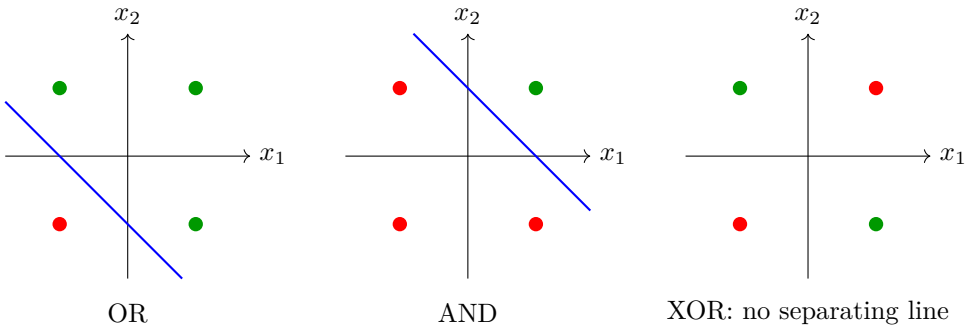


Figure 2.2: OR and AND are linearly separable (green: $+1$, red: -1); XOR is not.

2.5.1 The XOR problem and the first AI winter

There are Boolean functions that are *not* linearly separable, the simplest being the exclusive OR (XOR): the points $(-1, 1)$ and $(1, -1)$ must be classified as $+1$, while $(-1, -1)$ and $(1, 1)$ as -1, and no line can separate them (Figure 2.2). In 1969 Marvin Minsky and Seymour Papert, in their book *Perceptrons*, studied the perceptron from a geometric perspective and proved that this happens in any dimension: whatever the dimensionality of the input, if the function to be learned is not linearly separable, the perceptron cannot learn it.

This result had a huge impact and essentially stopped research on neural networks for years: what is the point of working on a model which is proven not powerful enough to solve non-linear problems? The history of the field shows a sequence of "summers" and "winters":

- from the 1940s–50s until the early 1970s: great hype on AI (not only due to the perceptron: other expected results, such as automatic translation, were not delivered either), followed by the **first AI winter**;
- 1985: the invention of *backpropagation* revived research on neural networks, with large investments (e.g., by DARPA during the Cold War, especially on natural language processing for Russian–English automatic translation), until a new disillusionment around the 1990s–2000: the **second AI winter**;
- 2012 onwards: the deep learning era. If history repeats itself, one should be prepared for a third winter.

2.5.2 Multiple layers, and why Hebbian learning fails

The solution to the XOR problem was conceptually simple. Even with logic gates, one does not implement an arbitrary circuit with a single gate: several layers of gates are combined, and with enough layers any Boolean function can be implemented. The same holds for perceptrons:

- with one layer (one perceptron) one can only draw a line (hyperplane);
- with two layers one can carve complex (convex) regions in the input space;
- with several layers one can carve any decision boundary.

The problem is that *Hebbian learning does not work with multiple layers*. Recall that, for a weight connecting input x to an output whose desired value is t, the update is $w^{(k+1)} = w^{(k)} + \eta\, x\, t$: both the input to the synapse and the desired output of the neuron must be known. With a single layer this is the case. With two or more layers:

- for a weight entering a neuron of the first layer, we know its input x, but we do not know the *desired output* of that intermediate neuron: we only know the desired output of the final neuron;
- for a weight entering the output neuron, we know the target, but we do not know the input for which the target should be high, since that input is the output of an intermediate neuron.

The deeper the network, the worse the problem. So the issue is not the representational power of multi-layer perceptrons — they can carve the input space as desired, and we will see that they can approximate any function — but the fact that *they cannot be trained with Hebbian learning*.

Between 1969 and 1985 researchers tried to fix the training of multi-layer perceptrons in various ways (e.g., the Widrow–Hoff rule, the delta rule), until the *backpropagation* algorithm was introduced for a different architecture: the **feedforward neural network**. It is sometimes (improperly) called a *multi-layer perceptron* (MLP), but it is not made of perceptrons: its activation functions are no longer thresholds. This is the subject of the next chapter.

Chapter 3

Feedforward Neural Networks and Backpropagation

At the end of Chapter 2 we were left with two facts: a single perceptron is a linear classifier and cannot learn non linearly separable functions; networks with multiple layers of perceptrons could, but they cannot be trained with Hebbian learning. There were actually two obstacles: Hebbian learning requires the input and the target of every synapse, which are unknown for hidden units; and, as we will see in this chapter, the threshold activation itself makes gradient-based training impossible. Between 1969 and the mid-1980s many partial fixes were proposed (the Widrow–Hoff rule of ADALINE, the delta rule, ...), but no general approach existed until *backpropagation*. We will not follow the original, cumbersome derivation of 1985; we will instead adopt the modern viewpoint, which presents learning as an *optimization* problem and backpropagation as *gradient descent* computed efficiently through *automatic differentiation*.

3.1 The feedforward neural network

The architecture we study looks very much like a multi-layer perceptron — so much that it is often called *MLP* — but it is *not* made of perceptrons: its units do not use a threshold as activation function. Removing the threshold is not a weakening: it is precisely what makes training possible.

3.1.1 Topology

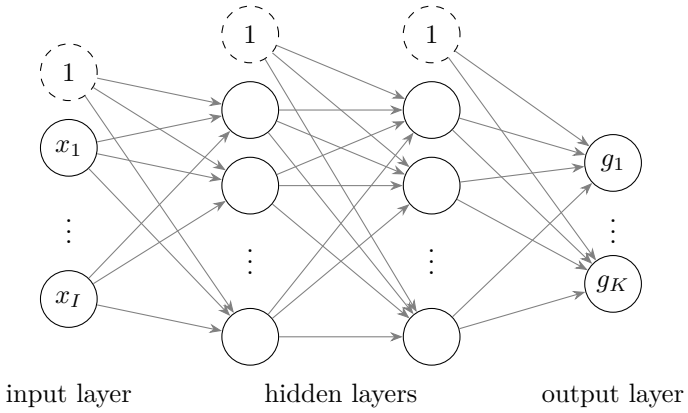


Figure 3.1: A feedforward neural network with two hidden layers. Dashed nodes are the bias units, always equal to one.

A feedforward neural network (Figure 3.1) is organized in layers.

- The **input layer** only redirects the inputs to all the neurons of the first hidden layer: it performs no real computation (one can think of an identity activation function). The number of neurons in the input layer is equal to the number of inputs *plus one*, the bias, which, as for the perceptron, is a fictitious input always equal to one.

- The **output layer** provides the final output of the network. Its number of neurons is fixed by the problem: if the codomain is one-dimensional there is one output neuron; if we must predict, e.g., the (x, y) position of something in an image, there are two; in general, if the problem requires K numbers, there are K output neurons. Thus, if a problem takes three numbers and predicts five, the input layer has three neurons (plus bias) and the output layer five.

- The layers in between are the **hidden layers**: looking at the network from the outside, they are hidden by the input layer on one side and by the output layer on the other.

The number of hidden layers and the number of neurons in each of them are *not* predefined: the more neurons and layers, the more parameters and the more powerful (complex) the model. They are design choices.

Definition 3.1 (Parameters and hyperparameters) *The **parameters** of a network are its weights (including biases). The **hyperparameters** are the design choices that define the structure of the model and of its training — e.g., the number of layers, the number of neurons per layer, the activation functions — and changing them changes the parameters themselves (e.g., their number).* Parameter tuning *means training the network;* hyperparameter tuning *means selecting the architecture.*

The weights are found by a training algorithm (backpropagation). There is no automatic rule to choose the number of layers and neurons: it is a matter of experience, trial and error, methods and tools (hyperparameter tuning). For the activation functions, instead, some rules of thumb exist, given below.

3.1.2 Connectivity and number of parameters

In a feedforward network each neuron is connected to *all* the neurons of the previous layer. Such layers are called *fully connected* or *dense* (the name used by many frameworks). If a layer with $J_{\ell-1}$ neurons (plus bias) feeds a layer with J_ℓ neurons, the number of weights between them is $(J_{\ell-1}+1)\,J_\ell$. Hence, for a network with I inputs and hidden layers of sizes $J_1, J_2, \ldots,$ and K outputs,

$$\#\text{params} = (I+1)J_1 + (J_1+1)J_2 + \cdots + (J_L+1)K. \tag{3.1}$$

Example 3.1 *Two layers of 10 neurons each are connected by* $10\cdot10+10 = 110$ *weights (*10×10 *connections plus 10 bias weights). Two layers of 100 neurons are connected by about* 10^4 *weights; two layers of 1000 neurons by about* 10^6*. Since the number of connections of a layer is quadratic in the number of neurons, the number of parameters grows very fast, and one easily reaches millions of parameters without even noticing.*

The dense connection pattern is the most general one, since a network with missing connections is equivalent to a dense one with some weights fixed to zero. Later, with convolutional neural networks, we will see *sparse* connectivity and weight sharing, which are far more efficient.

For simplicity we also assume that each layer is connected *only* to the previous one. This is not a requirement: *shortcut* (skip) connections, e.g., from the input to the third layer, are possible and very important (for instance in the U-Net and in residual blocks, studied in later chapters). Everything said here also holds with shortcut connections; we omit them only because formulas and diagrams would become heavier.

With this topology the signal propagates only *forward*: no loops, no connections within a layer. This is why these networks are called **feedforward** neural networks. Recurrent networks, which contain loops, will be studied in Chapter 7.

3.1.3 Activation functions

Activation functions must be differentiable, because training requires computing derivatives. This is the second reason why the threshold of the perceptron does not work: the step function is not differentiable at the origin and, even ignoring the origin, its derivative is *zero everywhere*. From the point of view of derivatives it is the worst possible function. At the time of the perceptron, derivatives were not a concern; for feedforward networks they are the key.

Let $a = \sum_i w_i x_i$ denote the *activation value* (the weighted sum) of a neuron.

Linear. $g(a) = a$, with $g'(a) = 1$. It is continuous, differentiable, with a very simple derivative, and it is indeed useful (see below). However it cannot be used everywhere: if all neurons were linear, the network would compute a linear function of a weighted sum of linear functions of weighted sums... i.e., a single linear function. The network would be no more powerful than a linear regression, with many more parameters than needed and the associated numerical issues.

Sigmoid. A continuous, differentiable version of the step function, close to 0 for negative arguments and close to 1 for positive ones:

$$g(a) = \sigma(a) = \frac{1}{1 + e^{-a}}, \qquad g'(a) = g(a)\big(1 - g(a)\big). \tag{3.2}$$

Until about ten years ago it was the best-known activation function. Its derivative can be computed from the *value* of the function: e.g., $\sigma(0) = 0.5$, hence $\sigma'(0) = 0.5 \cdot (1 - 0.5) = 0.25$. This is not essential but very convenient computationally, as we will see with the forward–backward computation.

Hyperbolic tangent. A continuous version of the sign function, ranging in $(-1, 1)$, anti-symmetric, tending to -1 for $a \to -\infty$ and to $+1$ for $a \to +\infty$:

$$g(a) = \tanh(a) = \frac{e^{a} - e^{-a}}{e^{a} + e^{-a}}, \qquad g'(a) = 1 - g(a)^2. \tag{3.3}$$

Again, the derivative is a function of the value of the function.

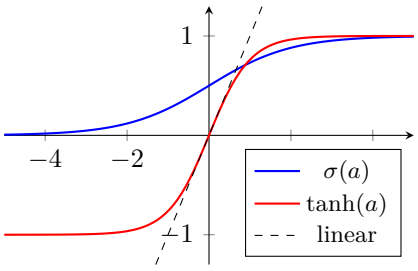


Figure 3.2: The three classical activation functions.

These are the historical activation functions, still in use, although they have been largely replaced in hidden layers by more modern ones (ReLU, Leaky ReLU, Swish, GELU, ...) for reasons that will become clear in Chapter 6.

3.1.4 Which activation where?

Viewing the network as a model with features as input, the choice of the output activation depends on the problem.

Regression. The output is a real number in $(-\infty, +\infty)$. Among the three functions above, only the linear one has this range: *use a linear activation in the output layer*. One could normalize the targets to $[-1, 1]$ and use a tanh, but there is no real advantage and there are risks: normalization is typically performed with the minimum and maximum observed in the training set, which need not be the minimum and maximum possible values. If extrapolation is needed, the network cannot produce it, because it has been forced to saturate between the observed extremes. (In some cases one wants the output to be, e.g., only positive; we will see how later.)

Binary classification. The two classes can be coded as $\{-1, +1\}$ (as in support vector machines, whose output is the sign of a combination of kernels), in which case the natural output activation is tanh; or as $\{0, 1\}$, in which case the natural output is the *sigmoid*. Most practitioners choose the latter, because an output such as 0.95 is naturally interpreted as the *posterior probability* of class 1, 0.7 as "class 1 but not very confident", 0.1 as "class 0 with high confidence".

Remark 3.1 *This is an* interpretation*: a neural network is not guaranteed to approximate a probability distribution. The output is better called the* confidence *of the network: a network can be very confident and completely wrong. Neural networks, when wrong, are often very confident in their mistakes.*

Multi-class classification. With $K > 2$ classes we use **one-hot encoding**: K output neurons, and each class is represented by a vector with a single 1 in the position of the class, e.g., $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$ for three classes. (One-hot encoding was also the standard way to encode words — a vector as long as the dictionary, with a one in the position of the word — until about 2012–2015; it is inefficient and problematic, and Chapter 8 shows how to do better.) If each output used a sigmoid, we could get, e.g., $(0.7, 0.8, 0.3)$: we would pick the maximum, but the decision is not clear-cut and the outputs do not sum to one. We would like the maximum to stand out and the outputs to sum to one, to keep the probabilistic interpretation. This is achieved by the **softmax**:

$$g_k(\mathbf{x}) = \frac{e^{a_k}}{\sum_{k'=1}^{K} e^{a_{k'}}}, \tag{3.4}$$

where a_k is the activation value of output neuron k. Dividing by the sum normalizes the outputs to sum to one; the exponential, being non-linear, enhances the largest activation (small values grow a little, large values grow exponentially). For the reference architecture with one hidden layer, $a_k = \sum_j w_{kj}^{(2)} h_j$ with $h_j = h(\sum_i w_{ji}^{(1)} x_i)$.

Summary (95% of the cases).

Problem	Output activation
Regression	linear
Binary classification	sigmoid (or tanh with ± 1 coding)
Multi-class classification	softmax

A binary problem can also be treated as a two-class problem with one-hot encoding and softmax; this is common, and only doubles the (small) number of weights of the output layer. Multiple independent sigmoids are used instead for *multi-label* classification, discussed in later chapters.

Hidden layers. The input layer is the identity. For hidden neurons, sigmoid and tanh are equally good, and the network is equally powerful with either; since there is no reason to mix them, one usually picks one and uses it everywhere. Nowadays one actually uses the *rectified linear unit* (ReLU) in hidden layers, for reasons explained in Chapter 6; for now we stick to sigmoid and tanh.

Remark 3.2 (Is this deep learning?) *A feedforward network approximates an input–output function. Whether we are doing deep learning depends on* how *it is used. If the inputs are engineered features and the output is the class, we are not doing deep learning, regardless of the depth of the network: in 1985 nobody was doing deep learning with these networks. If the inputs are the raw data (e.g., all the pixels of an image)*

and the network is trained end-to-end, then cutting the network at some hidden layer reveals a learned transformation of the input that the subsequent layers use as features: an intermediate, learned representation. In this sense deep learning is not a property of the network nor of its depth, but of training end-to-end and learning intermediate representations. People did try to feed pixels into fully connected networks in the 1990s, and it did not work well; the reasons, and the fix (convolutional networks), will become clear later. For now, think of the network as a universal function approximator.

3.2 Universal approximation

Theorem 3.1 (Universal approximation, Hornik 1991) *A single hidden layer feedforward neural network with S-shaped activation functions (such as sigmoid or* tanh*) can approximate any measurable function to any desired degree of accuracy on a compact set, provided that a sufficient number of hidden neurons is used.*

A linear regression approximates linear functions; a feedforward network approximates (essentially) any non-linear continuous function on a compact domain, with just *one* hidden layer. This is powerful, but some caveats are in order:

- the theorem states that a set of weights achieving the approximation *exists*, not that we are able to *find* it;
- an arbitrary accuracy may require an exponential number of hidden neurons (in the dimension of the domain), hence an exponential number of parameters, which are hard to find and set;
- a model with a huge number of parameters can learn the training set by heart and fail to generalize: *overfitting* is behind the corner.

The theorem justifies working with this model, but leaves two problems: *finding the weights* (this chapter) and *avoiding overfitting* (Chapter 5).

Remark 3.3 *Classification functions are not continuous, so the theorem above applies to regression. For classification, other results show that* two *hidden layers are sufficient. For this reason our examples use networks with one or two hidden layers. In practice deeper networks often make it easier to find good weights, but some tricks are needed to train them, which we will see later.*

3.3 Learning as optimization

Consider a model $y(\mathbf{x} \mid \boldsymbol{\theta})$ with input $\mathbf{x}$ and parameters $\boldsymbol{\theta}$: e.g., in linear regression a linear combination of the inputs with coefficients $\boldsymbol{\theta}$; in logistic regression, the logistic function of a linear combination. Given a training set $\mathcal{D} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_N, t_N)\}$, in most supervised problems *learning* means finding the parameters such that, for each input, the model output is *similar* to the desired target. For us the model is a neural network and the parameters are its weights:

$$g(\mathbf{x}_n \mid \mathbf{w}) \approx t_n \qquad \forall n. \tag{3.5}$$

How do we measure "similar"? For instance, by the sum of squared errors, as in *least squares regression*:

$$E(\mathbf{w}) = \sum_{n=1}^{N} \big(t_n - g(\mathbf{x}_n \mid \mathbf{w})\big)^2. \tag{3.6}$$

Taking the square prevents positive and negative errors from compensating each other; the absolute value would also work (L1 norm), but the squared (L2) error is more convenient. Chapter 4 shows deeper reasons behind this choice. Error functions are nowadays called **loss functions**.

Example 3.2 (Advertising) *A classic example from* An Introduction to Statistical Learning *considers the sales of a product as a function of the budget spent on TV advertising. A linear model fits a line minimizing the squared vertical distances (the residuals) between the points and the line. The only difference with a neural network is that the fitted curve is non-linear.*

Learning becomes the minimization of a loss function with respect to the parameters of the network. Since the network is non-linear in its parameters, this is a *non-linear optimization* problem rather than a quadratic one.

3.3.1 Gradient descent

To find the minimum of a function we look for stationary points, where the derivative is zero: not every stationary point is a minimum, but every minimum is a stationary point. For a neural network the equations $\partial E/\partial \mathbf{w} = 0$ generally have no closed-form solution. We thus resort to an iterative method.

Definition 3.2 (Gradient descent) *Starting from a (random) initial guess* $\mathbf{w}^0$*, iterate*

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \left.\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}\right|_{\mathbf{w}^k}, \tag{3.7}$$

where η *is the learning rate.*

If the derivative is positive (the function grows), we step backward; if negative, we step forward; the step is η times the derivative. Think of standing on a mountain and wanting to reach the bottom of the valley: if the ground in front of you goes up, turn and step backward; if it goes down, step forward. Only the local value of the derivative is needed (and with sigmoid and tanh it is easily computed from the function value).

For a linear model the squared error is a quadratic bowl and gradient descent is not even needed. For a neural network the error surface has a complex shape, but the algorithm is the same. Being greedy, it depends on the starting point, and the error surface of a non-linear model has many stationary points, including many local minima: starting from a different point, we may end up in a different (possibly much worse) local minimum.

Momentum. A rock rolling down a mountain does not stop in a small dip, thanks to its inertia. Similarly, we can add a memory of the previous update:

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \left.\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}\right|_{\mathbf{w}^k} - \alpha \left.\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}\right|_{\mathbf{w}^{k-1}}, \tag{3.8}$$

where α weighs the step taken in the direction of the previous gradient: if the gradient becomes zero, we keep moving a little.

Rprop. *Resilient propagation* drops the magnitude of the gradient: the step has a fixed size, and only the *sign* of the gradient is used to decide the direction.

These are just some variations of non-linear optimization methods; Section 6.5 discusses modern optimizers.

Local minima in practice. The classical remedy was to restart training many times from different initializations and keep the best solution. Today this is rarely done: large models may require days, weeks or months to train (large language models are trained for months on thousands of GPUs), and multiple restarts would multiply the cost. Moreover, theoretical and empirical results suggest that the larger the model, the more numerous the minima, but the more they are *equally good*. So one usually trains once from a good initialization (with safeguards, e.g., stopping when gradients misbehave). For small models, multiple restarts can still help.

3.3.2 Backpropagation is gradient descent

When backpropagation was introduced, it was derived through a complex formula propagating the contribution of the output error back to the inputs through linear approximations of each layer. What had been (re)discovered is simply gradient descent — known since the times of Newton and Gauss — *applied to a neural network*: start from random weights, compute the gradient of the error with respect to *every* weight, and update *all* the weights, in all layers at the same time, in the direction opposite to the gradient. This is possible because the network is a feedforward composition of differentiable functions.

3.3.3 An explicit gradient computation

Consider a network with I inputs, one hidden layer of J neurons with activation $h(\cdot)$, and one output with activation $g(\cdot)$:

$$g_1(\mathbf{x}_n \mid \mathbf{w}) = g\left(\sum_{j=0}^{J} w_{1j}^{(2)}\, h_j\Big(\sum_{i=0}^{I} w_{ji}^{(1)}\, x_{i,n}\Big)\right). \tag{3.9}$$

Each output is a non-linear function of a weighted sum of non-linear functions of weighted sums of the inputs: with more layers we obtain nested functions. With the squared error $E(\mathbf{w}) = \sum_n \big(t_n - g_1(\mathbf{x}_n \mid \mathbf{w})\big)^2$, let us compute the derivative with respect to $w_{35}^{(1)}$, the weight from the 5th input to the 3rd hidden neuron.

The derivative of a sum is the sum of the derivatives, and the derivative of a square is twice the argument times the derivative of the argument:

$$\frac{\partial E}{\partial w_{35}^{(1)}} = \sum_n 2\big(t_n - g_1(\mathbf{x}_n \mid \mathbf{w})\big)\left(-\frac{\partial g_1(\mathbf{x}_n \mid \mathbf{w})}{\partial w_{35}^{(1)}}\right), \tag{3.10}$$

since t_n does not depend on the weights. Next,

$$\frac{\partial g_1(\mathbf{x}_n \mid \mathbf{w})}{\partial w_{35}^{(1)}} = g'\Big(\sum_j w_{1j}^{(2)} h_j(\cdot)\Big) \cdot \frac{\partial}{\partial w_{35}^{(1)}} \sum_{j=0}^{J} w_{1j}^{(2)}\, h_j(\cdot). \tag{3.11}$$

In the sum $w_{10}^{(2)}h_0 + w_{11}^{(2)}h_1 + w_{12}^{(2)}h_2 + w_{13}^{(2)}h_3 + \cdots$, the bias h_0 does not depend on $w_{35}^{(1)}$, nor does h_1, h_2, h_4, …: only the third hidden neuron receives that weight. Hence only one term survives:

$$\frac{\partial}{\partial w_{35}^{(1)}} \sum_j w_{1j}^{(2)}\, h_j(\cdot) = w_{13}^{(2)}\, h'\Big(\sum_i w_{3i}^{(1)} x_{i,n}\Big) \cdot \frac{\partial}{\partial w_{35}^{(1)}} \sum_i w_{3i}^{(1)} x_{i,n} = w_{13}^{(2)}\, h'(\cdot)\, x_{5,n}, \tag{3.12}$$

because in $w_{30}^{(1)}x_0 + w_{31}^{(1)}x_1 + \cdots + w_{35}^{(1)}x_5 + \cdots$ only one term depends on $w_{35}^{(1)}$. Putting everything together:

$$\boxed{\frac{\partial E(\mathbf{w})}{\partial w_{35}^{(1)}} = -2\sum_{n=1}^{N}\big(t_n - g_1(\mathbf{x}_n \mid \mathbf{w})\big)\; g'\Big(\sum_j w_{1j}^{(2)} h_j(\cdot)\Big)\; w_{13}^{(2)}\; h'\Big(\sum_i w_{3i}^{(1)} x_{i,n}\Big)\; x_{5,n}.} \tag{3.13}$$

If, e.g., g and h are sigmoids, g' and h' are computed as $\sigma(1-\sigma)$. Analogous formulas hold for all the other weights; at each step all weights are updated.

3.3.4 Batch, stochastic and mini-batch gradient descent

Formula (3.13) is a *sum over all samples*: the gradient is computed on the whole dataset before taking one step. This is **batch gradient descent**: one iteration per epoch (compare with online learning in Chapter 2, where each sample produced an update, giving N iterations per epoch).

Training is performed on GPUs, which compute all these gradients and update all the weights in parallel. To do so, *both the model and the data must be in GPU memory*: with a terabyte of data one would need a terabyte of GPU memory, which is infeasible. Three options exist:

- **Batch gradient descent**: the gradient is computed on all N samples,

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N}\sum_{n=1}^{N}\frac{\partial E(\mathbf{x}_n, \mathbf{w})}{\partial \mathbf{w}}; \tag{3.14}$$

  ideal, but often not feasible.

- **Stochastic gradient descent (SGD)**: the gradient is approximated with one sample at a time,

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} \approx \frac{\partial E_{\mathrm{SGD}}(\mathbf{w})}{\partial \mathbf{w}} = \frac{\partial E(\mathbf{x}_n, \mathbf{w})}{\partial \mathbf{w}}; \tag{3.15}$$

  always feasible, unbiased, but with very high variance: one sample pulls in one direction, the next in another, and the trajectory is erratic. On average it converges to the same solution, and the noise can even help escape local minima.

- **Mini-batch gradient descent**: the gradient is computed on a subset (mini-batch) $\mathcal{B}$ of samples,

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} \approx \frac{1}{|\mathcal{B}|}\sum_{n\in\mathcal{B}}\frac{\partial E(\mathbf{x}_n, \mathbf{w})}{\partial \mathbf{w}}; \tag{3.16}$$

  a good trade-off between variance and computation, and the standard choice nowadays. The size $|\mathcal{B}|$ is the **batch size**, a hyperparameter set in every training script, bounded by the memory of the GPU.

The number of iterations per epoch is therefore N divided by the batch size: 1 for batch gradient descent, N for SGD (batch size one), $N/|\mathcal{B}|$ for mini-batches (e.g., with 32 samples per batch). Every load–update cycle has a cost, so it is convenient to use the largest batch that fits in memory.

Gradient accumulation. The full-batch gradient can be simulated even when the data do not fit in memory: one asks the library not to *apply* the gradient after each batch but to *accumulate* it; after all the batches have been processed, the accumulated gradient is applied once. In PyTorch this is particularly simple (several forward–backward passes, followed by a single optimizer step).

3.4 Automatic differentiation and the forward–backward pass

Deriving formulas like (3.13) is a standard but boring calculus exercise, and every time the architecture changes (a new layer, a new activation function) the derivatives must be recomputed. Who computes all these derivatives? This can be done *automatically*, once the structure of the network is known.

3.4.1 The chain rule

Recall the chain rule for composite functions. Let $x \in \mathbb{R}$ and $z = f(g(x))$. Defining $y = g(x)$, so that $z = f(y)$,

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx} = f'(y)\,g'(x) = f'(g(x))\,g'(x). \tag{3.17}$$

For instance, $\frac{d}{dx}\log(x^2) = \frac{1}{x^2}\cdot 2x$. With three nested functions $z = f(g(q(x)))$,

$$\frac{dz}{dx} = \frac{dz}{dg(q(x))}\,\frac{dg(q(x))}{dq(x)}\,\frac{dq(x)}{dx}. \tag{3.18}$$

A neural network is precisely a function of a function of a function… and (3.13) is nothing but the chain rule: derivative of the error with respect to the output of the network, times the derivative of the output with respect to its argument, times the derivative of the argument with respect to the hidden output (the weight $w_{13}^{(2)}$), times the derivative of the hidden output with respect to its argument, times the derivative of the argument with respect to the weight (the input x_5):

$$\frac{\partial E}{\partial w_{ji}^{(1)}} = \underbrace{-2\sum_{n}(t_n - g_1(\mathbf{x}_n \mid \mathbf{w}))}_{\partial E/\partial g}\cdot \underbrace{g'(\cdot)}_{\partial g/\partial a}\cdot \underbrace{w_{1j}^{(2)}}_{\partial a/\partial h_j}\cdot \underbrace{h_j'(\cdot)}_{\partial h_j/\partial a_j}\cdot \underbrace{x_i}_{\partial a_j/\partial w_{ji}^{(1)}}. \tag{3.19}$$

3.4.2 Forward pass and backward pass

Every factor in the product above can be obtained by *feeding the input through the network*:

- the inputs x_i are known;
- computing each neuron's activation value a gives its output and also the value of its derivative at that point, $h'(a)$ or $g'(a)$;
- the weights are known;
- at the output we obtain the network output, hence the error and the derivative of the error.

Then, to obtain the derivative with respect to any weight, we multiply the stored quantities along the path that goes *backward* from the error to that weight. For a weight of the last layer we multiply the derivative of the error, the derivative of the output node, and the input of that weight (the output of the hidden neuron); for a weight of the first layer, the path also includes the weight $w^{(2)}_{1j}$ and the derivative of the hidden node, ending with the input x_i. When a weight influences the output through several paths (e.g., with multiple outputs or layers), the contributions of all paths are summed.

Definition 3.3 (Forward–backward pass) ***Forward pass**: propagate the input through the network, computing and storing, for each node, its output and the value of its derivative at the current input. **Backward pass**: starting from the derivative of the loss, propagate backward multiplying the stored derivatives and weights to obtain the gradient with respect to every weight, and update the weights.*

The quantities to store are the weights, the outputs of the activation functions, and the values of their derivatives. For sigmoid and tanh storing the output is sufficient, since their derivative is a function of the output.

The approach is modular:

- *adding a node or a layer*: store its derivative in the forward pass and include it in the backward products;
- *a new activation function*: only its derivative changes;
- *a new loss function*: only the first factor $\partial E/\partial g$ changes; everything downstream is unchanged (e.g., using the logarithm of the error just changes the derivative of the first term);
- any node can be plugged in as long as we know how to compute its *output* and its *derivative* — it could even be a program (techniques exist to differentiate programs, a topic not covered here).

This is **automatic differentiation**, and it is why deep learning libraries make building networks so simple: each layer and activation knows how to compute its forward and backward pass. Moreover, all samples can be processed in parallel and gradients accumulated, and the weights are updated in place in GPU memory, making the forward–backward cycle very efficient.

Remark 3.4 (PyTorch) *For inference only the forward pass is needed. For training, the backward pass is also required. Keras'* `fit` *performs both together; in PyTorch the training loop is written explicitly:* if the backward pass is forgotten, the network does not train. *On the other hand, this makes it easy to perform several forward passes, accumulate gradients, and only then update (gradient accumulation).*

3.5 Laboratory: a first feedforward network in PyTorch

This section implements a feedforward network for classification on the Iris dataset. The labs run in Jupyter notebooks (e.g., on Google Colab): a notebook is a sequence of cells, each executing a piece of Python code on a shared global state, so that data loading, model definition, training and evaluation can be run and re-run separately. Connecting the notebook to Google Drive makes the saved files (models, logs) persistent.

3.5.1 Setup

```
import os, random, warnings
import numpy as np
import torch
import torch.nn as nn
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, confusion_matrix

SEED = 42                       # reproducibility
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

The networks of this first lab are small enough to be trained on a CPU; later, GPUs become essentially mandatory, and the `device` variable allows the same code to run on both.

3.5.2 Data exploration

The Iris dataset has 150 rows and 5 columns: four numerical features (sepal length, sepal width, petal length, petal width) and the class (three species, 50 samples each, encoded as $0, 1, 2$). The `shape` of a table is (rows, columns); similarly, tensors in neural network training usually have the batch size as first dimension.

Exploring the data is one of the most important steps of any machine learning pipeline: check the variables, their statistics (means, standard deviations, percentiles), missing values, class balance. Iris is "too clean" (no missing values, perfectly balanced); real-world datasets are not. Some observations:

- the feature means are quite different (e.g., sepal length is much larger than petal width), which must be taken into account;
- the integer class encoding implies an ordering that does not exist among species: classes should be treated categorically (one-hot or, equivalently, via a classification loss);
- from the pair plots, one class is *linearly separable* from the other two using just a pair of features, and the other two are almost separable. Any reasonable model must at least identify that class perfectly: an accuracy lower than about 33% would reveal a bug.

3.5.3 Train, validation and test splits

```
X = iris.drop(columns="target").values
y = iris["target"].values

X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=20, random_state=SEED, stratify=y)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=20, random_state=SEED,
    stratify=y_train_val)
```

Stratification keeps the class proportions in each split, which is important when the splits are small.

Why three splits?

- The **training set** is used to fit the parameters (weights).

- We care about *out-of-sample* performance: a model evaluated on the data it was trained on may have learned them by heart (overfitting).
- Hyperparameters (number of layers, learning rate, early stopping epoch, ...) must be chosen too: this is like a nested loop, where the outer loop iterates over configurations evaluated on the **validation set**, and the inner loop trains each network on the training set. During validation the parameters are frozen: the network is used in inference mode.
- Since the best configuration is selected on validation data, the validation performance is optimistic. The **test set**, never seen before, is used once to certify the final performance (e.g., what a model provider would declare to a client).

Setting data aside is a cost, especially for small datasets, but it is unavoidable. A common practice is to select the hyperparameters with the validation set and then retrain on training+validation with the selected hyperparameters before the final test. With only 20 test samples, a single error changes the accuracy by 5%: the split proportions must be chosen according to the application.

3.5.4 Normalization, tensors and data loaders

If features have very different scales (say means of 1000 and 0.1), with randomly initialized weights the large features dominate both the forward pass and the gradients, and small features may be effectively cut off from training. Hence we normalize, here with min–max scaling fitted on the training set:

```
scaler = MinMaxScaler().fit(X_train)
X_train, X_val, X_test = (scaler.transform(a) for a in (X_train, X_val, X_test))

from torch.utils.data import TensorDataset, DataLoader
def to_ds(X, y):
    return TensorDataset(torch.from_numpy(X).float(), torch.from_numpy(y).long())
train_ds, val_ds, test_ds = to_ds(X_train, y_train), to_ds(X_val, y_val), to_ds(X_test, y_test)

BATCH_SIZE = 16
def make_loader(ds, batch_size, shuffle, drop_last):
    return DataLoader(ds, batch_size=batch_size, shuffle=shuffle,
                      drop_last=drop_last, num_workers=2, pin_memory=True)
train_loader = make_loader(train_ds, BATCH_SIZE, shuffle=True,  drop_last=True)
val_loader   = make_loader(val_ds,   BATCH_SIZE, shuffle=False, drop_last=False)
test_loader  = make_loader(test_ds,  BATCH_SIZE, shuffle=False, drop_last=False)
```

`torch.from_numpy` converts NumPy arrays into PyTorch tensors, on which PyTorch operations are tracked so that gradients can be computed and backpropagated. A `Dataset` is a (fixed) collection of data, in RAM or on disk; a `DataLoader` wraps it and, during training, fetches batches in parallel threads, so that even datasets that do not fit in memory can be streamed from disk. Training batches are *shuffled* at every epoch (one practically never sees the same batch twice, which benefits gradient descent), and the last incomplete batch is dropped so that all batches have the same size. Validation and test loaders are not shuffled and do not drop data, so metrics are always computed on the same samples. Iterating over a loader yields tuples (features, labels) of shapes $(16, 4)$ and $(16,)$.

3.5.5 The model

```
class FeedForwardNet(nn.Module):
    def __init__(self, in_features=4, hidden_size=32, num_classes=3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features, hidden_size),   # hidden layer
            nn.ReLU(),                             # non-linearity
            nn.Linear(hidden_size, num_classes))   # output layer: logits
```

```
    def forward(self, x):
        return self.net(x)

model = FeedForwardNet().to(device)
```

A network is defined by extending `nn.Module`: the constructor declares the layers, and `forward` specifies the order in which data flow through them (here a `Sequential` stack, i.e., layers one after the other without loops; one could equivalently store each layer in its own attribute and call them explicitly in `forward`). Without the non-linearity, the model could only solve linear problems. The hidden size (32) is arbitrary.

The output layer has three neurons, one per class. Its outputs are called **logits**: they are *unnormalized* scores, which become class probabilities summing to one after a softmax; in PyTorch the softmax is included in the cross-entropy loss, which is why the model returns logits.

Parameter count. The hidden layer has $4 \times 32 + 32 = 160$ parameters, the output layer $32 \times 3 + 3 = 99$, for a total of 259. Being able to count parameters, remembering the biases, is a skill that will be needed often.

TensorBoard. TensorBoard logs and visualizes metadata during training (graph of the model, losses, metrics, histograms of weights and gradients); passing a dummy input of the right shape allows it to trace and display the computational graph of the network.

3.5.6 Training hyperparameters

- **Batch size**: the number of samples whose errors are averaged to produce one update. With batch size one the network is updated after each sample; with the whole dataset (if memory allows) once per epoch. Neither too small nor too large: 16 is enough here.
- **Number of epochs**: how many times the model sees the whole dataset. Just as a student rarely memorizes a book in one reading, a model needs repetitions (*repetita iuvant*); since data are limited, they are reused. Here 500.
- **Learning rate**: the factor multiplying the gradient in the update; one of the most important hyperparameters, able to make everything work or to break everything. With a huge learning rate (say 500 instead of 10^{-3}) the updates are dominated by noise and training becomes unstable, bouncing around the parameter space; with a tiny one (say 10^{-20}) virtually no information propagates and training would take forever. Default values are a good start; Chapter 5 and the following labs discuss how to choose it.

```
EPOCHS, LR = 500, 1e-3
criterion = nn.CrossEntropyLoss()                  # classification loss
optimizer = torch.optim.Adam(model.parameters(), lr=LR)
scaler_amp = torch.cuda.amp.GradScaler(enabled=(device.type == "cuda"))
```

The cross-entropy loss is the standard loss for classification (Chapter 4). *Adam* is an advanced version of gradient descent that uses momentum and adaptive step sizes (Section 6.5). The optional *mixed precision* performs some operations in 16-bit instead of 32-bit floating point, which does not affect the metrics but improves efficiency.

3.5.7 The training loop

Each epoch has two phases: *learning* on the training set and *evaluation* on the validation set. The evaluation phase should never be skipped.

```
def train_one_epoch(model, loader, criterion, optimizer, scaler_amp, device):
    model.train()                          # training mode: weights can change
    running_loss, preds, targets = 0.0, [], []
```

```
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()               # reset accumulated gradients
        with torch.autocast(device_type=device.type, enabled=(device.type == "cuda")):
            logits = model(xb)              # forward pass
            loss = criterion(logits, yb)
        scaler_amp.scale(loss).backward()   # backward pass
        scaler_amp.step(optimizer)          # weight update
        scaler_amp.update()
        running_loss += loss.item() * xb.size(0)
        preds.append(logits.argmax(dim=1).cpu()); targets.append(yb.cpu())
    preds, targets = torch.cat(preds), torch.cat(targets)
    return running_loss / len(loader.dataset), accuracy_score(targets, preds)

@torch.no_grad()                            # no gradients during evaluation
def validate_one_epoch(model, loader, criterion, device):
    model.eval()                            # evaluation (inference) mode
    running_loss, preds, targets = 0.0, [], []
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        running_loss += criterion(logits, yb).item() * xb.size(0)
        preds.append(logits.argmax(dim=1).cpu()); targets.append(yb.cpu())
    preds, targets = torch.cat(preds), torch.cat(targets)
    return running_loss / len(loader.dataset), accuracy_score(targets, preds)
```

For each batch: move data to the device, reset the gradients, compute the logits (forward), compute the loss, backpropagate (backward) and update the weights. The batch loss (an average) is multiplied by the batch size to accumulate the total error of the epoch. Predictions are obtained with `argmax`, which selects the class with the highest score (equivalently, it turns the score vector into a one-hot vector with a one at the maximum).

Why compute a *metric* such as accuracy besides the loss? The loss is (often) lower-bounded by zero, but its numerical value is hard to interpret: whether a cross-entropy of 0.001 is good depends on the problem. A bounded metric such as accuracy (the fraction of correct answers) is human-interpretable.

```
for epoch in range(1, EPOCHS + 1):
    tr_loss, tr_acc = train_one_epoch(model, train_loader, criterion, optimizer, scaler_amp,
    device)
    va_loss, va_acc = validate_one_epoch(model, val_loader, criterion, device)
    # optional: log losses, metrics, weight and gradient histograms to TensorBoard
    if epoch % 10 == 0:
        print(f"{epoch:4d} | train {tr_loss:.4f} {tr_acc:.3f} | val {va_loss:.4f} {va_acc:.3f}")
torch.save(model.state_dict(), "models/iris_ffnn.pt")
```

Save your models. Always save the trained model, using a name that depends on the date/time or on the performance, to avoid overwriting previous ones. Experiments run with hardware acceleration are generally not exactly reproducible (a trade-off between speed and determinism): a good model may never be obtained again.

3.5.8 Results

The training and validation loss curves decrease smoothly and coherently: a good learning curve. The validation accuracy reaches 100% well before epoch 500, suggesting that fewer epochs would suffice; how to choose the number of epochs automatically (early stopping) is discussed in Chapter 5. TensorBoard also shows the histograms of weights, biases and gradients over time: gradients start centred around zero and remain stable; the biases of the output layer, initially concentrated around the same value, split into three

groups — one per class. Inspecting these plots, in particular for the output layer, is a useful debugging tool (try training with a learning rate of 5000 and look at the weights and gradients).

Finally, the saved model is reloaded and evaluated on the test set with the same loop used for validation: the test accuracy is 95% (one error out of 20), coherent with validation. The *confusion matrix* gives a more detailed view, from which other metrics can be derived, useful e.g. with unbalanced classes.

Remark 3.5 (Exercise: Palmer Penguins) *The* Penguins *dataset is very similar to Iris but contains* missing values*. One must decide how to handle them: if only a few records are affected, they can be discarded or the values imputed; if a column has many missing values, one can impute them or discard the whole column.*

Chapter 4

Loss Functions and Maximum Likelihood Estimation

In Chapter 3 we trained networks by minimizing the sum of squared errors, a choice that was presented as "reasonable". Why this function, and not another? And is it a good choice for every problem? In this chapter we answer these questions by going back to a basic tool of statistics, *maximum likelihood estimation*, and we show that the loss function is not an arbitrary choice: it encodes our assumptions on the data and, together with the output layer, it *defines the task* the network solves.

4.1 Maximum likelihood estimation

4.1.1 An intuitive example

Suppose we take repeated measurements of the distance of a wall with a laser range finder. The instrument is affected by Gaussian noise with known variance σ^2 (its declared accuracy), so we obtain different values $x_1, \dots, x_N$ at each measurement, scattered around an unknown mean μ — the quantity we want to estimate, the expected distance of the wall:

$$p(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\,\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}. \tag{4.1}$$

Imagine we are offered several candidate Gaussians, all with the same variance but different means, one of which is centred where the samples are dense, while the others are shifted far away. Everybody picks the one centred on the samples. Why? Reason from the opposite side: if the true distribution were one of the shifted ones, some of the observed points would have been almost impossible to observe. Many points have been observed in a region, so that region should be very likely under the model: *the model should put a lot of probability where the points are*. The chosen Gaussian is the one that makes the observed data most likely.

Definition 4.1 (Maximum likelihood estimation) *Maximum likelihood estimation (MLE) chooses the parameters of a model that* maximize the probability of the observed data*.*

Remark 4.1 *It is very important to notice what MLE does* not *do: it does not choose the most likely parameters, i.e., it does not maximize the probability of the parameters. It chooses the parameters that maximize the probability of the* data*. The data are the only thing we have, and they tell us where the distribution is. (The alternative, Bayesian, viewpoint is discussed in Chapter 5.)*

4.1.2 The recipe

Given a distribution with parameters $\boldsymbol{\theta} = (\theta_1, \theta_2, \dots, \theta_p)$ and a dataset $\mathcal{D}$:

1. write the **likelihood** $L(\boldsymbol{\theta}) = p(\mathcal{D} \mid \boldsymbol{\theta})$ of the data as a function of the parameters;
2. (optionally) take the **log-likelihood** $\ell(\boldsymbol{\theta}) = \log L(\boldsymbol{\theta})$;

3. compute the derivatives $\partial \ell / \partial \boldsymbol{\theta}$ and set them to zero;
4. solve the resulting (possibly non-linear) system of equations, analytically or numerically (e.g., with gradient descent); check that the stationary point is a maximum (stationary points include minima too, but usually it is a maximum).

Why the logarithm?

- The logarithm is monotone, so the maximum of $\log L$ is at the same point as the maximum of L.
- It turns products into sums, which are much easier to differentiate.
- It avoids numerical underflow: the likelihood of a dataset is often a product of thousands of probabilities, each in $[0, 1]$. Multiplying 1000 values equal to 0.1 gives 10^{-1000}, far below the smallest positive number representable in double precision (around 10^{-308}): on a computer the product becomes zero. Summing logarithms has no such problem.

4.1.3 MLE of the mean of a Gaussian

Let $x_1, \ldots, x_N$ be **i.i.d.** — *independent and identically distributed* — samples from $\mathcal{N}(\mu, \sigma^2)$ with σ^2 known. Independence means that the joint probability of the dataset is the product of the probabilities of the samples; identical distribution means that each sample has the same distribution. Thus

$$L(\mu) = p(x_1, \ldots, x_N \mid \mu, \sigma^2) = \prod_{n=1}^{N} p(x_n \mid \mu, \sigma^2) = \prod_{n=1}^{N} \frac{1}{\sqrt{2\pi}\,\sigma} e^{-\frac{(x_n-\mu)^2}{2\sigma^2}}. \tag{4.2}$$

(If σ^2 were unknown too, we would differentiate also with respect to σ: two equations in two unknowns. Here we keep it as a constant.) The log-likelihood is

$$\ell(\mu) = \sum_{n=1}^{N} \log \left(\frac{1}{\sqrt{2\pi}\,\sigma} e^{-\frac{(x_n-\mu)^2}{2\sigma^2}} \right) = N \log \frac{1}{\sqrt{2\pi}\,\sigma} - \frac{1}{2\sigma^2} \sum_{n=1}^{N} (x_n - \mu)^2. \tag{4.3}$$

The first term does not depend on μ; differentiating the second (the derivative of $(x_n-\mu)^2$ is $2(x_n-\mu)\cdot(-1)$):

$$\frac{\partial \ell(\mu)}{\partial \mu} = \frac{1}{2\sigma^2} \sum_{n=1}^{N} 2(x_n - \mu) = 0. \tag{4.4}$$

Since $\sigma^2 > 0$ we can multiply both sides by $2\sigma^2$ (and drop the factor 2):

$$\sum_{n=1}^{N} (x_n - \mu) = 0 \implies \sum_{n=1}^{N} x_n = N\mu \implies \hat{\mu}_{\text{MLE}} = \frac{1}{N} \sum_{n=1}^{N} x_n. \tag{4.5}$$

The result is the familiar *sample mean*. What is worth remembering is that it is not an empirical recipe: it is the solution of a maximum likelihood problem. Now we apply exactly the same recipe to neural networks.

4.2 Regression: the sum of squared errors

By the universal approximation theorem, a neural network can represent essentially any function; so we may assume that *there exists* an ideal set of weights that generated our data. However, observations are noisy. We assume:

$$t_n = g(\mathbf{x}_n \mid \mathbf{w}) + \epsilon_n, \qquad \epsilon_n \sim \mathcal{N}(0, \sigma^2), \tag{4.6}$$

i.e., each target is the value of an unknown deterministic function (which the network can learn) corrupted by additive Gaussian noise with a fixed variance. Every observed point actually comes from the curve, plus some Gaussian noise, which is why the data look scattered and we do not see the clean curve.

Recall the following chain of facts:

- if $\epsilon \sim \mathcal{N}(0,1)$, then $x = 1 + \epsilon \sim \mathcal{N}(1,1)$;
- if $x = a + \epsilon$, then $x \sim \mathcal{N}(a,1)$;
- if $x = \boldsymbol{\beta}^\top \mathbf{x} + \epsilon$, then $x \sim \mathcal{N}(\boldsymbol{\beta}^\top \mathbf{x}, 1)$ — the assumption behind linear regression;
- if $t = g(\mathbf{x} \mid \mathbf{w}) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$, then

$$t \sim \mathcal{N}\big(g(\mathbf{x} \mid \mathbf{w}), \sigma^2\big). \tag{4.7}$$

The network, at each point $\mathbf{x}$, predicts the *mean* of the Gaussian from which the target is sampled. Training becomes the estimation of the mean of a Gaussian: same problem, same tool.

Likelihood. With i.i.d. samples,

$$L(\mathbf{w}) = p(t_1, \dots, t_N \mid g(\cdot \mid \mathbf{w}), \sigma^2) = \prod_{n=1}^{N} \frac{1}{\sqrt{2\pi}\,\sigma} e^{-\frac{(t_n - g(\mathbf{x}_n \mid \mathbf{w}))^2}{2\sigma^2}}. \tag{4.8}$$

Log-likelihood.

$$\ell(\mathbf{w}) = \sum_{n=1}^{N} \log \frac{1}{\sqrt{2\pi}\,\sigma} - \frac{1}{2\sigma^2} \sum_{n=1}^{N} \big(t_n - g(\mathbf{x}_n \mid \mathbf{w})\big)^2. \tag{4.9}$$

Maximization. The first term and the factor $\frac{1}{2\sigma^2}$ do not depend on $\mathbf{w}$; maximizing $-(\cdot)$ is minimizing $(\cdot)$:

$$\arg\max_{\mathbf{w}} \ell(\mathbf{w}) = \arg\max_{\mathbf{w}} \sum_{n} -\frac{1}{2\sigma^2} \big(t_n - g(\mathbf{x}_n \mid \mathbf{w})\big)^2 = \arg\min_{\mathbf{w}} \sum_{n=1}^{N} \big(t_n - g(\mathbf{x}_n \mid \mathbf{w})\big)^2. \tag{4.10}$$

This is exactly the sum of squared errors.

4.2.1 Why the derivation matters

The derivation gives a result we already used, but it is useful for (at least) three reasons.

1. We know the hypotheses behind the loss. Minimizing the squared error amounts to assuming that *the data come from the model, corrupted by additive Gaussian noise with constant variance.* Constant variance is called **homoscedasticity**: the variance of the noise does not depend on the mean, nor is related to it in any way. If the data are **heteroscedastic**, the squared error is not the best loss.

Example 4.1 (Heteroscedastic data) *Consider data whose spread around the underlying curve grows with the value of the function: tiny noise where the function is small, large noise where it is large. The squared error treats all residuals equally, so the points with large noise dominate: a residual of* 10 *"pulls" the curve with a squared error of* 100 *times that of a residual of* 1*. The fitted curve is unfairly pulled by the high-variance region and the residuals are no longer symmetric around it.*

2. If we know something about the noise, we can fix the problem. The most common approach is to ignore the problem: if there is no evident relation between the variance and the function, we keep the assumption. If we know (e.g., by plotting the data or the residuals) that the noise grows with the function, we can apply a transformation. For example, if the noise is *multiplicative*,

$$y = g(\mathbf{x}) \cdot \epsilon \quad \Longrightarrow \quad \log y = \log g(\mathbf{x}) + \log \epsilon, \tag{4.11}$$

so by predicting $\log y$ instead of y (i.e., learning $g' = \log g$) the noise becomes *additive*. Nobody guarantees that $\log \epsilon$ is Gaussian with zero mean, but at least it is additive, and often reasonably Gaussian; after the transformation the squared error is appropriate. Such transformations are called **variance-stabilizing**

transformations: e.g., the logarithm when the standard deviation is proportional to the value of the function, the square root when the variance is proportional to it. These are patches, which work if the distortion is not too large. As in classical machine learning, one can (and should) perform *residual analysis* with neural networks too: plot the residuals, check whether they are Gaussian, zero-mean, and uncorrelated with the predicted value. In textbook exercises the noise variance is known; in real problems the noise is never exactly Gaussian, and one has to accept some assumptions.

3. If the "function plus noise" model does not hold, the squared error is not the right loss. This model makes sense for regression, but *not for classification*: an output in $\{0, 1\}$ cannot be "a number plus Gaussian noise". In binary classification an error is either predicting 0 when the target is 1 or vice versa: the errors take only two values and are certainly not Gaussian.

4.3 Classification: the cross-entropy

4.3.1 Binary classification

Let us model the data as they are. In binary classification the target is $t \in \{0, 1\}$, so its distribution is a *Bernoulli*. We want the network output $g(\mathbf{x} \mid \mathbf{w}) \in [0, 1]$ (sigmoid output) to be the *posterior probability* of class 1: if the network outputs 0.99, the target should be 1 with probability 0.99; if it outputs 0.1, the target should be 1 with probability 0.1. Therefore

$$t_n \sim \mathrm{Be}\big(g(\mathbf{x}_n \mid \mathbf{w})\big), \tag{4.12}$$

and training the network means estimating the *parameter of a Bernoulli* as the output of the network. The Bernoulli probability can be written compactly as

$$p(t \mid g(\mathbf{x} \mid \mathbf{w})) = g(\mathbf{x} \mid \mathbf{w})^t \cdot \big(1 - g(\mathbf{x} \mid \mathbf{w})\big)^{1-t}. \tag{4.13}$$

This writing is a trick: if $t = 1$, the second factor is raised to 0 and equals 1, so the probability is $g(\mathbf{x} \mid \mathbf{w})$; if $t = 0$, the first factor equals 1 and the probability is $1 - g(\mathbf{x} \mid \mathbf{w})$. (Think of a biased coin that gives head with probability 0.9: observing head has probability 0.9, observing tail 0.1.) Unlike an "if–then–else", this expression is *differentiable*, which we need to maximize the likelihood.

Likelihood and log-likelihood. With i.i.d. samples,

$$L(\mathbf{w}) = \prod_{n=1}^{N} g(\mathbf{x}_n \mid \mathbf{w})^{t_n} \big(1 - g(\mathbf{x}_n \mid \mathbf{w})\big)^{1-t_n}, \tag{4.14}$$

$$\ell(\mathbf{w}) = \sum_{n=1}^{N} t_n \log g(\mathbf{x}_n \mid \mathbf{w}) + (1 - t_n) \log\big(1 - g(\mathbf{x}_n \mid \mathbf{w})\big). \tag{4.15}$$

Maximization. The argmax of the log-likelihood is the argmin of its opposite:

$$\boxed{\arg\min_{\mathbf{w}} \; -\sum_{n=1}^{N} \Big[t_n \log g(\mathbf{x}_n \mid \mathbf{w}) + (1 - t_n) \log\big(1 - g(\mathbf{x}_n \mid \mathbf{w})\big) \Big].} \tag{4.16}$$

This loss is the **binary cross-entropy**. So, when moving from regression to classification, several things change: the output must be a sigmoid, not linear, and gradient descent must minimize the cross-entropy, not the squared error.

Remark 4.2 *In the 1990s, neural networks were trained for classification with the sum of squared errors. It "worked", in the sense that it produced numbers, but it was not the right thing to do. When the cross-entropy was proposed, the same models suddenly got better results on all classification problems. Today, using the squared error for classification is considered a blunder: the cross-entropy is the basic choice.*

4.3.2 Multi-class classification

Consider first a binary problem with *two* outputs and one-hot encoding: $(1, 0)$ for class 1 and $(0, 1)$ for class 0, with outputs $\big(g(\mathbf{x} \mid \mathbf{w}),\, 1 - g(\mathbf{x} \mid \mathbf{w})\big)$. Let $\mathbf{t}_n$ be the one-hot target and $\mathbf{g}(\mathbf{x}_n \mid \mathbf{w})$ the vector of outputs. Then

- if $\mathbf{t}_n = (1, 0)$: $\mathbf{t}_n^\top \log \mathbf{g} = 1 \cdot \log g + 0 \cdot \log(1 - g) = \log g$;
- if $\mathbf{t}_n = (0, 1)$: $\mathbf{t}_n^\top \log \mathbf{g} = \log(1 - g)$;

exactly the terms of the binary cross-entropy. Hence the same loss can be written as

$$\boxed{E(\mathbf{w}) = -\sum_{n=1}^{N} \mathbf{t}_n^\top \log \mathbf{g}(\mathbf{x}_n \mid \mathbf{w}) = -\sum_{n=1}^{N} \sum_{k=1}^{K} t_{n,k} \log g_k(\mathbf{x}_n \mid \mathbf{w}),} \tag{4.17}$$

which holds for any number K of classes, with softmax outputs. This is the **categorical cross-entropy**, the generalization of the binary cross-entropy to vector outputs with one-hot encoding (traditionally the two are given different names, but they are the same formula). With, e.g., 1000 classes, $\mathbf{t}_n$ is a vector of 1000 zeros except for a single one; for each sample the only non-zero term is the logarithm of the output corresponding to the true class, so the derivative of the loss with respect to the outputs is simply $-1/g_k$ for the true class k and zero elsewhere: very easy to compute.

4.4 The loss defines the task

Summarizing:

Task	Output layer	Loss
Regression	linear	sum of squared errors
Binary classification	sigmoid	binary cross-entropy
Multi-class classification	softmax	categorical cross-entropy

Many other loss functions exist — dozens: for classification, regression, image segmentation, metric learning, image generation... They can be designed in three ways:

1. by making *statistical assumptions* on the data distribution and applying maximum likelihood, as done here;
2. by exploiting *background knowledge* on the task and the model. For instance, it is possible to define an error function such that Hebbian learning of the perceptron can be interpreted as gradient descent on it. In object detection, the error measures how much the predicted bounding box overlaps with the true one, normalized by the size of the boxes: this is the *Intersection over Union* (IoU); since it is not well suited to differentiation, variants such as the *Dice loss* are used, as well as the *focal loss* (these appear in the computer vision chapters);
3. by *inventing* whatever is needed.

The fundamental message is that *the loss function is the way we describe the task to the network.* If the task is classification and we describe it as regression, the network optimizes a regression problem and the result is sub-optimal. If the noise is heteroscedastic and we tell the network it is homoscedastic, the result is sub-optimal. The output layer and the loss function together define the task: looking at the code of a network, its output layer and its loss tell which task the network was meant for; conversely, to solve a problem one must choose the right output configuration and the right loss.

Chapter 5

Generalization, Overfitting and Regularization

With the material of the previous chapters we have all the ingredients to train a neural network: a stack of layers trained with backpropagation on a task defined by a loss function (the sum of squared errors for regression, the cross-entropy for classification). The universal approximation theorem guarantees that, with enough neurons, there exists a set of weights approximating any function. However, this theorem

- does not say that backpropagation will find those weights (it may get stuck in local minima);
- does not say how big the network must be (possibly hundreds of thousands of neurons for arbitrary precision);
- most critically, does not say anything about the performance on data that have *not* been used for training.

Being able to approximate any dataset does not imply being able to predict on new data. This chapter deals with this problem, which is the central problem of machine learning; part of it will be familiar to readers who already studied machine learning, but it is essential to fix the concepts and the vocabulary. The chapter closes with two laboratory sections on regularization techniques and on robust evaluation and hyperparameter tuning.

5.1 Generalization and overfitting

The purpose of machine learning is to learn a model on some data and use it on *new* data. The ability to perform well on unseen data is called **generalization**. Poor generalization affects every machine learning algorithm, and the more powerful the algorithm, the more likely the problem. Neural networks, being very powerful non-linear approximators, are particularly exposed. This is related to **Occam's razor**: when two models have similar performance, the simplest one should be preferred, because complex models tend to *overfit*: they give very good results on the training data and very bad results on new data.

5.1.1 The inductive hypothesis

Machine learning works thanks to an *inductive hypothesis*: whatever is learned on a limited set of data will also work on new data. For instance, we assume that the data come from a linear function and that a line fitted on some data will work on new data as well; or we assume the classes are linearly separable and that a logistic regression fitted on the data will classify new data well. The simpler the hypothesis on the model, the more likely it carries over to new data; the more complex the hypothesis, the easier it is to overfit.

Example 5.1 (Polynomial fitting) *Let the data be generated by a second-degree polynomial M_2 plus noise, and observe a few points.*

- *If our inductive hypothesis is a line M_1, the fitted model has a high error even on the training points: the model is too simple to mimic the data. This is* ***underfitting****.*
- *Increasing the complexity reduces the training error: with N points a polynomial M_N of high enough degree interpolates the training set with almost zero error. Yet, in some regions it behaves even worse than the line: a new point generated by the true M_2 would be predicted horribly, while the line, though imprecise, never makes huge mistakes. This is* ***overfitting****: an overly complex model fits the training data perfectly but fails to generalize.*

5.1.2 Measuring generalization: the test set

Before solving the problem we must be able to *detect* it, i.e., to measure how bad it is. The idea is to evaluate the model on data that have *not* been used for training: since they differ from the training data, they give a more accurate estimate of the real performance. It is like an exam: the exercises are similar to those solved in class, but not the same ones, to check whether the method was learned rather than memorized.

Remark 5.1 *The worst mistake one can make in machine learning is to evaluate, or select, a model by looking at its performance on the training data.*

Where does the independent test set come from?

- Sometimes *someone provides it*: e.g., a customer gives us data to develop a model but keeps secret the data on which the model will be evaluated (as in data science challenges, where the test set is hidden).
- Otherwise we create it ourselves: we hide part of the data, perform all the training on the rest, and evaluate on the hidden part. This is called a **hold-out** set; training and test sets do not overlap.

(Techniques that resample with replacement, such as the bootstrap or the jackknife, are useful with very small datasets in classical statistics, but are of no practical use with neural networks.)

Typically 10–20% of the data are held out at random. In classification, however, the extraction should not be completely random: with unbalanced classes, random sampling might yield a test set dominated by one class, or with a rare class missing altogether, making the training and test distributions very different. One should preserve the class proportions in each split: this is called **stratified sampling**.

5.1.3 Training, validation and test sets

If we only had a training and a test set, how would we decide whether to add neurons or layers? If we train a small and a big network and compare them on the test set, it is too late: testing comes at the very end, when all decisions have been made; one cannot make decisions based on future data.

Example 5.2 (Investing in the stock market) *Suppose we develop a model to decide whether to invest all our money in a stock. The real test is putting the money on the market: before betting everything on a single shot, we would like a preview of whether we are going to lose all our money, i.e., to select the model on data it has never seen before deploying it.*

Therefore the available data are split into three sets:

- the **training set**, used to fit the weights;
- the **validation set**, used for *model selection*, i.e., hyperparameter tuning and model comparison: we train networks with $10, 20, 50, 100, 1000$ neurons on the training set and choose the one with the best validation performance;
- the **test set**, used only at the end to estimate the true performance of the selected model.

The validation error is itself an evaluation on unseen data, but it is *optimistic*: the selected model is biased by the very fact of having been chosen because it worked well on the validation set. The test error is unbiased. Usually the two are close: ideally the training error is the lowest, the validation error a bit higher, and the test error a bit higher still (with small datasets the test error can, by chance, be lower than the validation one).

When the test error is much worse. If a model performs very well on training and validation but horribly on the test set, there is a problem, and it is usually in the data. Everything relies on the assumption that the training data and the data on which the model will be used come from the *same distribution*. If they do not (e.g., the test set contains outliers or phenomena that never appear in the training set), the model will not work. The typical remedy is to go back: check and clean the data, make sure the training set covers all the phenomena of the test set, and restart. This happens mostly with small or very unbalanced datasets, or with outliers.

Retraining on all the data. Once the hyperparameters have been selected, one can pool training and validation data together and retrain the selected model with all of them, squeezing out a little extra performance (useful with small datasets; negligible with large ones). This does not cause overfitting: the role of the validation set was to limit the complexity of the model, which is now fixed, and we are just adding data.

5.1.4 The bias–variance trade-off

Let the *complexity* of a model be its flexibility, i.e., roughly its number of degrees of freedom: how many different datasets it could fit. (It is not just the number of parameters: a linear and a non-linear model with the same number of parameters may have different complexity.)

- The **training error** decreases monotonically with complexity; a model complex enough can bring it to zero by memorizing the data, and even below the noise level, by fitting the specific noise of the training set.
- The **validation (or test) error** is high for very simple models (underfitting), decreases as complexity grows, reaches a minimum, and then increases again (overfitting).

There is a sweet spot where the model is complex enough to have a low error, but not so complex as to overfit. There is also a floor below which the error on new data cannot go: the variance of the *noise* in the data, the *irreducible error* (pure randomness cannot be predicted, otherwise it would not be noise).

This is the classical **bias–variance trade-off**: the *bias* is the error due to choosing a model too simple; the *variance* is the variability of the model with respect to the particular dataset used to train it, the error paid because the model is too flexible. Their combination (plus the irreducible error) is the *expected prediction error*, high for low complexity and for high complexity. The optimal complexity must be selected on validation data, never on training data.

5.2 Cross-validation

Definition 5.1 (Cross-validation) *Cross-validation is the use of the available training data both for training and for validating a model (as opposed to, e.g., validation by a human expert, which is typical of clustering).*

Hold-out validation. The procedure described so far (set aside 10–20% of the training data, train on the rest, validate on the held-out part, stratifying in classification) is the one typically used with neural networks, which take long to train and are trained on large datasets. Its drawback is that the estimate depends on how lucky we were in the split: with a small dataset of easy and difficult samples, putting all the difficult samples in validation would lead to a pessimistic estimate and possibly a bad model selection, and vice versa.

Leave-one-out cross-validation (LOOCV). Remove one sample, train on all the others, test on the removed sample; repeat for each of the N samples, and average the N errors. The estimate no longer depends on a lucky split, but it requires N trainings: with 1000 samples, 1000 trainings. It is used only with very small datasets (tens of samples), where including or excluding a sample makes a big difference.

Example 5.3 (Leave-one-group-out) *Suppose we learn a diagnostic model from data collected in several hospitals and we want the model to be independent of the hospital. We can* leave one hospital out*: train on*

all the other hospitals and test on the left-out one, then repeat for each hospital. This selects a complexity that generalizes to new hospitals. The same applies to subjects. This is feasible only if the groups are few (say fewer than 30).

k-fold cross-validation. A middle ground: split the training data into k folds (e.g., $k = 5$); for each fold, train on the other $k - 1$ folds and validate on it; average the k errors. Roughly speaking, if LOOCV would locate the optimal complexity exactly, k-fold locates it approximately, and the hold-out estimate has the largest uncertainty. The cost is k trainings per hyperparameter configuration, which is affordable only if we can pay k times the computation or parallelize it on k machines. Stratification still applies.

After k-fold cross-validation we have k models for the selected configuration. Which one should be used?

- *The best one*: not a good idea, since it might be the best only because its fold was lucky; we would be back to a hold-out estimate after paying k times the computation.
- *All of them, as an ensemble*: majority vote for classification, average for regression.
- *None*: discard them, keep the selected hyperparameters, and retrain one model on all the training data (not the test data!).

In principle the k-fold idea could also be nested for the test estimate, but the number of trainings grows multiplicatively ($5 \times 5 = 25$, $10 \times 10 = 100$), so this is rarely done. In practice, with neural networks: hold-out validation most of the time, k-fold if computation allows, leave-one-(group)-out only for special cases such as the hospital example.

How large should the splits be? There is no exact rule; a rule of thumb is 10–20% for the test set and a similar amount for validation, keeping at least 50–70% for training (neural networks are very data-hungry, and throwing away half of the data makes little sense). What matters is the *absolute* number of samples: the confidence interval of the estimated error depends on the cardinality of the test set, so with very large datasets 5% may be plenty, while with 10 samples holding out 2 is questionable. What really matters is that the splits are *representative*: "random" should not mean "careless". Stratified sampling is one example; in regression, one can sort the samples by target value and sample uniformly along the sorted list to cover the whole range. Deep learning does not exempt us from the good practices of machine learning: exploratory data analysis, outlier removal, data cleaning.

5.3 Early stopping

Unless very small, neural networks are prone to overfitting: a network with 5 inputs, 10 hidden neurons and 1 output already has more than 50 parameters; adding a second layer of 10 neurons brings it above 150; with layers of 20–25 neurons we get close to a thousand. From the 1990s on, many techniques were proposed to prevent overfitting. The first one is embarrassingly simple.

Consider now a plot with the training *epochs* on the horizontal axis (not the model complexity!). Gradient descent decreases the training error at every iteration, otherwise it would not be learning; but it cannot tell whether it is decreasing the error "in a good way" (learning) or "in a bad way" (overfitting). If the training error goes to zero, one should be suspicious. Monitoring the error on held-out data during training, one typically observes that the validation error first decreases and then starts increasing: from that point on the network is using its excess degrees of freedom to learn the noise in the training data.

Definition 5.2 (Early stopping) *During training, evaluate the model on the validation set at each epoch, and stop training (keeping the weights) at the point where the validation error is minimum.*

Early stopping starts from the assumption that the model is too complex, and prevents overfitting by stopping gradient descent before it exploits the excess of freedom. Hence even a network larger than needed will not overfit. Moreover, it saves computation: one may plan 300 epochs and stop at 20–30.

Who guarantees that the validation curve does not go down again later? Nobody. Ideally one should train for all the epochs and keep the best model so far; to save time one introduces a **patience** parameter:

if the validation error has not improved for, e.g., 10 epochs, training stops. This is an approximation, but it saves a lot of computation, which can be spent testing more architectures (e.g., five architectures with 20 epochs each instead of one with 100), usually a better investment than finding the exact minimum. Early stopping uses the validation set, so it is subject to the same variability as model selection; with enough data this is not an issue.

5.3.1 Hyperparameter tuning with early stopping

With early stopping we get, for free, an online estimate of the generalization error: the validation error at the stopping point. Model selection then consists in training several models with early stopping and comparing their validation errors at stopping time.

Example 5.4 (Choosing the number of hidden neurons) *Train networks with* $1, 2, 5, 7, 9, \ldots$ *hidden neurons, each with early stopping. With 1, 2 and 5 neurons the training and validation errors are both high and keep decreasing: the models underfit. With 7 or 9 neurons the training error keeps decreasing while early stopping interrupts training earlier and earlier: beyond a certain size, adding neurons pushes down the training curve but no longer improves the validation error, which plateaus. Early stopping prevents the extra neurons from causing overfitting.*

With two hidden layers, all combinations of the two sizes should be tested (1-1, 1-2, 2-1, 2-2, ...), and with more layers the combinations explode. Unless one has the computational resources of a large company, choices must be made: test a coarse set of configurations, pick the best, and refine around it. Sampling the hyperparameter space on a grid, possibly refining it hierarchically, is called **grid search**; Bayesian optimization is a more sophisticated alternative (see also Section 5.7).

5.4 Weight decay

Early stopping requires setting data aside. Can we limit overfitting without a validation set? *Regularization* techniques limit the freedom of the optimization when fitting the parameters. They are not new: ridge regression and the lasso are linear models with penalized weights. The same idea applied to neural networks was given a new name (just as gradient descent was renamed backpropagation): *L2 weight regularization* is called **weight decay**. The complexity of a model is related to the number of its *free* parameters; regularization does not let backpropagation set them freely, but constrains them to a certain region of parameter space.

5.4.1 Maximum a posteriori estimation

Maximum likelihood (Chapter 4) is the obvious way to estimate parameters for a *frequentist*. A *Bayesian* sees things differently.

Example 5.5 (The coin) *Before tossing a standard one-euro coin, what is the probability of head? Everybody answers* 50%*: we have a* prior belief. *Toss it and get head. A frequentist, maximizing the likelihood of the single observed toss, now estimates* $P(\text{head}) = 1$*; most people still say* 50%*. Toss again: another head. The frequentist still says* 1*; the Bayesian says that two tosses are not enough to change their mind. Third toss: tail. The Bayesian is happy they did not change their mind, the frequentist now says* 2/3*. After a thousand tosses with* 995 *heads and* 5 *tails, however, everybody starts suspecting a fake coin: the data overwhelm the prior.*

Bayesian estimation has a prior belief on the parameters, and the estimate must be consistent with it rather than just fitting the data. It is very robust with little data and converges to the frequentist (maximum likelihood) estimate when data are abundant. This is precisely what we need to limit overfitting with limited data: make assumptions on the weights and let the data prevail over the prior only when they are many. The resulting estimate is the **maximum a posteriori** (MAP) estimate: the *most likely parameters given the data*,

$$\mathbf{w}_{\text{MAP}} = \arg\max_{\mathbf{w}} p(\mathbf{w} \mid \mathcal{D}) = \arg\max_{\mathbf{w}} p(\mathcal{D} \mid \mathbf{w})\, p(\mathbf{w}). \tag{5.1}$$

What is our prior on the weights of a neural network? Few people have ever looked at the weights of a network (if you did, that is worrying). In the 1990s it was observed that networks with *small weights* show a smaller gap between training and validation error. Hence the prior: weights are small and zero-mean. A Gaussian is chosen because it makes the computation simple (statisticians call priors that keep the posterior in a convenient form *conjugate priors*; there is no rule to choose priors, and a statistician's joke goes "one's prior is another's posterior"):

$$w_q \sim \mathcal{N}(0, \sigma_w^2) \qquad \text{i.i.d. for each weight } q = 1, \dots, Q. \tag{5.2}$$

5.4.2 Derivation

For regression (Gaussian noise with variance σ^2, i.i.d. samples):

$$\mathbf{w}_{\text{MAP}} = \arg\max_{\mathbf{w}} \prod_{n=1}^{N} \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(t_n - g(\mathbf{x}_n \mid \mathbf{w}))^2}{2\sigma^2}} \prod_{q=1}^{Q} \frac{1}{\sqrt{2\pi}\sigma_w} e^{-\frac{w_q^2}{2\sigma_w^2}} \tag{5.3}$$

$$= \arg\min_{\mathbf{w}} \sum_{n=1}^{N} \frac{(t_n - g(\mathbf{x}_n \mid \mathbf{w}))^2}{2\sigma^2} + \sum_{q=1}^{Q} \frac{w_q^2}{2\sigma_w^2}, \tag{5.4}$$

where we took the logarithm, changed sign, and dropped the terms that do not depend on $\mathbf{w}$. Unlike in pure maximum likelihood, the noise variance σ^2 no longer disappears, since it divides only the first term while σ_w^2 divides the second. Multiplying by $2\sigma^2$ and setting $\gamma = \sigma^2/\sigma_w^2$:

$$\boxed{\mathbf{w}_{\text{MAP}} = \arg\min_{\mathbf{w}} \underbrace{\sum_{n=1}^{N} \big(t_n - g(\mathbf{x}_n \mid \mathbf{w})\big)^2}_{\text{fitting}} + \gamma \underbrace{\sum_{q=1}^{Q} w_q^2}_{\text{regularization}}}. \tag{5.5}$$

The new loss balances two terms: how well the model fits the data, and how large the weights are. With $\gamma = 0$ the regularization vanishes and the model may overfit; with $\gamma \to \infty$ the second term dominates, all weights go to zero, and the model underfits. γ is a knob trading off model fitting and model complexity. Why is the weight norm related to complexity? Think of a network with a thousand weights: if half of them are (close to) zero, the network is effectively simpler; and it is gradient descent that chooses which weights to shrink, while fitting the data. The name "weight decay" comes from the gradient of the penalty: the update becomes $w \leftarrow w - \eta(\partial E/\partial w + 2\gamma w) = (1 - 2\eta\gamma)w - \eta\, \partial E/\partial w$, so at each step the weights shrink (decay) by a constant factor before the gradient step.

Remark 5.2 *This is exactly ridge regression: least squares plus an L2 (quadratic) penalty. In a neural network, the penalty is added to the loss and one can decide whether to apply it to all layers or only to some, how large γ is, and even use a different γ per layer (or per weight).*

5.4.3 L1 versus L2

With an absolute-value penalty (as in the lasso),

$$E(\mathbf{w}) = \sum_{n} \big(t_n - g(\mathbf{x}_n \mid \mathbf{w})\big)^2 + \gamma \sum_{q} |w_q|, \tag{5.6}$$

the behaviour changes: the L2 penalty makes the weights small but does not force them to zero, while the L1 penalty tends to set as many weights as possible *exactly* to zero, yielding a *sparse* (and more efficient) model. The Gaussian prior leads to L2 regularization; a *Laplace* prior leads to L1. Again, it is the prior that determines the loss. In practice one simply picks the regularizer, but it is important to understand the rationale behind it.

5.4.4 Choosing γ

γ is a hyperparameter and, like any hyperparameter, is chosen by cross-validation:

1. split the data into training and validation sets;
2. for each candidate γ, minimize the weight-decay loss on the training set and compute the error on the validation set;
3. choose γ^* with the lowest validation error;
4. put all the data together and retrain with γ^*.

For small γ the validation error is high (overfitting); it decreases as γ grows, then increases again when γ is too large. It does not go to infinity: with $\gamma \to \infty$ all weights are zero and (assuming activation functions that output zero for zero input) the model always predicts zero, so the validation error tends to the sum of the squared targets, a large value we do not want. The search range is $[0, \infty)$, so one uses a coarse logarithmic-like grid $(0.1, 1, 2, 5, 10, 20, 50, 100, \ldots)$, finds the range of the minimum and refines it hierarchically. Retraining on all data with γ^* does not overfit, because γ^* limits the freedom of the model and we are only adding data; in the end, the test set says whether the model is good.

In frameworks one adds the penalty per layer (e.g., an L2 penalty of 0.001 on the first layer). Using a different γ for each layer is possible but tuning many γ's is expensive; usually a single γ is used for all layers (or for the layers that seem to need it).

Weight decay and early stopping together. They can be combined, and often are: early stopping plus a small γ. Which one is better? With a very small dataset, the ability of weight decay to prevent overfitting without sacrificing data is valuable. With a minimally decent dataset, early stopping works perfectly, is easier to implement and shortens training (weight decay alone requires training until convergence, i.e., more epochs). Early stopping should generally be preferred, with weight decay as an option on top.

5.5 Dropout

All these techniques share the same philosophy: we do not know the right model, so we pick a model a bit larger than needed (one that could overfit) and we limit its overfitting — by stopping early, by penalizing the weights, or, as now, by *stochastic regularization*.

In a large network with more parameters than needed, neurons tend to become correlated or *co-adapted*: thanks to the excess of degrees of freedom, an error made by one node can be compensated by another, which can lead to overfitting. To force each neuron to learn its own role without co-adapting with the others, during training we randomly switch off some of the inputs of the neurons.

Definition 5.3 (Dropout) *At each training iteration, each hidden unit of a layer is set to zero with probability p (independently). This is implemented by sampling a binary mask $\mathbf{m}$ for the layer and multiplying it element-wise (Hadamard product) with the layer activations: $\tilde{\mathbf{h}} = \mathbf{m} \odot \mathbf{h}$.*

Masked neurons have no output: they do not participate in the forward pass, hence they are not involved in the backward pass, and only the weights along the surviving paths are updated. At the next iteration a new random mask is sampled, so a different sub-network is trained. During training we are therefore training, at each iteration, a different random *sub-network* of the big network; at test time the full network behaves like the *average* of a huge number of sub-networks, each trained on a slightly different view of the data. This resembles an *ensemble* method, which explains why dropout reduces the variance of the model (in the bias–variance sense); it is neither bagging nor boosting, rather something akin to a mixture of experts.

Weight scaling. At test time the masks are removed, and the average of the sub-networks must be computed. If during training one third of the neurons of a layer were switched off, the next layer was trained to receive only about 2/3 of its inputs; once the mask is removed it receives all of them, so its activation values would be 3/2 times larger. To keep the expected activation unchanged, the weights are rescaled by the keep probability $1 - p$ (here 2/3). (Implementations often perform the equivalent "inverted" scaling during training, dividing the surviving activations by $1-p$; the exact implementation in a library may differ slightly from the description here.)

Dropout as a layer. Dropout is part of the network: it is added as a *layer* (e.g., after a dense layer), which takes the input, samples the mask, and outputs the masked input. It has no trainable weights, only the probability p. Its derivative with respect to each input is 1 if the corresponding mask bit is 1 and 0 otherwise, so it integrates naturally in the forward–backward algorithm: at each iteration the gradient is backpropagated taking into account the mask sampled for that iteration. Which units are dropped is decided automatically; the user chooses only p, which sets how aggressive the regularization is.

Practical remarks.

- p is a hyperparameter to be cross-validated, but grid-searching everything would require thousands of machines: often a default (0.1–0.3) is used, or a few values are tested.
- The three techniques can be combined because they act at different levels: early stopping on the *training loop*, weight decay on the *loss function* (gradient descent), dropout on the *model*.
- There is no guarantee that dropout prevents overfitting or finds the optimum: in some applications it helps, in others it gives results similar to early stopping and weight decay. It does not hurt, and it seems less sensitive to the choice of p than weight decay is to γ; but it does not replace early stopping, which remains simple and very effective. Dropout is worth knowing well because it appears in many famous architectures.

5.6 Laboratory: overfitting and regularization in practice

5.6.1 The Glass dataset and unbalanced classification

The *Glass Identification* dataset from the UCI Machine Learning Repository (a repository of benchmark datasets, which also contains Iris) was motivated by criminological investigations: identifying the type of glass found at a crime scene can be used as evidence. It contains 214 samples and 10 columns: 9 numerical features (the refractive index and the concentrations of chemical elements such as sodium, magnesium, aluminium, silicon, ...) and the class. Of the 7 original classes (building windows, vehicle windows, containers, tableware, headlamps, ...) one is absent, leaving 6. Observations:

- feature means are very different (e.g., silicon has mean around 72), so normalization is mandatory;
- classes are *strongly unbalanced* (one class has only 9 samples);
- pair plots are much messier than for Iris: some classes are separable, but non-linear models will likely be needed.

Metrics for unbalanced problems. Accuracy is very interpretable but misleading with unbalanced data: in a binary problem where 99% of the samples belong to one class, a model that always predicts that class reaches 99% accuracy. Starting from the confusion matrix (true/false positives TP, FP and negatives TN, FN):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{Precision} = \frac{TP}{TP + FP}, \tag{5.7}$$

$$\text{Recall} = \frac{TP}{TP + FN}, \qquad F_1 = 2\,\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{5.8}$$

Precision is the fraction of samples predicted positive that are actually positive; recall is the fraction of actual positives that are retrieved. F_1, their harmonic mean, summarizes both in a single number to optimize. In multi-class problems each class is treated as a binary *one-vs-all* problem, per-class precision and recall are computed and then averaged (*macro* average) or averaged with weights proportional to class frequencies (*weighted* average). In this lab the weighted F_1 is the validation metric.

Preprocessing and model. The pipeline is the same as in Section 3.5: stratified train/validation/test splits (crucial with unbalanced classes), min–max scaling, tensors, data loaders (batch size 32). The model is a configurable feedforward network:

```
class FFNN(nn.Module):
    def __init__(self, in_features, hidden_layers, hidden_size, num_classes, dropout_rate=0.0):
        super().__init__()
        layers = [nn.Linear(in_features, hidden_size), nn.ReLU()]   # at least one hidden layer
        for _ in range(hidden_layers - 1):
            layers.append(nn.Linear(hidden_size, hidden_size))
            if dropout_rate > 0:
                layers.append(nn.Dropout(dropout_rate))
            layers.append(nn.ReLU())
        layers.append(nn.Linear(hidden_size, num_classes))         # output layer (logits)
        self.net = nn.Sequential(*layers)
    def forward(self, x):
        return self.net(x)
```

With hidden layers of 512 neurons the network has far more parameters (hundreds of thousands) than samples (214). Counting parameters is a classic exam exercise: a dense layer from 512 to 512 neurons has 513×512 parameters, because of the bias of the previous layer.

5.6.2 Baseline: overfitting

Training the baseline (Adam, learning rate 10^{-3}, cross-entropy, 500 epochs) the training F_1 goes from 0.2 to essentially 1.0 and the training loss becomes tiny, while the validation F_1 goes from 0.2 to about 0.5 and stays there: a clear case of overfitting. The model always optimizes on the training set; overfitting means it has learned the training set by heart instead of patterns that generalize. The confusion matrix shows, in particular, much confusion between the two majority classes.

Bias and variance, visually. Picture a dartboard where each dart is a model of a family trained on a different realization of the dataset. *Low bias, low variance*: darts concentrated on the bull's eye. *High bias, low variance*: concentrated but off-centre (a family with a strong, wrong prior). *Low bias, high variance*: scattered around the centre: the model changes a lot with small changes of the dataset (powerful models such as neural networks). In practice one moves along the trade-off between simple models (high bias, low variance, e.g., linear regression) and complex ones (low bias, high variance, e.g., neural networks), since the total error is bias plus variance plus irreducible error, and the optimum lies in the middle.

Start big, then regularize. Should we start from a simple model and increase complexity, or from a complex model and reduce it? Regularization makes a network simpler *automatically during training*: starting from an overfitting model, one regularization run explores many effective complexities, whereas starting from a small model the only way to increase complexity is to add neurons and retrain. Hence the usual practice: make the model large enough to overfit, then regularize it back to the right complexity — like a sculptor who starts from a large block of stone and removes material, rather than gluing small rocks together.

5.6.3 Early stopping

Early stopping should be used in essentially every training: it is free and sensible. Overfitting appears as the divergence between training and validation curves; when it happens one can stop, reduce the learning

rate, or otherwise act. Implementation inside the `fit` function:

```
best_metric, best_epoch, patience_counter = -float("inf"), 0, 0
for epoch in range(1, epochs + 1):
    train_loss, train_f1 = train_one_epoch(...)
    val_loss, val_f1 = validate_one_epoch(...)
    history["val_f1"].append(val_f1)            # ... and the other metrics
    if val_f1 > best_metric:                    # improvement
        best_metric, best_epoch = val_f1, epoch
        torch.save(model.state_dict(), "best_model.pt")
        patience_counter = 0
    else:
        patience_counter += 1
        if patience_counter >= patience:
            break                               # stop the training loop
model.load_state_dict(torch.load("best_model.pt"))   # restore the best weights!
```

The last line is often forgotten, making early stopping useless: the best weights must be restored. With patience 200, the best validation F_1 is reached around epoch 100 and training stops slightly before epoch 300. Monitoring the loss instead of F_1 would stop earlier (around epoch 70) with a slightly lower F_1; neither choice is better in general. If time allows, set the patience high: on real data, validation curves are far from smooth (they go down, up, stay up for long, go down again). All these parameters are guidelines; non-linear optimization has no recipe that always works, and some trial and error, sometimes frustrating, is normal.

5.6.4 Dropout

Dropout is a PyTorch layer (no trainable parameters), added in the model constructor if `dropout_rate > 0`. It acts at each forward pass during training and is switched off at validation time. This is controlled by the mode of the model:

- `model.train()`: gradients are tracked and dropout is active;
- `model.eval()`: dropout is switched off (and, together with `torch.no_grad()`, no gradients are tracked, so evaluation is faster).

Intuitively, since random parts of the network are switched off, the network learns *redundant* parts that solve the same sub-task: dropout moves the model towards simpler models automatically. With a rate of 0.5 (quite high, half of the units switched off at each step) training is slower and the best epoch is later; if a model still overfits with such a high rate, it is better to remove layers and use a milder rate (e.g., 0.2). With only about 20 validation samples, differences of a single sample change the metrics substantially: the "best epoch" is partly a matter of chance, which is another reason to keep a reasonably sized validation set.

5.6.5 L1, L2 and elastic net

Overfitting models tend to have large weights. With ReLU networks the decision regions are bounded by hyperplanes; to separate perfectly samples that are very close to each other (possibly just noisy points) the network needs very steep hyperplanes, i.e., large weights. Penalizing the weight magnitude discourages this:

- **L2** (weight decay): sum of squared weights. Large weights are heavily penalized, small ones barely: weights become small but not exactly zero.
- **L1**: sum of absolute values. It promotes weights that are *exactly* zero: in one dimension, adding a parabola to the loss moves the minimum but keeps it smooth, while adding a cone $|w|$ creates a sharp tip at zero where the "ball" of gradient descent stops. L1 thus trains a network with effectively fewer parameters; applied to the first layer, a weight driven to zero reveals a useless feature, which gives L1 some interpretability.
- **Elastic net**: a weighted sum of both. Historically L2 is the most common; in practice, with small datasets, L1 works very well too.

Implementation, inside the training step:

```
loss = criterion(logits, yb)
if l1_lambda > 0:
    loss = loss + l1_lambda * sum(p.abs().sum() for p in model.parameters())
if l2_lambda > 0:
    loss = loss + l2_lambda * sum(p.pow(2).sum() for p in model.parameters())
```

PyTorch lets us add any differentiable operation to the loss: penalize only some layers, apply L1 to the first layer only to prune features, etc. Since the optimizer minimizes a single number, the *scale* of each term matters: with a very large λ the loss is dominated by the penalty and all the weights go to zero (a useless network). L1 and L2 terms have very different scales; it is good practice to plot the terms separately and make sure the task loss (the cross-entropy) remains the main driver, with the others in a reasonable order of magnitude. In the experiment, L2 regularization with early stopping still overfits, but less.

5.6.6 AdamW: decoupled weight decay

Optimizers differ greatly in how fast and how stably they reach a minimum. Visualizations of optimizer trajectories on two-dimensional loss surfaces (with a global minimum, a local minimum and a saddle-like region) show that plain SGD gets there but slowly and is almost never optimal; SGD with momentum is faster but spirals around minima; RMSprop and Adam add heuristic terms (adaptive per-parameter step sizes) that on average make the descent faster and more stable; Adam is the standard for neural networks.

If the L2 penalty is added to the loss and Adam is used, Adam adapts the per-parameter step size using the gradient of *both* the task loss and the penalty, and adapting on the L2 term has been shown to work poorly. **AdamW** *decouples* weight decay: Adam is applied to the gradient of the task loss only, and then all weights are shrunk by the same small factor (hence "weight decay": at each update a fraction of the previous weight is removed). AdamW is the de facto standard for training neural networks. In code the weight decay is passed to the optimizer, not added to the loss:

```
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=l2_lambda)
```

5.6.7 Final test

After all the experiments (the parameters fitted on the training set, the hyperparameters chosen on the validation set), the selected model is evaluated once on the test set: the weighted F_1 is a bit lower than in validation but in the same range, and the confusion matrix shows reasonable errors with the majority classes almost always correct. *At this point one must stop*: trying to improve the test number would mean optimizing on the test set, and the reported performance would no longer be reliable.

5.7 Laboratory: evaluation strategies and hyperparameter tuning

5.7.1 Four evaluation strategies

Real data arrive full of problems (missing values, inconsistent columns, heterogeneous sources to merge and clean); suppose this has been done and we have a clean dataset. Four increasingly robust ways to use it are:

1. **Training only.** All data are used for training. No estimate of out-of-sample performance is possible. Is it useless? Not entirely: a model unable to fit the training set is certainly not a good model, so this setting is useful to check that preprocessing is correct and that the model can reach (nearly) zero training error, i.e., that it is complex enough. (In generative tasks, splits are sometimes not needed at all, but such tasks are evaluated differently.)

2. **Test hold-out.** Train on a training set, evaluate on a held-out test set, report the test performance, and retrain the final model on all data. This provides an estimate of what the user can expect on unseen data while the delivered model has seen all the data; it is almost a gold standard nowadays, but it does not allow *model selection*.

3. **Training, validation and test.** The minimum sufficient setting for a fair evaluation. The training performance tells whether the model is learning; the validation performance is "an estimate of an estimate of an estimate" (it estimates the test performance, which estimates the performance on unseen data), and it is used for model selection; the test set gives the final estimate. Its weakness: with a few hundred samples, a complex problem and many classes, a single split may remove too much information, or the subsets may have different distributions, so we may be optimizing for something that is not our real goal.

4. **Cross-validation.** Repeat the training/evaluation k times (typically $k = 5$ or 10) on different splits and average the results; averaging reduces the influence of lucky or unlucky splits. A common variant first sets aside a fixed test set and cross-validates only on the rest; but if that single test set is biased (e.g., contains cherry-picked samples) the problem remains. A more robust variant is *k-shuffle-split*: at each of the k rounds, randomly (and, with unbalanced classes, stratifiedly) draw validation and test sets (e.g., 20% each) from the whole dataset, with a different random seed per round. Overlaps between rounds do not matter, and the sizes of validation and test sets can be controlled independently of k (with a 3-fold scheme they would be very large). The temptation to cheat by selecting the model looking at the test sets must be resisted: the truth comes out when the model meets unseen data.

What is model selection? Any change — a different architecture, a different hyperparameter value, even a different random seed — produces a different model; choosing among them is model selection. *Hyperparameter tuning* is model selection when the base model is fixed.

Which model to deliver after cross-validation? With classical models (random forests, k-means, ...) one retrains a single model on the whole dataset with the selected hyperparameters. With neural networks there is a complication: early stopping is model-specific, and each of the k models stopped at a different epoch (say 101 in one fold and 20 in another). Averaging the epochs and retraining is possible, but with heterogeneous folds it is questionable, and the optimal epoch for one subset need not be optimal for another. A cleaner solution is to keep the k models and use them as an *ensemble*: each model is evaluated on its own validation and test sets, and in production the predictions of the k models are averaged (regression values or classification confidences).

5.7.2 Implementation details

- **Splits and normalization inside the loop.** Splitting and normalization move from preprocessing into the cross-validation loop. Normalization statistics (minimum and maximum, or mean and standard deviation) must be computed *on the training set only* of each round: using validation or test data would leak information that is not available before production and lead to an optimistic estimate.
- **Different seeds per round**, otherwise the same split is repeated.
- **Same initial weights.** The initial state of the model can be saved and reloaded at each round, to reduce the noise due to initialization (perfect reproducibility is not possible with GPU acceleration anyway, and we accept that, trading it for efficiency).
- **Epochs and patience.** With early stopping, set the number of epochs very large (e.g., 1000–5000): it costs nothing, while underestimating it loses performance. Patience is a trade-off: if the model always converges in about 20 epochs, a patience of 100–200 is unnecessary. Use the cheaper evaluation settings first to become familiar with the data.

With a 20% test fraction (the dataset has a few hundred samples; with more data the fraction can decrease) and $k = 5$, the baseline gives per-round F_1 scores of, e.g., 71%, 68%, 59%, 73%, 71%: merely changing the random split changes the result by more than ten points. Imagine a clinician telling a patient that the survival chance is 71% in one scenario and 59% in another! Nothing is worse than a model that has not been evaluated properly: a model that is known to be bad is better than a promising one whose real performance is unknown. Increasing k improves the estimate, at the cost of time (here two minutes for a single 5-round evaluation; grids with hundreds of configurations multiply this).

5.7.3 The main hyperparameters

Batch size. Consider the extremes. With batch size 1 an epoch requires one update per sample (slow), and the last update of an epoch might come from an outlier, affecting the whole model just before the epoch-level validation used by early stopping. With batch size equal to the whole dataset (if it fits in memory) there is one update per epoch (efficient), but the model converges to the "average": all errors are squeezed into a single update always computed on the same data, which tends to sacrifice the rare samples. The best value lies in between and is found experimentally (here values from 4 to 128). Batch sizes are usually powers of two, because GPU computations are optimized for full matrices of such sizes (the habit of using powers of two for the number of neurons has no deep justification). More generally, one must accept sub-optimality: do not test 16, 17, 18 neurons; test around 100, around 500, and reason by neighbourhoods rather than single values, also because changing the split changes everything.

Learning rate. The factor multiplying the update. Picture the optimizer as a ball rolling in the loss valley (one dimension per weight); the learning rate is related both to its speed and to its diameter. A big ball explores faster but may get stuck bouncing between the two sides of a valley; a *learning rate scheduler* reduces the learning rate when the metric stalls for too many epochs, as if the ball shrank and could fall further down (it usually, though not always, works). A too small learning rate (e.g., 10^{-5}, and certainly 10^{-6}) leaves the ball stuck close to its starting point. Whatever the schedule, different initial learning rates should be tested; here they change the F_1 by about six points.

Model complexity. Tuning complexity alone is not very informative: to know whether a model is complex enough, the training set suffices (if it can overfit, it is complex enough). Once the right order of magnitude is known (not a billion parameters, not a million, maybe tens or hundreds of thousands), one can tune within that neighbourhood. In the experiment, beyond 128 neurons per layer there is no improvement.

A subtle trap. The default configuration (batch size 64, learning rate 10^{-3}) appears in every experiment, and gives 68.68% in one run and 64.45% in another: identical configurations give different results because training is not reproducible. Always save the best models; and once a good configuration is found, re-running it several times may reveal whether the first result was lucky (or yield an even luckier one).

5.7.4 Search strategies

- **Grid search**: for each of n hyperparameters choose a set of candidate values and test all combinations; the number of combinations explodes quickly (e.g., 432 combinations for a moderate grid, many hours of computation).
- **Incremental (coordinate-wise) search**: find the best batch size and fix it, then the best learning rate, then the complexity; possibly iterate. Many combinations are discarded, which may or may not matter.
- **Grouped search**: group hyperparameters by meaning. First tune the "learning" group (batch size, learning rate, complexity) jointly, reaching e.g. 74.8%; then, from that configuration, tune the "regularization" group (dropout, L1, L2/weight decay in AdamW), improving slightly; finally, a full grid over everything (432 combinations) reached about 75.8% before being stopped.
- **Random search**: sample random points of the search space for the available time budget and keep the best. Random search is the baseline every optimization method should be compared with: several times in the literature, years of increasingly sophisticated methods turned out not to beat it. It is also robust to the order of nested loops (with a grid, if the first hyperparameter of the outer loop takes its worst value, half of the time is wasted).
- **Model-based search**: use an optimization algorithm to tune the model, e.g., Bayesian optimization (an uncertainty-aware surrogate guides the exploration towards promising regions) or genetic algorithms (a fitness function). Libraries such as Optuna and Ray Tune implement these; used improperly, they are no better than random search.

Why not always cross-validate? Because of computational time: training a large language model ten times would require an enormous amount of energy. When data are plentiful and well distributed (few outliers, well-represented classes), a single training/validation/test split is acceptable; otherwise, whenever possible, cross-validation should be used.

Remark 5.3 (No free lunch) *No model is the best for every problem: do not fall in love with a model, because for sure there is a problem where it is the worst one.*

Chapter 6

Training Deep Networks: Activations, Initialization, Normalization and Optimizers

The techniques of Chapter 5 deal with generalization. This chapter collects several topics that were postponed so far and that are not about the architecture of feedforward networks as such, but about making *backpropagation and gradient descent* work well, especially in deep networks: why sigmoid and tanh are no longer used in hidden layers, how weights should be initialized, how to keep the signals in a numerically good range with batch normalization, and which variants of gradient descent are used in practice.

6.1 The vanishing gradient problem

S-shaped activation functions *saturate*: outside, say, $[-5, 5]$ their derivative is practically zero. Moreover their derivative is never larger than one: the maximum of σ' is 0.25 (at the origin), the maximum of $\tanh'$ is 1 (only at the origin); elsewhere they are smaller, going to zero. So we can say that their derivative is everywhere ≤ 1.

Recall the structure of backpropagation (Chapter 3): the gradient with respect to a weight of the first layer is the product of the derivative of the error, the derivative of the output neuron, a weight, the derivative of a hidden neuron, ..., and the input. With many hidden layers:

$$\frac{\partial E}{\partial w^{(1)}} = \frac{\partial E}{\partial g}\, g'(\cdot)\, w^{(L)}\, h'_{L-1}(\cdot)\, w^{(L-1)}\, h'_{L-2}(\cdot) \cdots w^{(2)}\, h'_1(\cdot)\, x. \tag{6.1}$$

The gradient of the first-layer weights involves the product of the derivatives of *all* the layers traversed by the signal. If all these derivatives are smaller than one — say 0.1 — with 10 layers the product is 10^{-10}, with 100 layers 10^{-100}, i.e., zero. The farther a layer is from the output, the smaller its gradient.

Definition 6.1 (Vanishing gradient) *In deep networks with activation functions whose derivative is smaller than one, the gradient shrinks exponentially as it is backpropagated towards the input layers; the first layers receive practically no gradient and their (random) initial weights are not updated.*

This phenomenon hindered the training of deep networks: in the 1990s people were happy with networks of one to five layers, and deeper networks simply did not give useful results (papers report only the good results). A sound analysis of the gradient magnitude through layers was carried out in the late 1990s in the context of *recurrent* neural networks, where the problem was first identified and studied, and where it was fixed in a specific way (Chapter 7); but it affects deep feedforward networks too.

If derivatives smaller than one are the problem, could we use activation functions with derivative larger than one? No: multiplying many numbers larger than one (e.g., 5^{100}) leads to the opposite problem, the **exploding gradient**, which causes numerical instability. The ideal activation has derivative exactly equal

to one: the linear function. This also explains the old advice to prefer tanh over the sigmoid: it does not "work better", it suffers less from vanishing gradients, since its derivative is on average closer to one; but it does not fix the problem. The linear function fixes it, but a network of linear units is linear. The solution is to make it *just a little* non-linear.

6.2 Modern activation functions

6.2.1 Rectified Linear Unit (ReLU)

Definition 6.2 (ReLU)

$$g(a) = ReLU(a) = \max(0, a) = \begin{cases} a & a > 0 \\ 0 & a \leq 0 \end{cases}, \qquad g'(a) = \begin{cases} 1 & a > 0 \\ 0 & a < 0 \end{cases}. \tag{6.2}$$

Advantages:

- the derivative is 1 on the positive side: no vanishing gradient through active units;
- *faster convergence* than sigmoid and tanh, which makes sense since gradients, hence steps, are larger;
- *sparse activation*: only part of the hidden units are active (non-zero), and products involving zeros are cheap; very sparse vectors can even exploit sparse implementations;
- *efficient computation*: a max is much cheaper in hardware than an exponential; the derivative is just a check on the sign of a.

Potential issues:

- *Non-differentiability at zero*: a set of measure zero, easily fixed by conventionally setting the derivative to 1 (or 0) for $a = 0$ (i.e., writing $a \geq 0$ instead of $a > 0$). Not a real problem.
- *Non zero-centred and unbounded output*: unlike sigmoid ($[0, 1]$) and tanh ($[-1, 1]$), the output of a neuron can grow large, possibly leading to numerical instability and unstable convergence. Not a big deal with a sensible initialization.
- *Dying neurons*: this is the real problem. If a neuron is pushed into a state where its activation value is negative for all inputs, its output is always zero and its gradient is zero: the neuron can never be rescued and never contributes to the output. One may design a network with, say, 10 000 neurons but effectively use only a couple of thousand, hence a model less powerful than designed, which may underfit. The problem is worse if it happens early in training, and with high learning rates: a big step can throw a neuron that should have a small positive activation into the negative region, and then it is too late to come back.

Remedies for dying neurons: make the network bigger (some neurons die, but enough survive), or use a variant with non-zero gradient on the negative side.

6.2.2 Leaky ReLU and ELU

Definition 6.3 (Leaky ReLU)

$$g(a) = \begin{cases} a & a \geq 0 \\ 0.01\, a & a < 0 \end{cases}. \tag{6.3}$$

A small gradient on the negative side gives a neuron that ended up on the wrong side the chance to recover (slowly, especially if far from the output). This small slope does not reintroduce vanishing gradients: the gradient would vanish only if all the neurons along the path were on the negative side. If switching from ReLU to Leaky ReLU changes nothing, there is nothing to worry about: it means dying neurons were not an issue (e.g., the network is big enough to afford them). It matters when the network is sized almost exactly right, so that dead neurons make it underfit.

Definition 6.4 (Exponential Linear Unit, ELU)

$$g(a) = \begin{cases} a & a \geq 0 \\ \alpha\,(e^a - 1) & a < 0 \end{cases}. \tag{6.4}$$

ELU joins the positive linear part with an exponential, making the function smooth (continuous and differentiable). The gradient on the negative side becomes very small for large negative inputs, so dying neurons can still occur, but it has been shown to work well.

6.2.3 Swish and the zoo of activations

A few years ago a "race" of activation functions took place, leaving us with half a dozen common choices (ReLU, Leaky ReLU, ELU, GELU, Swish, ...). A popular modern one is

$$\text{Swish}(a) = a\,\sigma(a), \tag{6.5}$$

which tends to 0 for $a \to -\infty$ (the sigmoid goes to zero), tends to the identity for $a \to +\infty$ (the sigmoid goes to one), and has a small negative dip in between. Empirically it converges more smoothly and sometimes faster: a network of ReLUs computes a piecewise linear function (a "polytope" made of planes), while Swish yields a smooth function, which seems to help slightly.

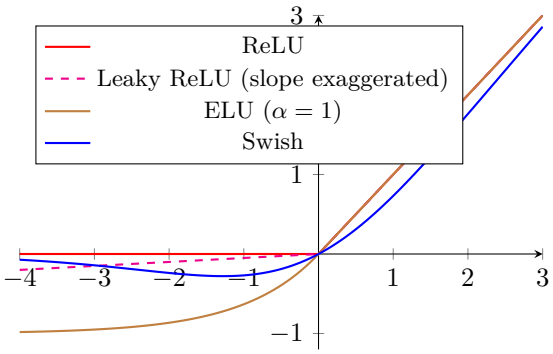


Figure 6.1: Modern activation functions for hidden layers.

In practice: moving from sigmoid/tanh to ReLU or Leaky ReLU gives a clear improvement, especially in deep networks; moving further to ELU or Swish may give some improvement, but not an astonishing one. All of this concerns *hidden* layers. The output layer still follows Chapter 3: linear for regression, sigmoid for binary and softmax for multi-class classification. If the output is known to be positive, one could use a ReLU output, or regress the square root of the target (or learn a quantity and square it), but most of the time linear, sigmoid and softmax are used.

6.3 Weight initialization

We said weights are initialized "randomly": but how?

- **All zeros** is a bad idea: looking at the backpropagation formula (derivative × weight × derivative × weight × ⋯), if all the weights are zero, all the gradients (except those of the last layer) are zero, and learning is extremely slow, starting only from the last layer.
- **Big values** are also bad: products of large weights lead to exploding gradients and, with sigmoidal activations, large activation values push units into saturation, where the derivative is tiny.
- Historically, weights are drawn **centred around zero** with small variance, where non-linear functions have their largest gradient, e.g., $w \sim \mathcal{N}(0, \sigma^2 = 0.01)$, so that weights lie roughly within ± 0.3. For small networks this is fine.

In deep networks the gradient is a product not only of derivatives (fixed by ReLU) but also of *weights*: if the weights are too small, the signal (and the gradient) shrinks as it traverses the layers; if too large, it grows. A compromise is needed.

6.3.1 Xavier initialization

Consider a neuron with linear activation (reasonable for a ReLU unit that is not dead) and random weights:

$$h_j = \sum_{i=1}^{I} w_{ji}\, x_i. \tag{6.6}$$

We would like the signals traversing the network to stay within a reasonable range (say ± 1, ± 2), neither tiny nor huge, so that gradient descent is numerically stable. The variance describes the expected squared deviation from the mean, i.e., the range of variability, so we study $\mathrm{Var}(h_j)$. For independent w and x,

$$\mathrm{Var}(w_{ji}x_i) = \mathbb{E}[x_i]^2\,\mathrm{Var}(w_{ji}) + \mathbb{E}[w_{ji}]^2\,\mathrm{Var}(x_i) + \mathrm{Var}(w_{ji})\,\mathrm{Var}(x_i). \tag{6.7}$$

The weights have zero mean by construction (we initialize them), and we assume zero-mean inputs (inputs are typically standardized, a trick discussed below). Then $\mathrm{Var}(w_{ji}x_i) = \mathrm{Var}(w_{ji})\,\mathrm{Var}(x_i)$. With i.i.d. weights and inputs, the variance of the sum is the sum of the variances:

$$\mathrm{Var}(h_j) = I \cdot \mathrm{Var}(w)\,\mathrm{Var}(x). \tag{6.8}$$

The range of the output of a linear (or active ReLU) unit therefore depends on the *number of inputs*, on the variance of the inputs and on the variance of the weights. Since the input of a layer is the output of the previous one, at initialization the signal is multiplied at each layer by $I \cdot \mathrm{Var}(w)$: e.g., a vector of ones becomes of the order of $I\,\mathrm{Var}(w)$ after one layer, $(I\,\mathrm{Var}(w))^2$ after two, and so on. To keep the variance unchanged through layers, at least at the beginning of training (backpropagation will then do its job, but we do not want initialization to create numerical problems from the start), Xavier Glorot proposed

$$I_{\text{in}} \cdot \mathrm{Var}(w) = 1 \quad \Longrightarrow \quad w \sim \mathcal{N}\left(0, \frac{1}{I_{\text{in}}}\right), \tag{6.9}$$

where I_{in} is the number of inputs of the layer (this holds for every layer, not only the first). For instance, a dense layer fed by 10 inputs is initialized with variance 0.1; a following dense layer fed by 100 hidden neurons with variance 0.01.

6.3.2 Glorot and He initialization

The same reasoning applied to the backward pass (from the output to the input) leads to $\mathrm{Var}(w) = 1/I_{\text{out}}$, with I_{out} the number of outputs of the layer. Glorot and Bengio proposed a compromise, the inverse of the average of the two:

$$w \sim \mathcal{N}\left(0, \frac{2}{I_{\text{in}} + I_{\text{out}}}\right). \tag{6.10}$$

More recently, He et al. observed that nobody uses linear activations: with ReLU, on average only half of the units are on the positive side, which halves the variance of the output. To compensate,

$$w \sim \mathcal{N}\left(0, \frac{2}{I_{\text{in}}}\right). \tag{6.11}$$

Take-home message. The linear assumption may seem crude, but in practice tying the variance of the initial weights to the number of neurons of the adjacent layers works well. Compared with naive initializations the improvement is noticeable; among Xavier, Glorot and He the differences are not dramatic. Libraries such as PyTorch already use sensible defaults, so usually nothing needs to be done; one can change the initializer, but "creative" choices (e.g., a wide uniform distribution) jeopardize the stability of training, especially at the beginning, where we want a well-conditioned setup.

6.4 Batch normalization

For gradient descent to be numerically stable, all numbers should be in a similar range. A standard preprocessing step is *standardization*: for each input feature, subtract the mean and divide by the standard deviation. It is a linear transformation that brings all inputs to zero mean and unit variance (no feature in the billions next to features close to zero). Then a well-initialized network does not amplify nor shrink the signal, and gradient descent can converge smoothly.

However, during training the distribution of the activations of each hidden layer changes, and the nice zero-mean, unit-variance condition may be lost inside the network (a phenomenon related to the so-called *internal covariate shift*). Moreover we train with mini-batches, and each batch is a sub-sample whose statistics may differ from those of the full dataset. The idea of batch normalization is to normalize also the hidden layers, *at the level of each mini-batch*, so as to keep the activations in a numerically stable range.

Definition 6.5 (Batch normalization) *Given the activations $\{x_1, \dots, x_m\}$ of a unit over a mini-batch $\mathcal{B}$ of size m:*

$$\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i, \qquad \sigma^2_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2, \tag{6.12}$$

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}, \qquad y_i = \gamma\,\hat{x}_i + \beta, \tag{6.13}$$

where ϵ is a small constant for numerical stability and γ (scale) and β (shift) are trainable *parameters, one per unit.*

A batch normalization layer is placed, like dropout, inside the network: typically after a dense layer (on the weighted sums) and before the non-linearity. Being a differentiable (linear, given the batch statistics) transformation, its forward and backward passes are known, and backpropagation takes it into account seamlessly. The learnable γ and β allow the network to learn the most appropriate scale and shift of each unit (even to undo the normalization if needed), so normalization does not reduce the expressiveness of the network.

Training versus inference. During training the statistics are those of the current batch, which are not the statistics of the whole dataset. At test time we may have a single sample, so there is no batch to compute the statistics on: the layer must know which mean to subtract and which scale to apply. For this reason, batch normalization layers keep running estimates (moving averages) of the mean and variance computed during training, which are used at inference time together with the learned γ and β. Hence a batch normalization layer *has parameters* (a scale vector and a shift vector, plus the running statistics), which appear in the parameter count of famous networks.

Effects. Batch normalization has been shown to:

- improve convergence and make training more stable and faster;
- allow higher learning rates (large gradients become less dangerous since the signals are kept in a good range);
- reduce the dependence on weight initialization (the goal of careful initialization was precisely to keep the outputs in a good range, which batch normalization enforces anyway; using both is fine);
- in some experiments, reduce the need for dropout (this should not be the main reason to use it).

6.5 Beyond plain gradient descent

Backpropagation is gradient descent, and many improvements of gradient descent exist. We already met *momentum* (Chapter 3): a step along the current gradient plus a step along the previous direction, which helps overcome small bumps and usually converges faster.

Nesterov accelerated gradient. Momentum first computes the gradient at the current point, steps along it, and then adds the step in the previous direction. Nesterov reverses the order: first take the step in the previous direction, then compute the gradient *at the look-ahead point* and step along it. Moving first along the old direction brings us to a better position on the error surface before looking at the gradient:

$$\mathbf{v}^{k+1} = \alpha\,\mathbf{v}^k - \eta\,\frac{\partial E}{\partial \mathbf{w}}\bigg|_{\mathbf{w}^k+\alpha\mathbf{v}^k}, \qquad \mathbf{w}^{k+1} = \mathbf{w}^k + \mathbf{v}^{k+1}. \tag{6.14}$$

Adaptive learning rates. The magnitude of the gradient differs across layers (it is multiplied by different numbers of factors), so each layer would have its own ideal learning rate; one also wants to compensate for vanishing gradients and to adapt the learning rate during training. This motivated a long list of variants: Rprop (1993), Adagrad (around 2010), Adadelta (2012), Adam (2014), AdamW (2017), and more. They keep a history of the past gradients *of each weight* and derive a per-weight step size. For reference, the main update rules are (with $\mathbf{g}^k$ the gradient at step k and all operations element-wise):

$$\text{Adagrad:}\quad \mathbf{r}^k = \mathbf{r}^{k-1} + \mathbf{g}^k \odot \mathbf{g}^k, \qquad \mathbf{w}^{k+1} = \mathbf{w}^k - \frac{\eta}{\sqrt{\mathbf{r}^k}+\epsilon} \odot \mathbf{g}^k, \tag{6.15}$$

$$\text{RMSprop:}\quad \mathbf{r}^k = \rho\,\mathbf{r}^{k-1} + (1-\rho)\,\mathbf{g}^k \odot \mathbf{g}^k, \qquad \mathbf{w}^{k+1} = \mathbf{w}^k - \frac{\eta}{\sqrt{\mathbf{r}^k}+\epsilon} \odot \mathbf{g}^k, \tag{6.16}$$

$$\begin{aligned}\text{Adam:}\quad & \mathbf{m}^k = \beta_1\mathbf{m}^{k-1} + (1-\beta_1)\mathbf{g}^k, \quad \mathbf{v}^k = \beta_2\mathbf{v}^{k-1} + (1-\beta_2)\,\mathbf{g}^k \odot \mathbf{g}^k, \\ & \hat{\mathbf{m}}^k = \frac{\mathbf{m}^k}{1-\beta_1^k}, \quad \hat{\mathbf{v}}^k = \frac{\mathbf{v}^k}{1-\beta_2^k}, \qquad \mathbf{w}^{k+1} = \mathbf{w}^k - \eta\,\frac{\hat{\mathbf{m}}^k}{\sqrt{\hat{\mathbf{v}}^k}+\epsilon}.\end{aligned} \tag{6.17}$$

AdamW applies the weight decay directly to the weights, decoupled from the adaptive update (Section 5.6): $\mathbf{w}^{k+1} = \mathbf{w}^k - \eta\big(\hat{\mathbf{m}}^k/(\sqrt{\hat{\mathbf{v}}^k}+\epsilon) + \lambda\mathbf{w}^k\big)$.

A typical benchmark. Consider an error surface with a long, narrow valley (or a saddle) where at each point one component of the gradient is almost zero and the other is the useful one. Plain SGD gets to the optimum but takes a long time, needing many function evaluations; momentum is faster, since accelerating along a consistent direction speeds up the descent; Nesterov behaves similarly with less bouncing; adaptive methods such as Adagrad and Adadelta are the fastest, since by looking at the history of the gradient of each weight they automatically speed up the descent in the useful direction.

In practice. About 90% of the time models are trained with Adam or AdamW: one provides an initial learning rate (Adam's default is 10^{-3}; for fine-tuning or large models smaller values such as 10^{-4} or 10^{-5} are common) and the method adapts it, usually working out of the box. Do not be surprised if exercises and challenges do not use plain SGD. More advanced methods (e.g., second-order ones) are studied in courses on numerical optimization; they are mostly unpractical when the function has millions of parameters.

Chapter 7

Sequence Modeling: Recurrent Neural Networks and LSTM

This chapter introduces the basics of *sequence modeling* with neural networks. It is not a full treatment of the topic (more advanced material, such as attention and transformers, is beyond our scope), but it provides the background needed to process time series and other sequential data, such as text.

7.1 From static to dynamic models

The feedforward networks studied so far are *static*: given an input A they produce an output A'; given B, an output B'; given C, an output C'; and if the order of the inputs is changed, each input still produces exactly the same output. There is no sequence, no memory, no dynamics: each time, the (trained) model behaves as if it were restarted from scratch.

Remark 7.1 (Networks are deterministic) *It is sometimes said that neural networks give "stochastic" outputs. This is wrong: given the same input ten times, a network returns ten times the same output (possibly a wrong one). Barring a cosmic ray flipping a bit in GPU memory, the output is fully deterministic.*

Often, however, the input $\mathbf{x}$ is one element of a *sequence*, and the order matters: reversing the letters of a word or the words of a sentence destroys its meaning. If the output at time t depends not only on the current input but also on past inputs, a static model is not sufficient: the model must be able to reason on the history of the inputs. We speak of *time* for convenience, but any ordering works: sequences of letters, words, names, actions, ...

We denote by $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \ldots$ the input vectors at steps $0, 1, 2, \ldots$ Two families of approaches exist.

7.1.1 Memoryless models

The first idea that comes to mind is to give the system a memory; but this is not strictly necessary. A very simple approach to account for, say, the last 10 steps is to concatenate them and feed them as a single input.

Autoregressive (linear) models. Classical system identification uses linear models such as AR, MA, ARMA, ARIMA and SARIMA (seasonal ARIMA). For instance, the next value can be predicted as a weighted sum of the two previous ones, $\mathbf{x}_2 \approx w_0\mathbf{x}_0 + w_1\mathbf{x}_1$. The number of past steps used is called the *lag* or number of *delay taps* (a term from telecommunications, where a channel introduces memory and time-correlated noise, and the transmitted symbol is reconstructed looking at a few past steps).

Feedforward networks with a time window. Replacing the weighted sum with a feedforward network with a non-linear hidden layer generalizes linear autoregressive models: $\mathbf{x}_2$ is predicted from $(\mathbf{x}_0, \mathbf{x}_1)$, $\mathbf{x}_3$ from $(\mathbf{x}_1, \mathbf{x}_2)$, $\mathbf{x}_4$ from $(\mathbf{x}_2, \mathbf{x}_3)$, and so on. Training does not happen online: a sequence is rewritten as

a standard dataset of input–output pairs, $((\mathbf{x}_0, \mathbf{x}_1), \mathbf{x}_2)$, $((\mathbf{x}_1, \mathbf{x}_2), \mathbf{x}_3)$, ... (the first steps, which have no history, cannot be targets), and a standard feedforward network is trained on it. Rewriting a sequence as a windowed dataset is one of the first steps in time series modeling.

7.1.2 Models with memory

Models with memory maintain a *hidden state*: each new input modifies the state, and the output is computed from the state (and possibly directly from the input, through a shortcut connection; shortcut connections are useful, among other things, against vanishing gradients, and will be studied in the computer vision chapters). This is nothing new.

Linear dynamical systems. In discrete time,

$$\mathbf{x}_{t+1} = A\,\mathbf{x}_t + B\,\mathbf{u}_t \qquad \text{(state equation)}, \tag{7.1}$$

$$\mathbf{y}_t = C\,\mathbf{x}_t \qquad \text{(output equation)}. \tag{7.2}$$

The state $\mathbf{x}$ is the memory, modified by the input $\mathbf{u}$ (the control action, in control theory); the output is what we measure, a function of the state (in some systems the input also affects the output directly, which corresponds to the shortcut connection). The state can be estimated with the Kalman filter. Replacing the linear maps A, B, C with neural networks makes the system non-linear; the crucial point is that the new state depends on the current input *and on the previous value of the memory*.

Hidden Markov models. The state is discrete and evolves through a transition matrix; the output is a stochastic function (a probability distribution) of the state; the most likely state sequence can be estimated with the Viterbi algorithm.

With neural networks the goal is not to estimate the state (which is the goal of Kalman filtering or Viterbi decoding), but to obtain correct predictions taking into account the past inputs.

7.2 Recurrent neural networks

A **recurrent neural network** (RNN) contains *feedback loops*: some neurons (the *context* or recurrent units) store a memory whose value is a function of the current input and of its own previous value, exactly like the state of a dynamical system.

Compared with a feedforward network on a fixed window, which has a *finite* memory (the output cannot depend on inputs older than the number of taps), a recurrent network ideally has an *infinite* memory: the state depends on the previous input and state, which depend on the input and state before, and so on back to the first input ever seen. The purpose of the feedback loop is precisely not to have to decide in advance how long the memory should be. (Why "ideally" will be clear shortly.) Loops as memory are familiar from digital electronics: a flip-flop, the basic element of digital memory, is a feedback loop that keeps or changes its stored bit depending on the input.

7.2.1 Equations

In its most complete form (an *Elman*-like architecture), the network has an input $\mathbf{x}^t$, a feedforward hidden layer h, a recurrent context layer c, and an output g:

$$c_b^t(\mathbf{x}^t, W_B^{(1)}, V_B^{(1)}) = c\left(\sum_{i=0}^{I} v_{bi}^{(1)}\, x_i^t + \sum_{b'=0}^{B} v_{bb'}^{(1)}\, c_{b'}^{t-1}\right), \tag{7.3}$$

$$h_j^t(\mathbf{x}^t, W_J^{(1)}, V_J^{(1)}) = h\left(\sum_{i=0}^{I} w_{ji}^{(1)}\, x_i^t + \sum_{b=0}^{B} v_{jb}^{(1)}\, c_b^{t-1}\right), \tag{7.4}$$

$$g^t(\mathbf{x}^t \mid \mathbf{w}) = g\left(\sum_{j=0}^{J} w_{1j}^{(2)}\, h_j^t(\cdot) + \sum_{b=0}^{B} v_{1b}^{(2)}\, c_b^t(\cdot)\right). \tag{7.5}$$

The recurrent loop simply delays by one step the value of the memory. Not all these components are mandatory: the network can be reduced down to a single recurrent neuron, but without recurrent neurons it is just a feedforward network.

History. Recurrent networks were studied extensively in the 1990s. In 1995 it was shown (Siegelmann and Sontag) that they can simulate any Turing machine, i.e., they are in principle equivalent to a computer. The architecture is often called the *Elman network*, after one of the authors who studied its training. Earlier architectures, where only the output was fed back as input (Jordan networks), were not truly recurrent in the same sense.

Remark 7.2 *A recurrent network is a deterministic system: there is no source of noise in it. One may assume that the data were recorded with noise (which justifies, e.g., training by least squares), but given the same initialization and the same input sequence the network always gives the same result.*

7.2.2 Backpropagation through time

If recurrent networks are so similar to feedforward ones, why were they less widespread? Because of training: backpropagation, and the forward–backward algorithm, require a *feedforward* network. With recurrent connections it does not work directly. The trick, given the dignified name of **backpropagation through time** (BPTT), is to *unroll* the network in time: copy the recurrent neurons once for each time step, $t, t-1, t-2, \ldots, 0$, for the whole length of the sequence. The unrolled network is feedforward (with inputs $\mathbf{x}^t, \mathbf{x}^{t-1}, \ldots, \mathbf{x}^0$) and backpropagation applies.

How many times to unroll? As many as the length of the sequence: 3 for a 3-sample sequence, 100 for a 100-sample one. However, if the sequences in a batch have different lengths, each of them requires a different unrolled model, which prevents parallelization. In **truncated BPTT** the network is unrolled for a fixed number U of steps (e.g., the longest or the shortest sequence in the batch, or a chosen window), so that all unrolled models are identical and all gradients and updates can be computed in parallel.

Weight sharing. In the unrolled network, the copies of the recurrent weights *must* remain identical. But after backpropagation the gradients of the different copies differ (the gradient of an earlier copy involves more factors), so even if the copies start equal they would diverge after one update. One solution is to update every copy and then replace them with their average. A more efficient, equivalent solution:

1. unroll the network for U steps;
2. initialize all the copies of a weight to the same value;
3. compute the gradient for every copy, and update the single shared value with the *average* of the gradients.

Since a single copy is kept and updated, these weights are said to be **shared** across time steps.

7.3 Long-term dependencies and the vanishing gradient

How long must the memory be? Consider the sentence "*Jane walked into the room. John walked in too. It was late in the day. Jane said hi to* __": the answer is "John" (or "him"), but to predict it the network must look back more than ten words. With text, and with time series, memories of 10, 20, 50, 100 steps or more may be needed.

In the 1990s recurrent networks trained with BPTT could not learn such long dependencies. A classic benchmark of the late 1990s: given a sequence of 0s and 1s, output 1 if there was at least one 1 in the previous 1000 inputs. Regardless of the size of the network, recurrent networks could not go beyond about 100 steps in the past. The reason is the vanishing gradient: unrolling for 100 steps gives a 100-layer feedforward network.

7.3.1 Analysis on a single recurrent neuron

Consider the simplest recurrent network: one input, one recurrent neuron, one output:

$$h^t = h\big(v\,h^{t-1} + w\,x^t\big), \qquad y^t = g(h^t), \tag{7.6}$$

a one-dimensional non-linear state system. For a sequence of length S, the derivative of the error at time t with respect to a weight collects contributions from all the past steps:

$$\frac{\partial E^t}{\partial w} = \sum_{k=1}^{t} \frac{\partial E^t}{\partial y^t}\,\frac{\partial y^t}{\partial h^t}\,\frac{\partial h^t}{\partial h^k}\,\frac{\partial h^k}{\partial w}, \qquad \frac{\partial h^t}{\partial h^k} = \prod_{i=k+1}^{t} \frac{\partial h^i}{\partial h^{i-1}} = \prod_{i=k+1}^{t} v\,h'\big(vh^{i-1} + wx^i\big). \tag{7.7}$$

Bounding the norms, with $\gamma_v \geq \|v\|$ and $\gamma_{h'} \geq \|h'(\cdot)\|$,

$$\left\| \frac{\partial h^t}{\partial h^k} \right\| \leq \big(\gamma_v\,\gamma_{h'}\big)^{t-k}. \tag{7.8}$$

This is an upper bound on how much the current output can influence (through the gradient) the state $t-k$ steps in the past.

- If $\gamma_v\gamma_{h'} < 1$ the bound goes to zero exponentially: *vanishing gradient*. With sigmoid or tanh, $\gamma_{h'} \leq 1$ (and ≤ 0.25 for the sigmoid): using these activations in the recurrent state was one of the reasons recurrent networks could not be trained to remember: after a few steps the gradient does not reach the past.
- Using a ReLU (derivative 1) fixes $\gamma_{h'} = 1$, but this is only half of the story: the recurrent weight still matters. If $|v| < 1$ the gradient still vanishes ($v^{t-k} \to 0$); if $|v| > 1$ it *explodes*, the gradient grows going back in time and the optimization oscillates without converging. Only $v = 1$ works.

This result (around 1997) explained the vanishing gradient, suggested activation functions with derivative close to one, and showed that long memories cannot be obtained with generic recurrent connections. The only trainable recurrent unit for long memories is a linear unit with a self-loop of weight one — but such a unit merely *integrates* its (weighted) input: it accumulates whatever enters and cannot change it afterwards.

7.4 Long Short-Term Memory

The best-known solution came in 1997 from Sepp Hochreiter and Jürgen Schmidhuber: the **Long Short-Term Memory** (LSTM), still today the most used recurrent architecture. The idea is precisely to use a linear unit with a unit self-loop, called the **constant error carousel** (its derivative is always one, so the error circulates unchanged), and to *learn gates* deciding *when* to write into the memory, *what* to keep and *when* to read from it. The original block had a *write* gate, a *keep* gate and a *read* gate. Gradients can now flow through time without vanishing: to decide at time 1000 whether something should have been written at time 0, the error is backpropagated through the carousel, whose derivative is one.

7.4.1 The modern LSTM cell

We use the notation of a well-known tutorial on LSTMs (C. Olah's blog). First, the plain recurrent cell: the input $\mathbf{x}_t$ is concatenated with the previous state $\mathbf{h}_{t-1}$, and

$$\mathbf{h}_t = \tanh\big(W[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}\big), \tag{7.9}$$

i.e., a non-linear function of a weighted sum of the input and of the previous value. If the input has I components and there are H recurrent neurons, W is an $H \times (H + I)$ matrix.

The LSTM has a *cell state* $\mathbf{C}_t$, the memory line, which at each step is multiplied by one value and added to another; these values are controlled by gates. Let σ be the sigmoid.

Candidate value and input gate. What could be written into memory is a non-linear function of the input and of the previous output:

$$\tilde{\mathbf{C}}_t = \tanh\big(W_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C\big). \tag{7.10}$$

It is not necessarily written: the *input gate*

$$\mathbf{i}_t = \sigma\big(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i\big) \tag{7.11}$$

multiplies it: if $\mathbf{i}_t \approx 1$ the candidate is written, if $\mathbf{i}_t \approx 0$ it is not. The candidate is always computed; the gate decides whether it is the right moment to write it.

Forget gate. The input term can only *add* to the memory, never erase it. Erasing is the job of the *forget gate*:

$$\mathbf{f}_t = \sigma\big(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f\big)\ : \tag{7.12}$$

if $\mathbf{f}_t \approx 1$ the memory is kept, if ≈ 0 it is erased. Being sigmoids, gates are not exactly binary: one could keep half of the memory and add half of the input, although this is not common.

Memory update.

$$\boxed{\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t.} \tag{7.13}$$

With $\mathbf{f}_t = 0$ and $\mathbf{i}_t = 0$ the memory is erased; with both equal to one, the memory is kept and the new value added (integration); every combination is possible.

Output gate. The output is not the memory itself but a function of it, gated:

$$\mathbf{o}_t = \sigma\big(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o\big), \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t), \tag{7.14}$$

which adds non-linearity and allows deciding when to use the content of the memory (the role of the original read gate).

These formulas are all that is needed: in PyTorch one writes them (or uses the built-in layer) and trains with backpropagation through time. How the gates are used is up to training: for a task that needs only a one-step memory, the network can learn to erase and rewrite the memory at every step. The LSTM solved the 1000-step benchmark, one of its first success stories.

Efficiency. LSTMs are still widely used for time series: transformers are not always an option (e.g., on a microcontroller). At inference an LSTM step costs a few matrix multiplications, sigmoids, hyperbolic tangents, two element-wise products and a sum. They are very parameter-efficient: recognizing time sequences takes thousands to at most about a million parameters, not billions.

7.4.2 Gated Recurrent Unit

The **GRU** is a variant with fewer parameters: it merges the forget and input gates into a single *update gate* $\mathbf{z}_t$, has no separate cell state nor output gate (the memory is read directly), and has a *reset* gate $\mathbf{r}_t$ deciding how much the previous state contributes to the candidate:

$$\mathbf{z}_t = \sigma\big(W_z[\mathbf{h}_{t-1}, \mathbf{x}_t]\big), \qquad \mathbf{r}_t = \sigma\big(W_r[\mathbf{h}_{t-1}, \mathbf{x}_t]\big), \tag{7.15}$$

$$\tilde{\mathbf{h}}_t = \tanh\big(W[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t]\big), \qquad \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \tag{7.16}$$

Writing implies erasing: the new memory is always a convex combination of the previous memory and of what is written. About ten years ago GRUs were often preferred to LSTMs in natural language processing, having fewer parameters (hence less prone to overfitting) with similar performance. Today the saving is not significant; some papers find GRUs more convenient, others LSTMs. In practice, one chooses an LSTM or a GRU layer and its number of units, i.e., the size of the hidden state vector (the number of "lines" of memory): with 10 units, the state is a vector of 10 elements updated with the logic above.

7.5 Architectures and tricks

Stacked (hierarchical) recurrent layers. Recurrent layers can be composed in arbitrary graphs: an input feeds a first recurrent layer (RNN, LSTM or GRU), whose output feeds a second recurrent layer, then dense layers and finally, e.g., a softmax. The first memory evolves at the rate of the input; the second one at the rate of the first memory, giving a sort of hierarchical representation in time (a "fast" and a "slow" memory, both of which can be connected to the output). At each time step the network is the same, with a different input. At each time step the content of the memory is a *feature* summarizing all the past inputs.

Bidirectional RNNs. If the whole sequence is available (not online prediction), processing it in one direction gives at each step a representation of the past only, ignoring the future. A *bidirectional* RNN/LSTM traverses the sequence in both directions; at each time step the two representations (from the past and from the future) are concatenated. This cannot be used online (the future of the current sample is unknown), but it is very convenient for fixed sequences, e.g., to classify whether a sentence is positive or negative after reading all of it. In a framework, one simply uses a bidirectional layer (two LSTMs, whose outputs are concatenated).

Initial state. How should the memory be initialized at time zero? The simplest choice is zero. But the error can also be backpropagated to the initial state, which is just another input of the unrolled network: the initial state can be treated as a set of *trainable parameters*, initialized at random and updated by backpropagation like the weights. This sometimes improves results at inference time. (A bidirectional layer has, presumably, a learned initial state for each direction.)

Which model? Feedforward with time window, RNN, LSTM or GRU: it is a design choice (a hyperparameter) that depends on the problem. For very short memories a feedforward network is enough, but its input, hence its number of parameters, grows with the window. For short histories, recurrent networks are very parameter-efficient and easy to train; for long memories, use LSTMs (or GRUs). One has to study the data, understand the problem and design the model to fit it.

7.6 Sequence-to-sequence problems

So far we considered fixed-size inputs and outputs (static data). With recurrent networks, inputs and/or outputs can be sequences:

- **one-to-many** (fixed input, sequence output): e.g., *image captioning*: an image is the input and a recurrent network produces a description word by word ("a group of young people playing a game of frisbee"); the memory is needed to know what has already been said (having said "a", it is time to say "group");
- **many-to-one** (sequence input, fixed output): e.g., *sentiment analysis*: "I really like the color of my new phone" is positive, "I didn't really enjoy the camera of my phone" is negative. The whole sequence is read and the class is predicted at the end (a perfect case for bidirectional networks). Note that the loss is computed only on the last output, and the error must be backpropagated from there through the whole sequence (the library takes care of it, but the setting is very different from the next one);
- **many-to-many**, either *delayed* (read the whole input sequence, then produce the output sequence) or *synchronized* (an output for each input).

Machine translation. Translating element by element (synchronized many-to-many) might seem more accurate, but the delayed version is the one used. When translating, one first reads the whole sentence and then translates its *meaning*: languages may be very far apart in structure (think of Chinese and English), so a word-by-word translation does not work. The **encoder–decoder** (sequence-to-sequence) model reads the input sentence with an *encoder* RNN, whose final state summarizes its meaning; a special end-of-sequence token signals that it is time to start translating, and a *decoder* RNN produces the translation one word at

a time. At each step the decoder outputs (e.g., via a softmax over the vocabulary) a pointer to the next word, and the word produced is given as input to the next step of the decoder, starting from a special *beginning-of-sequence* token ("I saw a cat on a mat" → `<GO>` → translated words → `<EOS>`).

Representing words. In the example each word is represented as a vector. The classic approach is *one-hot encoding*: with a vocabulary of, e.g., 400 000 words, each word is a vector of zeros with a single one at the index of the word in the vocabulary ($0010\ldots0$ is the third word, $100\ldots0$ the first). This is not a smart representation; Chapter 8 presents better ones.

7.7 Lab: time-series classification with recurrent networks

This laboratory applies recurrent networks to a *many-to-one* problem: classifying the physical activity a person is performing from the signal of a wearable accelerometer.

7.7.1 The dataset and its preparation

The *Activity Recognition* dataset contains, for each sample, the three accelerations along the x, y, z axes, a timestamp, the identifier of the user (36 users) and the activity being performed (walking, jogging, sitting, standing, going upstairs, going downstairs). The first steps are always the same and should never be skipped:

1. **Inspect the data**: look at the columns, their types and their ranges; plot the signals of some users for each activity. The raw data are stored as `float64`; converting them to `float32` halves memory with no loss of useful precision (networks are trained in single or even half precision).
2. **Check for anomalies before normalizing**: outliers, missing values or broken readings must be handled before computing statistics, otherwise a single absurd value ruins the normalization of the whole feature.
3. **Look at the class distribution**: the classes are strongly *unbalanced* (walking and jogging dominate). Accuracy is then misleading, since a model predicting always the majority class has high accuracy; we use the (macro) **F1 score** instead.

Splitting without leakage. Consecutive samples of the same user performing the same activity are almost identical. If windows of the same recording ended both in training and in test, the test score would be overly optimistic (*data leakage*). We therefore split *by user*: we create a unique identifier combining user and activity to keep the recordings separated, and assign, e.g., 5 users to validation and 5 to test, the remaining ones to training. This is the "leave-group-out" idea of Chapter 5: the model will be used on *new* people, so it must be evaluated on people it has never seen. The labels are mapped to integers with a dictionary (e.g., `{'Walking': 0, 'Jogging': 1, ...}`).

Normalization. Statistics are computed on the training set only and then applied to validation and test. Here we use min–max normalization,

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}},$$

with $x_{\min}, x_{\max}$ the training minimum and maximum of each feature; alternatives are the z-score $x' = (x-\mu)/\sigma$ and the *robust scaler* (median and interquartile range, less sensitive to outliers).

Windowing. A recording is a long sequence; the network is fed fixed-length *windows* of it. Two hyperparameters define the procedure:

- the **window size** (e.g., 200 time steps), i.e., how much history the model sees to decide;
- the **stride** (e.g., 50), i.e., how much the window moves between two consecutive samples. A stride smaller than the window gives overlapping windows (here 75% overlap): more training samples, but more correlated with each other. Stride equal to the window gives disjoint windows. It is a trade-off between quantity and redundancy of data.

At the end of a recording the last window may be incomplete: one can either *pad* it (e.g., with zeros) or discard it. Each window inherits the label of its recording. The resulting tensor has shape `(batch, window, features)` = `(B, 200, 3)`. The batch size is typically a power of two, chosen as large as the GPU memory allows (and not larger than useful for generalization).

Listing 7.1: Building windows from each recording.

```
def build_sequences(df, window=200, stride=50):
    X, y = [], []
    for uid in df['id'].unique():           # one recording = user+activity
        rec = df[df['id'] == uid]
        data = rec[['x', 'y', 'z']].values
        label = rec['activity'].values[0]
        pad = (window - len(data) % window) % window
        data = np.concatenate([data, np.zeros((pad, 3), 'float32')])
        for idx in range(0, len(data) - window + 1, stride):
            X.append(data[idx:idx + window])
            y.append(label)
    return np.array(X), np.array(y)
```

7.7.2 A configurable recurrent classifier

We define a single module whose recurrent core can be selected among `nn.RNN`, `nn.LSTM` and `nn.GRU`, with configurable number of stacked layers, hidden size, dropout and bidirectionality. The classifier is a linear layer applied to the output at the *last time step* (many-to-one).

Listing 7.2: Recurrent classifier with selectable cell.

```
class RecurrentClassifier(nn.Module):
    def __init__(self, input_size, hidden_size, num_layers, num_classes,
                 rnn_type='GRU', bidirectional=False, dropout_rate=0.2):
        super().__init__()
        rnn_map = {'RNN': nn.RNN, 'LSTM': nn.LSTM, 'GRU': nn.GRU}
        self.rnn = rnn_map[rnn_type](
            input_size, hidden_size, num_layers=num_layers,
            batch_first=True, bidirectional=bidirectional,
            dropout=dropout_rate if num_layers > 1 else 0)
        out = hidden_size * (2 if bidirectional else 1)
        self.dropout = nn.Dropout(dropout_rate)
        self.classifier = nn.Linear(out, num_classes)

    def forward(self, x):                   # x: (B, T, F)
        out, _ = self.rnn(x)                # out: (B, T, out)
        last = out[:, -1, :]                # output at last time step
        return self.classifier(self.dropout(last))
```

Some remarks on the design:

- The `dropout` argument of the recurrent layer is applied *between stacked layers* (it has effect only with ≥ 2 layers); a separate `nn.Dropout` before the classifier regularizes the dense head. Dropout "through time" on the recurrent connections (*recurrent dropout*) is a different technique, applying the same mask at every time step to the hidden state.
- With `bidirectional=True`, two recurrent layers read the window forward and backward and their outputs are concatenated: the feature size doubles, and so do the recurrent parameters. (For the backward direction, the output at the last index has seen only the last sample; using the final hidden states `h_n` of both directions is a cleaner alternative.)
- A small custom `recurrent_summary` function prints layer by layer the output shapes and the number of parameters, since the standard summary utilities do not handle the tuple outputs of recurrent layers.

7.7.3 Experiments and lessons learned

Training uses cross-entropy, AdamW, early stopping on validation F1, and logging to TensorBoard to compare the runs side by side. The experiments are instructive:

- A **vanilla RNN** (about 50k parameters) collapses into predicting essentially the two majority classes, confusing them with one another: it is stuck in a poor local minimum, and the vanishing gradient prevents it from exploiting the long window.
- A **bidirectional RNN** is even *worse* (it predicts almost always class 0): more parameters do not mean a better model. *Capacity* (how many functions the model can represent) is not *capability* (what training actually manages to find).
- An **LSTM** stays stuck on a plateau for many epochs and then suddenly starts improving; a **bidirectional LSTM** (about 0.5M parameters) reaches about 82% validation F1.
- A **GRU** converges faster (best epoch around 41) to similar values; the bidirectional GRU behaves similarly.

The LSTM behaviour recalls the phenomenon called *grokking*: a network that for a long time seems not to learn (or to only memorize) may suddenly generalize after many more epochs. The practical lesson is to keep the early-stopping **patience high** with recurrent models, and to always **save the trained models** (and the best weights) so that runs do not have to be repeated.

Cross-validation for time series. To tune hyperparameters reliably, a single split is noisy. We use a *k-shuffle-split* cross-validation over *users*: at each of the k iterations a random group of users is held out for validation. Crucially, *all* data preparation (normalization statistics, windowing) is done *inside* the loop, using only the training users of that iteration; computing it once on everything would leak information. Tuning the window size and the stride this way (a window of 400 turned out better than 200) raises the validation F1 to about 90.5%, and the final model scores about 87% on the test users.

Start from simple baselines. Before deploying a large model, always compare it with trivial baselines. A cautionary tale from forecasting: on a benchmark where a prize-winning transformer model (Informer) was proposed, a baseline with *zero* parameters that simply repeats the last observed values beat it, and nearly linear models turned out to be competitive with the most complex architectures. Complexity must be justified by results.

7.8 Lab: time-series forecasting

In forecasting the target is not a class but the *future of the series itself*: a many-to-many problem.

7.8.1 Data and sequences

We use an air-quality/weather dataset with three variables (dew point, temperature, pressure), sampled regularly and with evident seasonality. Temperature is the easiest to predict, being the most regular; one could also study it with classical tools (seasonal decomposition into trend, seasonality and residual; ARIMA models).

Chronological split. With forecasting, data cannot be shuffled before splitting: the model must be evaluated on the *future* with respect to what it was trained on. The series is split *chronologically*: the first part for training, then validation, and the last part for test. This mimics deployment (*backtesting*); more robust estimates are obtained using several validation chunks at different times.

Window, stride and telescope. Besides the window (the past the model sees) and the stride, we have the **telescope**: the number of future steps to predict (here 50). Input and target are both slices of the series:

Listing 7.3: Building input/target pairs for forecasting.

```
def build_sequences(x, window=200, stride=10, telescope=50):
    X, Y = [], []
    for idx in range(0, len(x) - window - telescope, stride):
        X.append(x[idx:idx + window])                          # past
        Y.append(x[idx + window:idx + window + telescope])     # future
    return np.array(X), np.array(Y)
```

The stride is used to subsample the *training* set; for validation and test we use stride 1, to evaluate at every possible starting point. Both inputs and targets have shape `(batch, time, features)`: `(B, 200, 3)` and `(B, 50, 3)`.

7.8.2 Direct forecasting

The simplest model is a recurrent *encoder* whose last hidden state is mapped by a linear layer to all the future values at once: a "many-to-150" network, since telescope $\times$ features $= 50 \times 3 = 150$ outputs, reshaped to `(B, 50, 3)`.

Listing 7.4: Direct multi-step forecaster.

```
class DirectForecaster(nn.Module):
    def __init__(self, n_feat=3, hidden=128, telescope=50):
        super().__init__()
        self.telescope, self.n_feat = telescope, n_feat
        self.rnn = nn.GRU(n_feat, hidden, batch_first=True)
        self.head = nn.Linear(hidden, telescope * n_feat)
    def forward(self, x):
        _, h = self.rnn(x)                  # h: (1, B, hidden)
        y = self.head(h[-1])                # (B, 150)
        return y.view(-1, self.telescope, self.n_feat)
```

The loss is the mean squared error (regression, Chapter 4). Plotting the error as a function of the forecast step shows the expected behaviour: the error *grows with the horizon*, the far future being harder to predict than the near one.

Learning-rate scheduling. This lab also introduces schedulers, which change the learning rate during training:

- **ReduceLROnPlateau**: monitors a metric (`mode='min'` for a loss) and multiplies the learning rate by a `factor` (e.g., 0.1) when it does not improve for `patience` epochs, down to a `min_lr`. Its patience must be *smaller* than the early-stopping patience, otherwise training stops before the learning rate is ever reduced;
- **step decay**: reduce the rate by a factor every fixed number of epochs;
- **exponential decay**: multiply by $\gamma < 1$ at every epoch;
- **cosine annealing**: the rate follows a cosine from its maximum to its minimum (possibly with restarts). The name comes from metallurgy: annealing heats a metal and cools it slowly so that atoms settle into a low-energy configuration; similarly a large rate lets the optimizer explore, and slowly reducing it lets it settle in a good minimum;
- **warm-up**: start with a small rate and increase it during the first epochs, to avoid destabilizing the randomly initialized network (common with Adam and transformers).

The general principle is the trade-off between *exploration* (large steps, early in training) and *exploitation* (small steps, refining the solution).

7.8.3 Autoregressive forecasting

Alternatively, a model can be trained with a short telescope (e.g., 10 steps) and applied *autoregressively*: its predictions are appended to the input window and fed back to predict the next 10 steps, and so on, 5 times to reach the 50-step horizon. Errors accumulate along the way, since the model is fed its own (imperfect) outputs. Note that the training losses of models with different output lengths are not comparable: the comparison must be done on the same 50-step test targets.

7.8.4 Sequence-to-sequence with attention

The most elaborate model is an *encoder–decoder* (Section on sequence-to-sequence problems):

- a **bidirectional encoder** reads the input window and produces an output vector for each time step;
- an **attention** mechanism: at each decoding step, a linear layer scores each encoder output with respect to the current decoder state, a softmax turns the scores into weights summing to one, and the weighted sum of the encoder outputs gives a *context vector*, concatenated to the decoder input. The decoder thus decides at every step which parts of the past to look at;
- a **decoder** RNN that generates the future one step at a time, followed by a fully connected output layer.

(The input gate of an LSTM is itself a form of attention: it weights how much of the input to let in.) The attention weights can be plotted, but their interpretation as "what the model looks at" must be taken with caution.

Teacher forcing. During training, the decoder input at each step can be either its own previous prediction or the *ground-truth* previous value. Feeding the ground truth (*teacher forcing*) stabilizes and speeds up training, but creates a mismatch with inference, where the ground truth is not available. A compromise is to apply teacher forcing with some probability (e.g., 20%) at each step. Training of these models is noisy, and the code is long: a practical suggestion is to use large language models to have the code explained line by line, while always checking it.

Chapter 8

Autoencoders and Word Embeddings

In Chapter 7 we saw recurrent networks reading and producing sequences of words. Those words must be represented as vectors, and one-hot encoding — a vector of 400 000 entries with a single relevant bit — is clearly far from optimal. This chapter presents *word embeddings*, the standard way to represent text, and uses them as an opportunity to introduce several important concepts: *unsupervised learning* with neural networks, *autoencoders*, and the very idea of an *embedding*.

8.1 Unsupervised learning, revisited

Recall the paradigms of Chapter 1. Given samples $\mathbf{x}_1, \mathbf{x}_2, \ldots$ (records of a database, images, any vectors):

- in **supervised learning** each input has a target t_n (a class or a value), and we learn a function producing outputs y as close as possible to t on new inputs;
- in **unsupervised learning** we exploit regularities in the data to build a *representation* that can be used for prediction or classification. It is not about matching a target but about representing the data differently. Clustering is one instance: it represents the data by groups (e.g., an input is represented by the index of its cluster);
- in **reinforcement learning** actions produce effects in an environment, which returns rewards, and we learn the actions maximizing the long-term reward.

The practical test to decide whether an algorithm is supervised or unsupervised: *do we need an external labeler, besides the dataset, to define the learning task?* (If the targets are already in the dataset, a labeler was involved earlier.) If not, the task is unsupervised.

8.2 Autoencoders

Definition 8.1 (Autoencoder) *An autoencoder is a neural network trained to reproduce its input at its output,* $g(\mathbf{x} \mid \mathbf{w}) \approx \mathbf{x}$*, through a hidden layer (the* bottleneck*, or latent code) with significantly fewer units than the input. The part from the input to the bottleneck is the* encoder*, the part from the bottleneck to the output is the* decoder*.*

Learning the identity looks useless; but forcing the signal through a bottleneck means *compressing* the input into a lower-dimensional space such that the decompression is as close as possible to the original. Training requires only the inputs, which makes autoencoders unsupervised. The loss is a simple *reconstruction loss*, e.g., the squared error:

$$E = \sum_{n=1}^{N} \left\| \mathbf{x}_n - g(\mathbf{x}_n \mid \mathbf{w}) \right\|^2. \tag{8.1}$$

Sparsity. One may additionally require that as many hidden units as possible output zero, adding a term on the *activations* of the bottleneck:

$$E = \sum_{n=1}^{N} \left\| \mathbf{x}_n - g(\mathbf{x}_n \mid \mathbf{w}) \right\|^2 + \lambda \sum_{n=1}^{N} \left\| \mathbf{h}(\mathbf{x}_n \mid \mathbf{w}) \right\|_1. \tag{8.2}$$

Note the difference with L1 regularization, which penalizes the norm of the *weights*: here we penalize the norm of the *outputs* of the hidden neurons (a somewhat unusual loss that conditions on the hidden layer, but still differentiable). λ trades off reconstruction and sparsity. If only one hidden neuron is allowed to be active, similar inputs collapse onto the same neuron and are reconstructed by (a non-linear function of) their average: a non-linear version of k-means clustering, where each hidden neuron reacts to one group of inputs. Once again: *the loss function tells the task*; to understand what a network does, look at its output and its loss, not at its architecture.

Applications. A simple trick — a bottleneck — enables many uses:

- **non-linear compression**: e.g., encode a 128-dimensional vector into 64 numbers and decode it back; the encoder and the decoder can be placed at the two ends of a communication channel, transmitting the compressed version (neural networks have been studied as non-linear compression systems);
- **non-linear dimensionality reduction**, a sort of non-linear principal component analysis, and **feature extraction**: the latent code is a learned embedding usable as features;
- **pre-training**: when only a few samples are labeled, train an autoencoder on all the data, cut away the decoder, and train the supervised model in the lower-dimensional latent space;
- **denoising**: a model trained on clean data maps a noisy input to the latent representation closest to the data it has seen, which does not include the noise; the reconstruction contains only the part of the signal the model was trained on, rejecting the noise. Even better, one can train explicitly as a denoiser: add noise to the input and ask the autoencoder to reconstruct the *clean* signal;
- **anomaly detection**: inputs that the model cannot reconstruct well (large difference between input and output) are flagged as anomalies;
- **data generation**: the decoder maps a latent vector to a sample. A plain autoencoder is deterministic; *variational* autoencoders put a probability distribution on the latent units and sample from it, generating different outputs (image generation is beyond the scope of this book);
- **word embeddings**: the idea behind them is very close to an autoencoder (without being exactly one), as we now see.

8.3 The problem of representing words

Words are sequences of characters with a meaning, but computationally they are *items of a vocabulary*: "cat" is item 537, "dog" item 143, just because the vocabulary has been sorted that way; agreeing on the vocabulary, "word 7" identifies a word. With one-hot encoding, a text is a sequence of extremely sparse vectors: with a (small) vocabulary of 100 000 words only one number out of 100 000 is non-zero (realistic vocabularies have 400 000 to a million words). The space is used very badly: a space that could represent a huge number of configurations is used to represent as many items as there are words. Audio spectra and images, by contrast, are dense: every value carries information, even if extracting the meaning is hard.

Bag of words. To represent a whole document, take a vector as large as the vocabulary and set to one the positions of the words appearing in the text. This reduces sparsity, but a text of 1000 distinct words still has only 1000 non-zeros out of a million. Refinements weigh each word by its frequency in the document relative to its frequency across documents (*TF-IDF*: term frequency–inverse document frequency). Still, the representation is sparse and high-dimensional.

The curse of dimensionality. Learning a probability distribution in a high-dimensional space requires a number of samples that grows exponentially with the dimension, otherwise the space cannot be filled. Moreover, in very high dimensions Euclidean distances become meaningless: all points tend to be at the same distance from each other.

No notion of similarity. Consider the sentences

"*Obama speaks to the media in Illinois*" and "*The President addresses the press in Chicago*".

They have essentially the same meaning (one more specific about the president, the other about the city). With one-hot encoding, "speaks" and "addresses", "Obama" and "President", "Illinois" and "Chicago" are just different items: their vectors are *orthogonal* (zero cosine similarity), the Euclidean distance between any two distinct words is always $\sqrt{2}$, and the Hamming distance always 2. Every pair of different words is equally different.

One-hot and bag-of-words representations did work, more or less, until about 2015 (document classification with bags of words was common). But once this problem is fixed, results get much, much better.

8.3.1 Distributed representations

The goal is a representation in a (much) lower-dimensional space where words with similar meaning are mapped to nearby points, so that moving a little from a word yields a similar word, and where relations are regular: *king*→*queen* should be the same displacement as *man*→*woman*; *walking*→*walked* as *swimming*→*swam*; each capital should be at a similar offset from its country.

In a **sparse** (local) representation a word such as "home" activates one dimension and all others are zero. In a **distributed** (dense) representation the word is spread over all dimensions: "Obama" goes from $(0, \dots, 0, 1, 0, \dots, 0)$ to something like $(0.12, \dots, -0.25)$, mapping a vocabulary of 10^6 items into a space of a few hundred dimensions (say 100–1000). The desired transformation combines:

- **compression** (dimensionality reduction), against the curse of dimensionality — hence the autoencoder background;
- **smoothing**: from discrete 0/1 values to a continuous representation, where small moves correspond to small changes of meaning;
- **densification**: few dimensions, densely populated, instead of a huge, empty space.

8.4 Neural language models

As often happens in science, the solution came from trying to solve a different problem: *language modeling*.

Definition 8.2 (Language model) *A language model assigns a probability to every sentence (sequence of words) $w_1, w_2, \dots, w_T$ of a language.*

In any language some sentences are likely and others essentially impossible; knowing the probability of every sentence means having a model of how the language behaves. Approaches include local representations (n-grams, bag of words) and continuous ones (latent semantic analysis, latent Dirichlet allocation, distributed representations); the topic is the subject of natural language processing courses. By the chain rule of probability,

$$P(w_1, \dots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \dots, w_{t-1}) = P(w_T \mid w_1, \dots, w_{T-1})\, P(w_{T-1} \mid w_1, \dots, w_{T-2}) \cdots P(w_1). \tag{8.3}$$

The last factor is the distribution of the first word of a sentence: articles such as "a" or "the" are likely, "zoo" much less (a probability distribution, not a rule). The full model is intractable: it needs a distribution over the vocabulary conditioned on every possible sequence of previous words; with a vocabulary of 400 000 words and sentences of 20 words, $400\,000^{20}$ configurations, which cannot even be stored, let alone estimated (one would need a dataset much larger than that).

N-grams. The next word probably does not depend on the first word said today, but on the last few: approximate

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}), \tag{8.4}$$

i.e., model the language by the probabilities of *n-grams*. Even trigrams over 400 000 words give $400\,000^3$ entries. Such models (e.g., Katz's back-off model, 1987) required smoothing tricks to deal with the many n-grams with zero counts. With $n = 10$ and 10 000 unique words, the model lives in a 10-dimensional hypercube with 10^4 slots per dimension, 10^{40} parameters, far more than modern large models; the unit probability mass is spread over all these slots, all nearly zero, and even if data could fill it the model would be useless. Using the whole web as training set does not help, because the number of unique words grows to millions (and explodes with multiple languages). In practice, contexts were limited to two previous words (trigrams). Today's experience with large language models shows how much context matters; a trigram model is like querying a chatbot with a two-token prompt. Still, it was the state of the art.

8.4.1 The Neural Network Language Model

In 2003 Yoshua Bengio and colleagues (Bengio later received the Turing Award together with LeCun and Hinton for deep learning) proposed to predict the next word from the previous $n-1$ words with a neural network: being continuous, the model does not have a hypercube to fill, and the probability is distributed among the parameters of the network, allowing larger n. The task is still unsupervised: any text can be used, no labeler is needed. It is not exactly an autoencoder (a word is predicted from its neighbours, not from itself), but the spirit is similar.

The architecture has three stages:

1. **projection**: each of the $n-1$ context words, one-hot encoded, is multiplied by a projection matrix C of size $|V| \times m$ ($|V|$ the vocabulary size, m the chosen embedding size). Multiplying a one-hot vector by C simply selects one row: each word is mapped to an m-dimensional vector, and the content of C is learned. The $n-1$ embeddings are concatenated into a vector of size $(n-1)m$, a distributed encoding of the context;

2. **compression**: a hidden layer ($h = 500$–1000 neurons in the original models, tanh);

3. **classification**: an output layer with $|V|$ units and a softmax, predicting the next word (one-hot). It is an "unsupervised classification" problem: one-hot previous words in, one-hot next word out, trained with the cross-entropy and backpropagation (stochastic gradient descent) on lots of text.

The computational complexity per training example is

$$Q = n \times m + n \times m \times h + h \times |V|, \tag{8.5}$$

dominated by the last term, the output layer, which depends on the vocabulary size.

Results. Compared with the best n-gram models (Katz trigram), perplexity improved by 24% on one benchmark (Brown corpus: 5-gram context, $m = 30$, $h = 100$) and 8% on another (AP News: 6-gram, $m = 100$, $h = 60$), training for three weeks on 40 CPU cores (GPUs were not yet used). The corpora had about 1.2 million and 14 million training words, with vocabularies of tens of thousands of words. The context was still short, but it beat every previous model. Bengio, having obtained the best neural language model of the time, stopped there.

8.5 word2vec

About ten years later, Tomáš Mikolov and colleagues at Google noticed the interesting part of Bengio's model: the projection matrix maps one-hot words into a dense space of fixed size — it *encodes words in a lower-dimensional distributed representation*. Forget about the language model and the classification task: focus on learning this projection, so that words retain their meaning in the low-dimensional space, and train it on as much data as possible. This is **word2vec** (2013).

Simplifications. The bottleneck of Bengio's model is training cost, especially the hidden and output layers. Hence:

- remove the hidden layer, keep only the projection;
- use the projection both to go down into the embedding space and, transposed, to go back up to the vocabulary (as presented here, a shared projection and "lift"; in the original implementation the output matrix is a separate set of weights, with the same role);
- replace next-word prediction with *predicting a word from its surroundings*.

The distributional hypothesis. A principle from 1950s linguistics (J. R. Firth): "*you shall know a word by the company it keeps*". There is enough information in the words surrounding a word to predict it, so a word could be replaced by the words around it. For instance, the missing name in "____, *who has been called an excellent player by Pelé at the age of just 22, had scored 40 goals in 58 internationals, occasionally as an attacking midfielder, and was called a true phenomenon*" (Neymar) can be guessed from its context.

Two architectures:

- **Continuous bag of words (CBOW)**: remove one word and predict it from the n surrounding words ($n/2$ before and $n/2$ after). Each context word is one-hot encoded and projected with the shared matrix, and the projections are *averaged* (not concatenated): like a bag of words, but summing embeddings instead of one-hot vectors. The average represents the average meaning of the context; projected back to the vocabulary size and passed through a softmax, it predicts the most likely missing word. Training is as for Bengio's model.
- **Skip-gram**: the reverse: from a word, predict each of the surrounding words.

These ideas later shaped large language model training: GPT-style models are trained to predict the next word (generative prediction); BERT, a transformer-based encoder, is trained to predict masked pieces of text.

Complexity. The projection matrix is still $m \times |V|$, but only one matrix needs to be trained (used directly for the projection and transposed for the output). With further tricks (hierarchical softmax, negative sampling; details in the original papers) the complexity per example becomes

$$Q_{\text{CBOW}} = n \times m + m \times \log_2 |V|, \tag{8.6}$$

far less than Bengio's $n \times m \times h + h \times |V|$. One can now train not on 14 million but on 6 billion words, with a vocabulary of a million words: with $m = 1000$, skip-gram takes about two days and CBOW about two and a half, whereas the neural network language model with only $m = 100$ would take about 14 days. This was a big jump in natural language processing. (The next big jump, not covered here, came with *transformers*, again from Google: they can be trained an order of magnitude faster than LSTMs on texts a thousand times larger — trillions of tokens, the whole web.) In predicting words, word2vec representations were about three times more accurate than the neural network language model.

8.5.1 Regularities of the embedding space

Something interesting happens. Projecting words from a 10^6-dimensional space to a 10^3-dimensional one, *without forcing any structure*, the embedding space exhibits regularities: approximately constant offsets between a capital and its country, a verb and its past tense, an adjective and its superlative, male and female forms (princess–prince, queen–king, heroine–hero, actress–actor, she–he, landlady–landlord, female–male, cow–bull, hen–cock), opposites. Mikolov and colleagues observed these structures (e.g., with a PCA projection) in a paper published a few months after word2vec. Vector arithmetic works:

$$\begin{aligned}
\text{vec(king)} - \text{vec(man)} + \text{vec(woman)} &\approx \text{vec(queen)},\\
\text{vec(Paris)} - \text{vec(France)} + \text{vec(Italy)} &\approx \text{vec(Rome)},\\
\text{vec(Windows)} - \text{vec(Microsoft)} + \text{vec(Google)} &\approx \text{vec(Android)},\\
\text{vec(Einstein)} - \text{vec(scientist)} + \text{vec(painter)} &\approx \text{vec(Picasso)},
\end{aligned}$$

and similarly he→his, she→her, copper→Cu, gold→Au, and so on.

Why does it work? Go back to the distributional principle. The vector of a word is (roughly) the average of the words that usually surround it. The vector of "king" contains the average of the words appearing around kings; removing the words typically associated with "man" (he, his, him, ...) and adding those associated with "woman" (she, her, ...) is like turning "*he ruled the country with his crown on his head*" into "*she ruled the country with her crown on her head*": the result is the average context of "queen". The embedding does not represent the word itself: it represents the words that usually surround it.

8.5.2 Using embeddings

Nobody uses one-hot encoding of words anymore: given a standard vocabulary and the learned matrix, each word of a text is mapped to its vector, and the sequence of vectors is fed, e.g., to an LSTM (Chapter 7). More advanced tokenizations split text not into words but into sub-word units (e.g., *byte-pair encoding*, chunks of letters similar to syllables, or character n-grams), but the principle is the same. Embeddings support many operations by simple vector arithmetic and nearest-neighbour search:

- *queries*: "restaurants in Mountain View that are not very good" ≈ vec(restaurant) + vec(Mountain View) + vec(not very good), then retrieve the reviews closest to it; vec(Czech) + vec(currency) has *koruna* among its nearest words; vec(Vietnam) + vec(capital) gives Hanoi, Ho Chi Minh, Vietnam, Vietnamese. This works for short queries of two or three words; summing the embeddings of a 100-word prompt does not give anything reasonable (this is a twelve-year-old model);
- *document similarity*: embed the words of two documents and compare, e.g., their averages or the word-by-word distances (the *word mover's distance* between the two "president" sentences above is small);
- *synonyms*: nearest neighbours of a word;
- *nearest-neighbour classification*: assign a text to the class whose representative word is closest. For a class "sad", the closest words include *saddened*, *heartbreaking*, *disheartening*, ...

Remark 8.1 (Why one-hot, and not binary codes?) *Why were classes and words encoded as one-hot vectors rather than, e.g., binary codes ($\lceil \log_2 K \rceil$ bits for K classes: $00, 01, 10, 11$ for four classes) or error-correcting codes (a few extra bits to correct a wrong bit)? One-hot is the simplest, and leads naturally to softmax outputs. Binary and error-correcting output codes were tried in the 1990s for classification without particular success: they are still discrete, and the outputs become correlated (the class is given by the whole output vector, not by a single neuron), so the classes are learned all together or not at all, making training harder. Then interest moved to document classification with bags of words, and between Bengio's 2003 model and the 2010s neural language modeling received little attention: neural networks were out of fashion (the years of support vector machines and of techniques such as latent Dirichlet allocation). After 2012 (ImageNet; convolutional networks, introduced in 1998, are as old as LSTMs) neural networks came back, and word2vec (2013) changed text representation.*

8.5.3 GloVe

In word2vec the regularities of the embedding space emerge "by chance", as a by-product of the correlations in the text. **GloVe** (Global Vectors for Word Representation, Pennington, Socher and Manning, 2014) instead *enforces* them, with a loss function built on global word co-occurrence statistics: words that co-occur should have related embeddings, and ratios of co-occurrence probabilities should be captured by vector differences. The resulting space shows the same kind of regularities (e.g., short–shorter–shortest, strong–stronger–strongest, dark–darker–darkest) and supports nearest-neighbour search: the nearest words to *frog* are *frogs*, *toad*, *litoria*, *leptodactylidae*, *rana* (Latin/Italian for frog), and *lizard*. The last one is "wrong" (a lizard is a reptile, a frog an amphibian), but understandable: the embedding is built from context, and frogs and lizards appear in similar contexts (small, green, four-legged animals).

8.6 A dictionary of layers

At this point we can look back at what we have built. A network is a composition of layers: input, fully connected (dense) layers, batch normalization, dropout, softmax output; recurrent layers (RNN, LSTM, GRU, and their bidirectional versions); possibly shortcut connections between non-adjacent layers. Embedding layers map tokens to dense vectors. The computer vision chapters that follow add another class of layers, *convolutional* layers, together with traditional ways of composing them (e.g., the U-Net for image segmentation). The rest of the book should be read as a natural continuation, with the focus on images.

Chapter 9

Digital Images, Linear Filtering and Image Classification

From this chapter on, the input of our neural networks will be *digital images*. **Computer vision** is the field that aims at giving computers (in our case, neural networks) the capability of gaining a *high-level understanding* of digital images or videos. Examples of what "high-level understanding" means:

- *object detection*: a network processes each frame of a street video and outputs *bounding boxes* with labels around cars, persons, trucks (early detectors handled about 20 object classes; today about 80 is standard). From these, one can count vehicles and persons, track them, estimate their apparent speed: a surveillance camera can send just numbers (how many cars are parked in a region, how many are waiting at a traffic light) without any human intervention;
- *human pose estimation*: estimating the positions of the joints of multiple persons in a video, even in unusual poses and activities, which makes, e.g., activity recognition much easier;
- *industrial inspection*: e.g., classifying the *defect maps* of silicon wafers (the locations of defects reported by inspection machines). Patterns in the defect locations reveal problems in the production process (e.g., a robot scratching the wafers while manipulating them); sample checks by humans were replaced by a deep network determining whether the pattern is normal, a known defect type or an unknown one;
- *image captioning*: generating a textual description of an image (already in 2015, the early years of deep learning).

Before 2010–2012, computer vision tools were mostly based on mathematical, statistical and geometric modelling: assumptions were made explicit and algorithms were designed step by step from them. Those tools are still in use and still an active research area (e.g., geometric vision, image processing). But to understand the *semantics* of an image (what it contains, how many cars there are, where the objects are), machine learning, and specifically deep neural networks, became essentially the only tool. This chapter introduces the background: what a digital image is, the basic operation of linear filtering, and an attempt to classify images with the networks seen so far, showing why it does not work well.

9.1 Digital images

A color (RGB) image is encoded in a computer as **three matrices** of integers, i.e., a three-dimensional array of size $R \times C \times 3$ (rows, columns, channels). Each matrix is an *intensity* image (it can be displayed as a grayscale image) telling, for each **pixel**, how much red, green or blue light is there. A pixel is the smallest intensity measurement of the image: zooming in, one sees the "Minecraft-like" squares.

Intensities are integers in $\{0, \dots, 255\}$, i.e., one byte (8 bits) per channel. A very red pixel has, e.g., the triplet $(253, \approx 0, \approx 0)$; a white pixel has all three values high (above 230, and close to each other); a black pixel has all three low. With 3 bytes per pixel one represents $2^{24} \approx 16.7$ million colors.

Example 9.1 (How big is an image?) *A phone camera of 48 megapixels acquires* 48×10^6 *pixels, each of 3 bytes:* 144 *MB of raw information. Yet the file of a picture is about* 2.5 *MB, less than* 3% *of that. The reason is that images are saved in* compressed *formats (JPEG, PNG). JPEG, for instance, splits the image into* 8×8 *blocks, represents each in the DCT (discrete cosine transform) domain, quantizes the coefficients and then encodes them with lossless entropy-coding schemes like those of a zip file. Mathematical modelling of images is still very much in use: it is what makes your pictures that small. The amount of compression depends on the content: a white desk compresses a lot (it is very redundant), a highly textured scene like grass seen from far away compresses much less.*

A neural network, however, cannot read the compressed stream: the visual content is not visible in the bits of a JPEG file. The image must be decoded into its three matrices, so it occupies the full 144 *MB in RAM; and since networks process floating-point inputs (at least 4 bytes per value), even four times as much.*

For videos it is worse: a full-HD frame has about $1920 \times 1080 \approx 2$ *million pixels, i.e., about* 6 *MB per frame, and at 24 frames per second about* 150 *MB per second of uncompressed video (more than twice that at 60 fps). Videos cannot naively be handled as a collection of images.*

The first difficulty is thus clear: **images are very big**. This is why the first image classification datasets contained tiny images.

9.2 Local operations and linear filtering

Consider a single channel (a grayscale image, or one color channel; extending to three channels is straightforward). The simplest operation on an image takes as input the value of a single pixel and returns an output depending only on that value: gamma correction, histogram equalization, many contrast-enhancement transformations. These are *point-wise* operations: they map an image to an image, but process each pixel individually, so they are not really image-processing operations. A genuine image operation, for each pixel, takes into account a **neighborhood** of it.

Definition 9.1 (Neighborhood) *A neighborhood* U *is a set of* displacements *from a central pixel, e.g., the* 3×3 *neighborhood*

$$U = \{(u, v) : u, v \in \{-1, 0, 1\}\},$$

which includes the null displacement $(0, 0)$*. The neighborhood of pixel* (r, c) *in image* I *is the set of intensities*

$$\{I(r + u, c + v) : (u, v) \in U\}.$$

A **local (spatial) filter** T takes the neighborhood of (r, c) and outputs a single number, placed in the output image G at the same location (r, c):

$$G(r, c) = T\big[I\big](r, c) = T\big(\{I(r + u, c + v) : (u, v) \in U\}\big).$$

The very same operation is repeated over the entire image. This is a fundamental *prior* that will be key when building neural networks for images.

Moving average. The simplest such operation is linear: the average of the neighborhood,

$$G(r, c) = \frac{1}{|U|} \sum_{(u,v)\in U} I(r + u, c + v),$$

with $|U| = 9$ for a 3×3 neighborhood. Its effect is to *blur* the image. Consider a thin bright line (intensity 255) on a dark background. On a pixel of the line, the neighborhood contains 3 line pixels and 6 background pixels, so the output is one third of the line intensity: gray. The black pixel next to the line also has one third of its neighborhood on the line, so it becomes the same gray. The result is a thicker, less bright line.

Correlation. To make the operation learnable we need parameters. Replace the fixed coefficient $1/9$ with a weight $w(u, v)$ for each displacement:

$$G(r, c) = (I \otimes w)(r, c) = \sum_{(u,v)\in U} w(u, v)\, I(r + u, c + v). \tag{9.1}$$

The matrix w (here 3×3, with arbitrary values such as $7, -3, 2, 2.1, \dots$) is called the **filter** (or kernel, or weights). Its size defines the neighborhood, and the operation is entirely defined by it. One slides the filter over the image and, at each location, computes the element-wise product between the filter and the neighborhood and sums the results: a sequence of *multiply-and-accumulate* operations, which computers perform very efficiently. This operation is the **correlation** between the image and the filter. A linear combination with learnable weights is exactly what happens inside a neuron: here, it is a linear combination of the values of each pixel neighborhood, applied at every pixel. A naive implementation uses four nested loops: two over the pixels of the image, two over the displacements of the filter.

Remark 9.1 (Correlation vs. convolution) *In signal processing and numerical linear algebra one meets the* ***convolution****:*

$$(I \circledast w)(r, c) = \sum_{(u,v)\in U} w(u, v)\, I(r - u, c - v) = \sum_{(u,v)\in U} w(-u, -v)\, I(r + u, c + v),$$

i.e., the correlation with a filter flipped upside-down and left-to-right. This definition has nice properties in harmonic analysis: the Fourier transform of the convolution is the element-wise product of the Fourier transforms of image and filter. In convolutional neural networks, however, the weights have no predefined meaning: they are learned*, so there is no point in flipping them. "Convolutional" neural networks actually compute* correlations*; we keep using the name "convolution" as everybody does, meaning correlation.*

Example 9.2 (Template matching) *Take a binary image (0/1 values) containing text, and as filter a small portion of the same image (e.g., a character). The output of the correlation is an image of the same size. At each location, it equals the number of pixels where both the filter and the neighborhood are nonzero. Where the neighborhood is all black, the output is zero; where some white pixels of other characters overlap white pixels of the filter one gets intermediate values; the* ***maximum*** *is exactly at the location where the filter and the image coincide, and equals the number of ones in the filter. This is the simplest form of* template matching*, i.e., object detection: modern detectors do not work exactly like this, but this operation is inside them.*

The method is rudimentary: with the inverted (negative) image, white regions of the image overlap all the white pixels of the filter, and the maximum is attained in many wrong places. Proper template matching requires some normalization (e.g., normalized cross-correlation).

Multichannel filtering. When filtering a color image, one usually processes each channel separately with the same operation. Alternatively, one may use a three-dimensional filter: correlate the red filter with the red channel, the green with the green, the blue with the blue (spatially, as in (9.1)), and *sum* the three results into a single output channel. This is exactly what convolutional layers do (Chapter 10).

9.3 Image classification with a neural network

Definition 9.2 (Image classification) *Given a set of labels Λ, image classification assigns to each image the label it most likely belongs to. A classifier is a mapping*

$$f : \mathbb{R}^{R\times C\times 3} \to \Lambda \qquad \text{or} \qquad f : \mathbb{R}^{R\times C\times 3} \to [0, 1]^L,$$

where in the second case the output is a vector of probabilities over the $L = |\Lambda|$ classes (e.g., "wheel" 65%, "tire" 30%, ...).

The networks seen so far take *vectors* as input: each input neuron holds one number, each layer is a column of neurons, and the output is a vector of probabilities. An image is a three-dimensional array, so the obvious workaround is to **flatten** (unfold) it into a vector, e.g., column-wise, first the red channel, then the green, then the blue, and feed it to a multilayer perceptron trained on a set of labeled images.

Dimensionality. For a phone picture, this vector would have 144 million components. The smallest classical color dataset is **CIFAR-10**: 10 classes (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck), images of $32\times32\times3$ pixels, smaller than an app icon. Even so the flattened vector has $32\cdot32\cdot3 = 3072$ components. A single hidden layer as large as the input would already have $3072^2 \approx 9$ million weights. It is reasonable to use no hidden layer at all: input, flattening, and an output layer of 10 neurons with softmax, i.e., $3072 \times 10 + 10 = 30\,730$ parameters. (Being able to compute the sizes and the number of parameters of a network from its code is an essential skill.)

9.3.1 The network is a linear classifier

Assume such a network has been trained (with categorical cross-entropy). The output neuron of class i computes the *score*

$$s_i = \sum_{j=1}^{d} w_{ij}x_j + b_i = \mathbf{w}_i^\top \mathbf{x} + b_i,$$

followed by the softmax. Since the softmax (and, more generally, any monotonic activation) does not change the ordering of its inputs, the class with the largest probability is also the one with the largest score: after training we can drop the softmax (needed only to compute the loss) and look directly at the scores. Stacking the rows $\mathbf{w}_i^\top$ into a matrix $W \in \mathbb{R}^{L\times d}$ (L classes, d input components):

$$\mathbf{s} = W\mathbf{x} + \mathbf{b}, \qquad \hat{y} = \arg\max_i \; s_i.$$

This is a **linear classifier**. It can be seen as L separate classifiers, each computing the score of one class independently (the weights are learned jointly, but at inference the scores are computed separately), and the output is the class with the largest score. Each row of W can be split into three parts: the first third multiplies the red pixels, the second the green, the third the blue.

Remark 9.2 (Why nonlinearities) *Stacking layers without nonlinearities is pointless: $W_3(W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_3 = \tilde{W}\mathbf{x} + \tilde{\mathbf{b}}$ with $\tilde{W} = W_3W_2W_1$, again a linear classifier, with a huge number of wasted parameters. Nonlinearities are not used "because they work better": without them, a deep network collapses to a linear classifier.*

9.3.2 Two interpretations of a linear classifier

Geometric interpretation. Each image is a point in a d-dimensional space ($d = 3072$ for CIFAR-10), each coordinate being the intensity of a pixel in a channel. Each row of W defines a hyperplane $\mathbf{w}_i^\top\mathbf{x}+b_i = 0$ (in two dimensions, $w_1x_1 + w_2x_2 + b = 0$ is a straight line; the function is positive on one side and negative on the other). The classifier tries to separate the clusters of points of each class by hyperplanes, assigning the class whose score is the largest. As we will see, this is rather hopeless for images.

Template-matching interpretation. Ignore the bias (it only shifts the scores). Each row $\mathbf{w}_i$ has d entries, as many as the pixels of the input image: inverting the flattening, we can rearrange it into an image as large as the input and visualize it. The row of the "car" neuron, displayed as an image, looks like a car! This is not by chance. The score $\mathbf{w}_i^\top\mathbf{x}$ multiplies each pixel of the input image by the corresponding entry of the template, red channel by red channel, green by green, blue by blue, and sums everything: it is the **correlation** between the input image and a *template* $\mathbf{w}_i$ of the same size, evaluated at a single location (since filter and image have the same size, there is only one position, hence a single number). As in template matching, the score is larger the more the image resembles the template.

So a linear classifier computes the correlation of the input with L *learned templates*, one per class, and outputs the class whose template is most similar. The templates are learned jointly so that each image has the largest correlation with the template of its own class. The CIFAR-10 templates reveal what the network learned:

- planes and ships lie on a *blue* background (sky, sea), deer and birds on a *green* background;

- cars are *red* (many red cars in the training set, red being a very distinctive color), and their shape is visible because cars were mostly aligned in the same way;
- horses have *two heads*: in the training set horses look either left or right, and the template maximizing the correlation for the horse class (while minimizing it for the other classes) is a superposition of both.

Note that weights can be negative: a template can *penalize* the presence of red or green in some region, so that the correlation is high only when there is blue there.

Remark 9.3 (Invariance and data augmentation) *The two-headed horse is a key intuition: had all training horses looked left, the template would not be mirrored. To make a network* invariant *to a transformation (here, horizontal flipping), one has to provide training images displaying that transformation (horses looking both left and right). This is the idea behind* ***data augmentation*** *(Chapter 11), one of the few tools to steer a network towards learning invariances. The same template visualization can be done for linear classifiers on time series.*

9.4 Why image classification is challenging

1. **High dimensionality.** Even the tiny CIFAR images are huge inputs. When CIFAR was released, only about 8% of the datasets in the UCI machine learning repository (the standard repository at the time) had inputs of comparable dimension: there were not even datasets to develop this research.
2. **Rich content and label ambiguity.** An image contains many things. Labeling a picture of a man drinking a beer as "man" tells the network that the image does *not* belong to class "beer": if the network finds evidence of beer, it is penalized. Single labels may give conflicting supervision.
3. **Transformations that do not change the semantics**: illumination changes, deformations, viewpoint changes, occlusions, background clutter, scale. Seen as points in $\mathbb{R}^d$, the same content under different illumination is moved to completely different positions (close to the origin along many coordinates in a dark image, far in a bright one): no set of hyperplanes can cope with this.
4. **Intra-class variability**: objects of the same class can look very different.
5. **Vector distances are meaningless for images.** The classifiers seen so far implicitly rely on distances between vectors (Euclidean, or ℓ_1: $d(I_1, I_2) = \sum_{r,c,k} |I_1(r,c,k) - I_2(r,c,k)|$, as in a k-NN classifier). Starting from a face image X_0, one can craft three images: a darkened version, a version with the face shifted, and a version with eyes and mouth occluded. The first two have exactly the same semantic content as X_0, the third has lost much of it; yet the three have the *same* distance from X_0. A t-SNE visualization of CIFAR-10 (images placed on a plane so that images close according to the Euclidean distance are nearby) shows that a frog is often closer to a truck than to another frog.

The Euclidean distance between images, seen as vectors, does not correlate with the difference of their semantic contents: a classifier based on such distances cannot be successful. We need something better than treating images as vectors: **convolutional neural networks**, designed precisely to learn suitable representations of images, are the subject of the next chapter.

Chapter 10

Convolutional Neural Networks

Chapter 9 showed that flattening an image into a vector and feeding it to a multilayer perceptron is a failure: such a network only learns the dominant colors of a class and its coarsest shapes (the templates). This chapter introduces the first neural network model designed for images, the *convolutional neural network* (CNN).

10.1 From hand-crafted to data-driven features

Before deep learning, the main approach to image classification was **feature extraction**. Consider a real-world example: depth images of items on a conveyor belt of a logistics plant must be classified as *envelope*, *parcel* or *double* (two items) for correct dispatching. A domain expert knows, e.g., that an image containing two items covers roughly twice the area of an image with a single item; a skilled programmer writes an algorithm computing that area from the depth image. Combining expert knowledge and programming one obtains one or more *features*: area, perimeter, maximum height, slope... The better the programmer and the deeper the insight into the problem, the more informative the features. The image is thus mapped to a vector, and any classifier for tabular data (e.g., a neural network with three output neurons) can be trained on it.

Pros of hand-crafted features.

- If one already knows that a feature (e.g., the area) is very informative, it is better to extract it explicitly and use the data-driven component only for the classification, instead of asking the classifier to learn what is already known.
- Features are *interpretable*: one knows what is being measured and can adjust or improve them using domain expertise.
- The classification problem becomes low dimensional (a dozen features gives a 12-dimensional problem), so few annotated training samples may suffice.

Cons.

- It requires programming effort and *domain expertise*. Everybody can reason about parcels, but for, e.g., ECG tracings one needs a medical doctor to identify relevant features.
- Features do not transfer: those designed for parcels are useless for grading the quality of fruit on another conveyor belt; one has to start from scratch every time.
- Most importantly, it does not allow solving visual recognition problems on natural images that are trivial for us: a child distinguishes a car from a motorbike, but nobody can write a program computing a feature that tells them apart.

The deep learning revolution. Deep learning makes the feature extraction step *data driven*: the feature extractor itself becomes a neural network, trained *jointly* with the classifier to minimize the classification (or regression) loss. The features are no longer based on our intuition and programming skills, but optimized to make predictions accurate. The feature extraction network becomes rather large and its features are completely uninterpretable, but this makes it possible to solve recognition problems on natural images that were unthinkable with hand-crafted features. Note that hand-crafted features are not just a thing of the past: the "feature extractor + classifier" view is the right perspective on deep networks too. Data-driven feature extraction is meant for images, signals, text, videos, multimedia data; for tabular data, which are already vectors ready for classification, one rather uses standard feature selection procedures.

10.2 The architecture of a CNN

Recall the correlation (9.1). With a filter with a single one at the center and zeros elsewhere, the output is the input image itself; with a uniform filter, the image is blurred (the larger the filter, the stronger the blur); with ones along a diagonal, the image is blurred along that direction (a motion blur; in image processing such kernels are called *point spread functions*, since they show how a single point spreads). Following the template-matching view, the output is large where the image locally resembles the filter.

From vectors to volumes. A multilayer perceptron maps a vector to a vector: each layer computes $\mathbf{s} = W\mathbf{x} + \mathbf{b}$ followed by a nonlinearity, and neurons are arranged in columns. In a CNN, an image is a *volume* $H \times W \times C$ (a three-dimensional array), and each layer maps a volume into another volume. For instance, a $128 \times 128 \times 3$ image may become, after the first layer, a $128 \times 128 \times 8$ volume: an "image" with 8 channels. The numbers in these volumes are the outputs of neurons, still organized spatially as the pixels of an image; they are called **activations** or **feature maps**.

As a rule of thumb, moving through the network the *spatial* dimensions H', W' *decrease* while the number of *channels increases*. In the extreme case the spatial dimension becomes 1×1: the volume has become a vector, which can be fed to a standard multilayer perceptron for classification.

In the usual illustrations, each value of an output volume is computed from a region of the input volume (a small box) which is *local* in space but spans the *whole depth*, i.e., all the channels. This operation is implemented by the convolutional layers. A CNN is built from:

- **convolutional layers**, mixing local spatial information and all the channels, and typically increasing the number of channels;
- **activation** (nonlinear) layers;
- **pooling layers**, reducing the spatial extent;
- a final **dense** (fully connected) part, i.e., the classifier.

The input of a CNN, like that of any classifier, usually has a fixed size (a classifier for 4-dimensional vectors cannot process 5-dimensional ones); we will see later that CNNs can be made to handle inputs of different sizes.

10.3 Convolutional layers

Let us derive the formula of a convolutional layer. Denote by x the input volume (the image, or the activations of the previous layer) with C channels, and by $a(r, c, 1)$ the value of the output neuron at row r, column c of the first output channel. It must combine the values of x in a neighborhood of (r, c) and across *all* the channels. The simplest choice, a sum

$$a(r, c, 1) = \sum_{k=1}^{C} \sum_{(u,v)\in U} x(r + u, c + v, k),$$

is linear (good: gradients are easy to compute for training) but has nothing to learn. Introducing learnable weights and a bias:

$$a(r,c,1) = \sum_{k=1}^{C} \sum_{(u,v)\in U} w^1(u,v,k)\, x(r+u, c+v, k) + b^1. \tag{10.1}$$

The filter w^1 is *three-dimensional*: it has as many channels as the input. The first channel of the input is correlated with the first channel of the filter, the second with the second, and so on; all the results are summed together and a single (learnable) bias is added. This is the three-dimensional correlation of Chapter 9.

Weight sharing. In (10.1) the weights and the bias do not depend on (r, c): the same filter is used for all the output neurons of the same channel. This is the main difference with respect to a multilayer perceptron, where every output neuron has its own weights and bias.

Multiple filters, multiple output channels. One filter produces one output channel (a single-channel image). To obtain more output channels, one uses more filters: a layer with N filters produces an output volume with N channels,

$$a(r,c,\ell) = \sum_{k=1}^{C} \sum_{(u,v)\in U} w^\ell(u,v,k)\, x(r+u, c+v, k) + b^\ell, \qquad \ell = 1, \ldots, N. \tag{10.2}$$

Each output channel is computed by its own filter only: there is no mixing among filters. The layer is still a linear combination of its inputs plus a bias, so all the machinery of neural networks (gradients, gradient descent, backpropagation) applies unchanged; the novelty is that the operation is *local in space* and *global along the channels*.

Number of parameters. A filter of spatial size $S_x \times S_y$ (e.g., 3×3, 5×5, 7×7, 11×11) applied to an input with C channels has $S_x S_y C$ weights plus one bias. With N filters:

$$\#\text{params} = (S_x \cdot S_y \cdot C + 1) \cdot N.$$

The depth C of the filters is *not* a hyperparameter: it always equals the number of channels of the input volume (in the code one does not even specify it). The hyperparameters are the spatial size of the filters (`kernel_size`) and their number N, which determines the depth of the output volume. Growing the depth of the activations through the network is just a matter of using more and more filters.

Example 10.1 (Counting parameters)

- *One grayscale input channel, one 5×5 filter: one output map; with "same" padding it has the size of the input, with "valid" padding it is $(H-4) \times (W-4)$. Parameters: $25 + 1 = 26$.*
- *One input channel, two filters: an output with two channels, each computed by correlating the input with one filter.*
- *Three input channels, two 5×5 filters: each filter is $5 \times 5 \times 3$ (three matrices). The output has two channels. Parameters: $(5 \cdot 5 \cdot 3 + 1) \cdot 2 = 76 \cdot 2 = 152$.*
- *Three input channels, 32 filters 3×3: $(3 \cdot 3 \cdot 3 + 1) \cdot 32 = 896$ parameters.*

The number of parameters does not depend on the spatial size of the input nor on the stride: these affect only the size of the output.

Example 10.2 (Filters of AlexNet) *Loading a trained AlexNet, its first-layer filters are $11 \times 11 \times 3$ (large; filter sizes have shrunk over the years), i.e., $121 \cdot 3$ weights plus one bias each. Since they have three channels, they can be displayed as color images. One filter shows a blue–orange–blue striped pattern. Applying it to the picture of a salamander, the output map (a single-channel image of the same size; weights can be negative, so outputs can be negative: gray ≈ 0, black negative, white positive) has its largest activations exactly where the image locally resembles the filter, i.e., along the diagonal yellow-against-dark edges of the salamander. A second filter with a diagonal yellow-against-blue pattern activates near the spots. The first layer is doing* template matching with learnable templates*, scanning the whole image in space and always grabbing all the channels. Filters of deeper layers have many channels and cannot be visualized as images.*

Padding. How is the correlation computed at the boundaries, where the filter falls partly outside the input? Three options, for an $H \times W$ input and a 5×5 filter:

- **valid**: compute the output only where the filter is entirely contained in the input. The output shrinks to $(H-4) \times (W-4)$ (two rows lost at the top, two at the bottom, two columns on each side);
- **same**: pad the input with zeros (or other values, e.g., by reflecting the image) so that the output is $H \times W$;
- **full**: compute the output wherever filter and image overlap at least partially, giving $(H+4) \times (W+4)$.

Stride. By default the stride is 1×1: the correlation is computed at every location. With stride 2×2 it is computed only at every second row and column, halving the output size. This is equivalent to computing the full correlation and then subsampling, but much cheaper. Strided convolution was not the mainstream way of reducing the size in early networks, but it is popular in modern architectures. In general, for input size H, filter size S, padding P (on each side) and stride s, the output size is

$$H' = \left\lfloor \frac{H + 2P - S}{s} \right\rfloor + 1.$$

Being able to compute, given the code of a network, the size of each activation and the number of parameters of each layer (acting as a "Python interpreter") is a good test of having mastered the architecture.

How many filters, how many layers? The size of the output volume is set by the number of filters, the stride and the padding. Choosing the number of filters is like choosing the number of neurons of a hidden layer; in principle one could use k-fold cross-validation, but training CNNs is expensive: with 64 filters the first activation is 64 times as large as the image. Except for very simple problems, architectures are not cross-validated; rather, one relies on established designs (and on frameworks that optimize the hyperparameters).

Convolution vs. correlation, once more. As discussed in Chapter 9, there is no need to flip the filters, since they are learned: as long as they are used in the same way in which they are learned, no harm is done. Also, in classic image processing color images are filtered channel by channel, while a convolutional layer *always mixes all the channels*.

10.4 Activations, pooling and the dense part

Nonlinearities. A stack of convolutional layers alone is linear, and can be rewritten as a single convolution with a larger filter, exactly as for multilayer perceptrons. A scalar nonlinear function is applied to each entry of the output volume. The most widely used is the **ReLU**, $\max(0, x)$: negative values are set to zero, positive values propagate untouched. Variants include the leaky ReLU (negative values are scaled down instead of zeroed); sigmoid, tanh and others can also be used. After a ReLU, in the salamander example only some neurons remain bright, the others are set to zero. In many frameworks the activation is an attribute of the convolutional layer: beware that the default may be linear (no activation).

Pooling. To reduce the spatial extent, the most popular solution since the first deep networks is **max pooling**: each channel is processed separately, divided into tiles (typically 2×2), and only the maximum of each tile is propagated. For example, the tiles $\left(\begin{smallmatrix}1&1\\5&6\end{smallmatrix}\right)$, $\left(\begin{smallmatrix}2&4\\7&8\end{smallmatrix}\right)$, $\left(\begin{smallmatrix}3&2\\1&2\end{smallmatrix}\right)$, $\left(\begin{smallmatrix}1&0\\3&4\end{smallmatrix}\right)$ give $6, 8, 3, 4$. The stride is usually equal to the pool size, so a 2×2 max pooling halves both rows and columns, keeping 25% of the values. Max pooling after a ReLU is perfectly consistent: one keeps the positive entries and propagates the strongest ones, i.e., the neurons that "get hot". Max pooling with stride 1 does not reduce the size but still adds a nonlinearity. In early networks max pooling was preferred over strided convolution because it adds a nonlinearity. *Average pooling* propagates the mean of each tile; it is a special, non-learnable convolution (with a uniform filter) and adds no nonlinearity.

Why pooling? The receptive field. Max pooling throws away 75% of the values: why is it useful? The **receptive field** of a neuron is the region of the input image that affects its value. A neuron after a 3×3 convolution depends on a 3×3 region of the image. After pooling, a second 3×3 convolution operates on a summarized volume, and its neurons depend on a much larger region of the original image. Pooling thus enlarges the receptive field at zero parameter cost. This explains the typical behaviour of CNNs: the first layers, seeing small regions, respond to simple patterns (lines, edges, corners, small circles); deeper layers, whose neurons see large portions of the image, respond to complex patterns (a whole cat face). The filters are not intrinsically different: they just see information coming from a larger region.

Flattening and the classifier. Ideally, the last volume is $1 \times 1 \times C$ with large C; in practice one does not necessarily get there. A **flattening** layer unfolds the last volume into a vector (a $2 \times 2 \times C$ volume becomes a $4C$-dimensional vector), which is fed to one or more **dense** layers, ending with as many outputs as classes (e.g., a softmax over 128 classes).

Summary. The convolutional part, alternating convolution (+ activation) and pooling, bridges an image to a vector: it is the **feature extraction network** (or *backbone*), and the final vector is the data-driven *feature vector* (or *latent representation*). The classifier on top is the *feature classifier*. Both are trained jointly to minimize the loss: the parameters of the convolutional layers are optimized so that the feature vectors are *maximally separable* by the classifier. Stacking many such layers makes the network *deep*.

Example 10.3 (A CNN in the browser) *A well-known demonstration (from one of the first deep learning courses) runs a small CNN on CIFAR-10 in a web browser and displays the activations channel by channel. The network uses 10 filters in every convolutional layer (a choice made for visualization). In the first layers one still recognizes the shape of the car, since operations are local; through convolutions, ReLUs and poolings, the shape is progressively lost. The final activation is* $4 \times 4 \times 10$*, i.e., a 160-dimensional feature vector fed to the classifier of the 10 classes.*

Remark 10.1 (Spot the error) *A popular illustration of a CNN (on Wikipedia) shows a subsampling step that* increases *the number of feature maps. This is wrong: pooling operates channel by channel and never changes the number of channels (a strided convolution could, but that is not what the figure says).*

10.4.1 Feature space as a meaningful embedding

At the end of Chapter 9, a t-SNE map of CIFAR images based on Euclidean distances between pixels placed frogs next to trucks. Now take a CNN trained on a large classification dataset, drop the classifier, and compute the Euclidean distance between the *feature vectors* of images (e.g., of the harder CIFAR-100). The t-SNE map changes completely: coffee machines are next to each other, TV screens are close together, landscapes cluster, red cars and yellow objects group together, regardless of orientation and pixel-level similarity. Distances in the latent space correlate with the semantics. This is no coincidence: if the network is a successful classifier, its classifier can draw boundaries separating the classes in the feature space, so images of the same class must be close. The mapping from an image to its feature vector is called an **embedding**; it is the basis of image retrieval and of many practical uses of deep networks.

10.5 Lab: images in Python and a first CNN

10.5.1 Handling images

Images are loaded with the Pillow library (PIL) and converted to NumPy arrays of type `uint8` (unsigned 8-bit integers, the most compact representation for values in 0–255), of shape (height, width, channels), e.g., $830 \times 735 \times 3$. A grayscale image has no channel dimension: a single intensity per pixel.

- **Channel order matters**: Pillow and Matplotlib use the RGB convention, while OpenCV, very common for images, uses **BGR**. Loading an image with one library and displaying it with another without care swaps red and blue.

- **Grayscale conversion**: averaging the three channels, or better a weighted average reflecting the sensitivity of the human eye, highest for green, then red, then blue (e.g., $0.299R + 0.587G + 0.114B$), which gives a more pleasing image; Pillow has a ready-made function.
- Since images are arrays, any array transformation applies: flips (`fliplr`, `flipud`), rotations, transpose (defined for matrices, i.e., per channel; a "transposed" color image can be obtained by a horizontal flip followed by a 90° rotation).

10.5.2 Convolutions by hand

A simple implementation of the convolution (correlation) of a single-channel image with a kernel uses two nested loops over the image (after padding it so that output and input shapes coincide), cutting at each location a region as large as the kernel, multiplying element-wise and summing:

Listing 10.1: Naive 2D correlation of one channel.

```
def conv2d(img, kernel):
    kh, kw = kernel.shape
    padded = np.pad(img, ((kh // 2, kh // 2), (kw // 2, kw // 2)))
    out = np.zeros_like(img, dtype=np.float32)
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            region = padded[i:i + kh, j:j + kw]
            out[i, j] = np.sum(region * kernel)   # no bias here
    return out
```

Classic kernels from the computer vision literature (convolution was born in "old school" image analysis to extract hand-crafted features) illustrate the effects: the *identity* kernel leaves the image unchanged; *box blur* (uniform) and *Gaussian blur* (more concentrated) smooth it; *sharpening* kernels enhance details; several edge detectors (Laplacian-like, Prewitt, Sobel) highlight contours. The Sobel kernels

$$S_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}, \qquad S_y = S_x^\top,$$

detect edges in the two directions (S_x has zeros in its central column); the classic edge magnitude is $\sqrt{(I \otimes S_x)^2 + (I \otimes S_y)^2}$. Applying an edge kernel only to the red channel of a color image draws the contours in red while the surroundings turn cyan. Applying a random kernel repeatedly (e.g., 5 times) stretches the image along a diagonal more and more: stacking convolutions produces representations visually very different from the input. Fortunately, a CNN *learns* its kernels, so there is no need to choose among these heuristics.

10.5.3 Why convolutions instead of dense layers

- **Translation equivariance.** Consider a cat at the top of an image and the same cat at the bottom right. In a dense network the flattened pixels are permuted, every hidden value changes and the output may be completely different. A convolution slides the same operation over the image: when it meets the cat it produces the same response wherever the cat is. The class of an image should not change when the subject moves, and the convolution exploits this.
- **Efficiency.** Images are large; a dense hidden layer comparable to the input size has an exploding number of parameters, while convolutions extract more information with far fewer parameters.

The general principle (the core of *geometric deep learning*) is to design layers that exploit the geometry of the data. For time series, a translation in time should not change the class, hence 1D convolutions; for videos one wants invariance both in space and in time, e.g., with a recurrent convolutional network (a CNN on each frame whose features are fed to a recurrent network), rather than feeding everything into a huge dense network.

10.5.4 A CNN for CIFAR-10

CIFAR-10 comes with a fixed split, so that all research results are comparable: 50 000 training images (here 5 000 are kept for validation) and 10 000 test images of $32 \times 32 \times 3$ pixels, with balanced classes. Preprocessing: cast to float and divide by 255 (feeding integers in 0–255 is possible, but large inputs lead to large weights and unstable training). PyTorch expects tensors of shape (batch, channels, height, width), so the arrays of shape (batch, height, width, channels) are permuted with `x.permute(0, 3, 1, 2)`.

The model is built from reusable blocks. A block contains one or more convolutions (stride, if any, applied only to the last one: applying stride 2 in every convolution would shrink $32 \to 16 \to 8 \to 4$ very quickly), a ReLU, and a 2×2 max pooling if no stride was used. The number of channels starts at 32 and is multiplied by 2 at each block (32, 64, 128…, the common practice in the literature). The output size of the backbone is computed by feeding a dummy input; the head is just flatten, dropout and a linear layer.

Listing 10.2: A simple configurable CNN.

```
class ConvBlock(nn.Module):
    def __init__(self, c_in, c_out, n_convs=1, stride=1, pool=2):
        super().__init__()
        layers = []
        for i in range(n_convs):
            s = stride if i == n_convs - 1 else 1
            layers += [nn.Conv2d(c_in if i == 0 else c_out, c_out,
                                 kernel_size=3, stride=s, padding=1),
                       nn.ReLU()]
        if stride == 1:
            layers.append(nn.MaxPool2d(pool))
        self.block = nn.Sequential(*layers)
    def forward(self, x):
        return self.block(x)

class SimpleCNN(nn.Module):
    def __init__(self, in_shape=(3, 32, 32), n_classes=10, n_blocks=2,
                 c0=32, mult=2, dropout=0.2):
        super().__init__()
        blocks, c_in, c = [], in_shape[0], c0
        for _ in range(n_blocks):
            blocks.append(ConvBlock(c_in, c)); c_in, c = c, c * mult
        self.features = nn.Sequential(*blocks)
        with torch.no_grad():
            n_flat = self.features(torch.zeros(1, *in_shape)).numel()
        self.classifier = nn.Sequential(nn.Flatten(), nn.Dropout(dropout),
                                        nn.Linear(n_flat, n_classes))
    def forward(self, x):
        return self.classifier(self.features(x))
```

With two blocks, the first convolution has $(3{\cdot}3{\cdot}3{+}1){\cdot}32 = 896$ parameters and the second $(3{\cdot}3{\cdot}32{+}1){\cdot}64 = 18\,496$; yet the single linear layer of the head ($8 \cdot 8 \cdot 64 \cdot 10 + 10 = 40\,970$) holds the largest share of the parameters. This shows how parameter-efficient convolutions are. Since the convolutional parameters do not depend on the input size, the backbone can process images of any size; the bottleneck is the dense head with its fixed number of inputs. Replacing flattening with an average over the spatial dimensions of each channel (*global average pooling*, Chapter 12) yields networks that accept images of any size.

Results. Training uses cross-entropy, a learning rate of 10^{-3}, early stopping with patience 50, dropout only in the dense head (not in the convolutions), mixed precision and the usual training loop. The 2-block network reaches a validation F1 of about 0.74 (state of the art on CIFAR-10 is above 90%, but this network trains in minutes). Plotting the activation maps for a test image (the most naive way of inspecting a CNN) shows that the first layer responds to simple lines and edges, while later maps are smaller and less interpretable. A 3-block network overfits sooner (early stopping restores the best weights) and improves to

about 0.8. A 4-block network learns faster (more parameters) but performs *worse*. A likely reason: the spatial size goes $32 \to 16 \to 8 \to 4 \to 2$, so in the last block the 3×3 kernel is larger than the volume and it processes more padding than actual image. When stacking blocks, watch how many pooling/stride layers reduce the size; it is often better to add more convolutions per block before reducing. TensorBoard's graph view lets one inspect the nested modules (feature extractor: Conv–ReLU–MaxPool blocks; classifier: Flatten–Dropout–Linear) and their weights. As always, models are selected on validation, and only the final model is evaluated once on the test set: that is the number to report as the generalization performance.

Chapter 11

Understanding and Training CNNs: Inductive Bias, Augmentation and Transfer Learning

This chapter digs deeper into convolutional layers, showing that they are a very special kind of dense layer, and derives from this the *inductive bias* that makes CNNs suitable for images. It then introduces the receptive field and the training of CNNs, and the two main practical tools to train data-hungry CNNs when few annotated images are available: data augmentation and transfer learning.

11.1 A convolutional layer is a constrained dense layer

Recall (10.2): a convolutional layer computes linear combinations over a small spatial extent with learnable weights (the filters, whose depth equals the number of input channels) and one bias per filter, the same operation being applied over the whole input volume. Being linear, it can be written as a general linear map

$$\mathbf{a} = W\mathbf{x} + \mathbf{b}.$$

What do W and $\mathbf{b}$ look like? Since $\mathbf{a}$ and $\mathbf{x}$ are vectors, we first flatten the volumes. Let the input activation be x of size $H \times W \times C$ and the output a of size $H \times W \times C'$ ("same" padding, which only simplifies the borders; C' filters). Flattening channel by channel, $\mathbf{x}$ has HWC entries (the first chunk of HW entries is the first channel, the second chunk the second channel, and so on) and $\mathbf{a}$ has HWC' entries.

Dense connectivity. If the two activations were connected by a dense layer (as in a multilayer perceptron), each output would be a combination of *all* the inputs with its own weights: the entry a_* is the row $\mathbf{w}_*$ of W times $\mathbf{x}$ plus b_*. After training, the weights take whatever value they need, so W is a *dense* matrix of size $(HWC') \times (HWC)$, i.e., H^2W^2CC' entries. Even for the tiny 32×32 CIFAR images, a late activation with $HW \approx 10^3$ and $C = C' \approx 10^2$ gives $10^3 \cdot 10^3 \cdot 10^2 \cdot 10^2 = 10^{10}$ weights: ten billion parameters for one layer. For 256×256 activations it would be a nightmare.

One channel in, one filter. Consider a single input channel and a single 3×3 filter. Now $\mathbf{x}, \mathbf{a} \in \mathbb{R}^{HW}$ and $W \in \mathbb{R}^{HW \times HW}$. The output at (r, c) depends only on the 9 inputs in the 3×3 neighborhood: in the flattened (column-wise) input these are three groups of three consecutive entries, separated by zeros. So each row of W has only 9 nonzero entries: the convolutional layer is a dense layer with **sparse connectivity**. Moreover, the 9 nonzero weights are the *same* for every row (the filter is shared), just shifted: moving to the pixel below shifts the nonzero entries by one position. The matrix W is thus sparse, with a *Toeplitz*-like structure of diagonal stripes, and contains only 9 distinct parameters. The bias vector is *constant*, since there is a single bias per filter: $9 + 1 = 10$ parameters in total.

Multiple channels and filters. With C input channels, the row of W corresponding to an output neuron of the first channel contains, for each input channel, the 9 weights of the corresponding slice of the first filter, in the sparse pattern above: W is made of C blocks side by side. All the rows for the first output channel (HW rows) have the same structure with weights shifted. The rows of the second output channel have the same structure but *different* weights (a different filter). So W is a block matrix with $C' \times C$ sparse, Toeplitz-like blocks; $\mathbf{b}$ is *block-constant*, one constant block of length HW per output channel.

In summary, a convolutional layer is a dense layer whose weight matrix has

- **sparse connectivity**: each output is connected only to a few inputs (a local neighborhood, but all the channels);
- **weight sharing**: the same parameters are used for every output neuron of a channel.

11.1.1 Inductive bias

Why look at the same operation from this angle? Because it reveals the *structural prior* (**inductive bias**) built into the architecture. Often one cannot say a priori whether a 7-layer perceptron is better than a 5-layer one or than a random forest; but sometimes principles guide the choice of the model, and CNNs are such a case:

1. **Locality**: each activation is computed from a small spatial region, so it is influenced only by the local content of the input. In images, nearby pixels are much more likely to be related than randomly selected pixels.
2. **Translation equivariance**: weight sharing applies the same operation at every location. In a multilayer perceptron every output neuron computes a different combination of the inputs; in a CNN every output neuron applies the same operation at a different location. If some weights compute a feature that is distinctive for a class, that pattern is searched over the whole image.

In images, semantically meaningful patterns occur locally and at any location: this is why CNNs are the right model. If one wants a model that behaves differently at each pixel and considers all pixels to compute each activation, one uses a dense layer, with all its dimensionality problems.

Example 11.1 (When not to use a CNN) *Consider a table with a row per person and columns age, height, weight, shoe size. One may be tempted to treat it as a matrix and apply a* `Conv2d`*. But then each feature mixes only neighboring entries: if rows are ordered by time this may make sense, if they are in random order locality does not hold. Worse, translation equivariance means that the weights found useful to mix age, height and weight are also used to mix height, weight and shoe size, which makes no sense. Knowing what a model does lets one choose the appropriate one (the difference is not always this apparent, but sometimes it is).*

Remark 11.1 (Invariance by design and by data) *Translation equivariance is guaranteed by design in the convolutional part; flattening followed by a multilayer perceptron breaks it (we will come back to this with global average pooling and segmentation networks). There is no structural guarantee of* rotation *invariance: the only practical way to obtain it is through* data augmentation*, training the network with rotated copies of the images with the same label, which hopefully teaches it that rotation is irrelevant for the class. This is an indirect way, not by design; architectures made rotation invariant by design exist but are extremely sophisticated.*

11.2 The receptive field

Because of locality, a neuron, even deep in the network, is not necessarily influenced by all the pixels of the image.

Definition 11.1 (Receptive field) *The receptive field of a neuron is the region of the input that contributes to computing its value. The receptive field of a network is usually the receptive field of a neuron (at the center, ignoring border effects) in the output of the last convolutional layer.*

In the first layer, the receptive field is the filter size (3×3 for 3×3 filters). Going backward from a 1×1 neuron:

- each 3×3 convolution adds $S - 1 = 2$ (one on each side): $1 \to 3 \to 5 \to 7 \to 9 \to \dots$; in 1D, a neuron of the second layer depends on 3 neurons of the first, which depend on 5 inputs;
- a 2×2 max pooling *multiplies* it by 2: only one value is propagated, but it is taken from a 2×2 region.

For instance, conv–pool–conv–pool–conv backward from 1×1: $1 \to 3$ (conv) $\to 6$ (pool) $\to 8$ (conv) $\to 16$ (pool) $\to \dots$; with another ordering one gets sequences such as $2, 4, 8, 10, 20, 22$. Pooling makes the receptive field grow fast.

The meaning is that each deep neuron is the output of a learnable function (a cascade of convolutions, nonlinearities and poolings) applied to a 22×22 region (say), and the same function is applied at every location of the image. Neurons of the first layers have small receptive fields and respond to small textures and low-level patterns; deeper neurons refer to large regions and become maximally activated by higher-level patterns, e.g., an eye, a nose, an ear. The classifier then takes these responses ("how likely there is an eye…"). We do not train the network to detect eyes: if eyes are distinctive for the classes, the network will most likely learn a feature that responds to them.

11.3 Training a CNN

A CNN is trained by backpropagation like any network. Convolutional layers are linear layers, so gradients flow through them; one only has to account for the constraint that shared parameters are the same (the gradient of a shared weight is the sum of the gradients of all its occurrences). For the other layers:

- **max pooling**: the derivative is 1 for the input that was propagated (the maximum) and 0 for the others. It suffices to remember, during the forward pass, where each maximum came from, and route the gradient there;
- **ReLU**: the derivative is a step function (0 for negative inputs, 1 for positive). It is not differentiable at 0, but in practice nobody worries, also because the loss is non-convex anyway and we are not guaranteed to reach a global minimum.

11.3.1 The ImageNet breakthrough

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) was a yearly competition: classifying images into 1 000 categories (e.g., telling a leopard from a jaguar or a cheetah). Until 2011 the winning solutions relied on expert-designed features from computer vision (e.g., SIFT descriptors) combined in elaborate pipelines (bag-of-words representations, hierarchical classifiers), with small yearly improvements and large errors. In **2012** *AlexNet*, the first CNN of modern times, dropped the error dramatically, and from then on only deep networks won. Its top-5 predictions are sensible: a leopard is classified as leopard with high probability, followed by jaguar and cheetah. Later CNNs even surpassed human performance.

CNNs were not new (they date back to 1998, Chapter 12). Two enabling factors made the 2012 breakthrough possible:

1. the availability of **large curated and annotated datasets** to train powerful models;
2. **computational power**: parallel architectures (GPUs) to train many parameters. At the time there were no frameworks such as TensorFlow or PyTorch: pioneers computed gradients explicitly.

Although CNNs are parsimonious compared to dense networks (which would be impossible to train or even to build at these sizes), famous architectures (AlexNet, VGG, ResNet, …) have millions of parameters. Training them requires a lot of data, but in real problems one is rarely given millions, or even a hundred thousand, annotated images. CNNs are **data hungry**, and two strategies help when the training set is small: data augmentation and transfer learning.

11.4 Data augmentation

Example 11.2 (Counting sea lions) *In a 2018 Kaggle competition, the US National Oceanic and Atmospheric Administration (NOAA) asked to count the Steller sea lions in the Aleutian Islands, an endangered species whose population had decreased by about* 90%*. The data were aerial drone images of about 20 megapixels (*$5\text{k} \times 4\text{k}$*), and the task was to count the animals of each category: adult males, subadult males, adult females, juveniles and pups (five numbers per image). Here we tackle it as the classification of small patches (*90×90*, a reasonable size given the resolution and the size of the animals), adding a sixth* background *class for patches with no sea lion (Chapter 14 returns to this as a segmentation problem).*

The first model was very unsuccessful: an adult male was classified as a female, a subadult as pup or adult male with almost equal probability. There was little training data: the population is endangered, and the social structure (one adult male with many females and pups) makes some classes particularly rare.

The image of a subadult male looks like that because the drone was flying in a certain direction; had it flown in another direction, we would have acquired a rotated image. We can thus *simulate* many images by rotating, translating, flipping the original, and the label remains the same.

Definition 11.2 (Data augmentation) *Data augmentation generates, from each training image, a set of transformed images that are all associated with the label of the original. Transformations are*

- ***geometric****: affine transformations, rotations, perspective changes, shear, flips, translations, scaling;*
- ***photometric****: changes of intensity only, e.g., brightness, contrast, saturation, additive noise.*

The only criterion is that the transformations must be ***label preserving****.*

Counterexamples: scaling a pup too much changes its apparent size and teaches the network that size is irrelevant for distinguishing pups from adults. Classifying the time shown by an analog clock without digits (an exam question that made many students angry): rotation must not be used, since it changes the target. One must know which transformations are allowed for the task at hand.

Mixup. When it is hard to define label-preserving transformations (medical or industrial images), *mixup* generates samples without worrying about it. Take two random training images x_1, x_2 (possibly of different classes, e.g., a cat and a dog) with one-hot labels y_1, y_2, draw $\lambda \in [0, 1]$, and train on

$$\tilde{x} = \lambda x_1 + (1 - \lambda)x_2, \qquad \tilde{y} = \lambda y_1 + (1 - \lambda)y_2,$$

e.g., a blended image with target 70% cat, 30% dog. This provides samples near the class boundaries by linear interpolation. The distribution of λ is a hyperparameter; mixup can also be applied within a class (then the target stays one-hot).

Benefits and caveats.

- Augmentation *promotes invariance* to the chosen transformations. Some invariances (translation) are partly built into CNNs, others must be taught this way. The two-headed horse template (Chapter 9) is exactly this effect.
- Augmentation is not equivalent to gathering more data: some variations (different backgrounds, superimposed objects, ...) cannot be simulated, and real samples are more informative. Still, feeding many modified copies *reduces overfitting*, since the network cannot learn details specific to the training images.
- In modern frameworks augmentation is performed *on the fly* as images are loaded: there is no need to store a huge augmented dataset, and at each epoch the batches contain different versions, so the network never sees exactly the same image twice.
- It can *mitigate class imbalance*, augmenting rare classes (e.g., subadult males) more heavily.
- Beware of **hidden traces**: transformations may introduce artifacts (e.g., the empty corners produced by rotations). If only one class is heavily augmented, the network may learn that the artifact indicates that class. Networks are lazy and will take any *shortcut*.

Test-time augmentation (TTA). Augmentation is a training-time technique, but it can also be used at inference: feed the network several augmented copies of the test image (rotated, translated, flipped) and aggregate the predictions (majority voting, averaging probabilities or logits). This is an *ensemble* obtained from a single model. The rationale: the network is not perfectly invariant, so an image misclassified in one orientation may be correctly classified on average over many. There is research on automatically optimizing the augmentation policies (even with reinforcement learning), but the practical criterion is to use the transformations that are meaningful, i.e., label preserving, for the problem.

11.5 Transfer learning and fine-tuning

The networks winning ImageNet classify a million images into 1 000 categories (from *tench* and *goldfish* to *great white shark* and *toilet tissue*), outperforming humans. This means that in their latent space all those classes are well separated: recall the t-SNE map of Chapter 10. Probably, then, those features are good for many classification problems, not only the one the network was trained for. The intuition is that the convolutional part extracts *general-purpose* features (a good mapping from images to vectors preserving semantics), while the classifier on top is *task specific*.

Transfer learning ("standing on the shoulders of giants"):

1. take a powerful pre-trained CNN and remove the task-specific part, i.e., all the fully connected layers, keeping only the feature extraction network (the embedding);
2. attach a new classifier for our problem (e.g., from "adult male" to "background");
3. **freeze** the weights of the feature extractor and train only the new classifier.

Freezing is essential: the new layers are randomly initialized, so the initial loss is computed on random predictions; backpropagating it through the whole network would destroy the excellent pre-trained weights. Since only the classifier is trained, and its size can be adjusted to the data (fewer outputs, chosen number of hidden layers), a small training set may suffice.

Fine-tuning. Transfer learning works best when data are few and the images come from the same domain on which the feature extractor was trained. With more data, or a different domain (thermal, medical images…), after transfer learning one can also train some layers of the feature extractor: **fine-tuning**. Fine-tuning is never done at the beginning: first train the new classifier (transfer learning), then, once the loss is meaningful, unfreeze part of the backbone (typically the last layers) and train with a small learning rate.

11.6 Lab: augmentation, retrieval and transfer learning

11.6.1 Augmentation on a small dataset

The first dataset contains a small number of images of *animals* and *items* (label 1 and 0), very heterogeneous: an "animal" is a broad concept and pictures are taken in very different environments. Preprocessing: normalize to $[0, 1]$, convert BGR to RGB, make the image square by cropping the central part (padding would be an alternative, e.g., for very rectangular images), resize, then split into training, validation and test.

Always start from a **baseline**: a simple model you understand, to verify whether later choices actually improve. On average a sophisticated pre-trained CNN beats a two-block network, but "never say never". The baseline is a simple CNN (conv–ReLU–maxpool twice, "same" padding, which for 3×3 kernels means padding 1, flattening, linear classifier); most of its parameters are in the final dense layer.

Remark 11.2 (Dropout and convolutions) *Dropout is usually applied only in the dense head, after flattening, not inside the convolutional part. Convolutions are already sparse and parameter-efficient, and they*

perform local pattern matching: zeroing individual activations inside a filter's pattern disrupts locality. Moreover, the computational unit of a convolutional layer is a whole filter, not a single weight: if one wants dropout in convolutions, it makes more sense to drop entire channels (spatial dropout). Practice sometimes contradicts intuition (neural architecture search finds architectures that "make no sense" and yet work), but this is the safe default.

The baseline learns the training set perfectly almost immediately: a clear case of *overfitting* (224×224 inputs allow memorizing patterns that do not describe the animal at all, possibly patterns that are simply absent in the other class). Validation F1 is about 66%.

Augmentation in PyTorch is defined with `transforms.Compose`, a list of random transformations, each applied with some probability:

- **flips** (horizontal, vertical): not always meaningful (people are rarely upside down; digits and clocks change meaning);
- **translations** (`RandomAffine` with a maximum fraction of the size), **rotations** (in degrees), **zoom** in and out: empty areas are filled with zeros by default (other frameworks also allow mirroring). For a dataset like MNIST, where all digits are centered, translations are useless if test digits are centered too, but needed if the model will read digits from arbitrary pictures. Stacking many strong transformations may push the content outside the image;
- **color** transformations: brightness, contrast, saturation (useful, e.g., in segmentation, where geometric transformations must be applied consistently to the masks);
- **random erasing** (*cutout*): with some probability, a rectangle of the image is erased (filled with zeros or random values), forcing the network to look at different parts of the object. A rectangle too large may remove all the useful information; several small rectangles can be used.

Transformations that look disruptive to us (swapping RGB channels, acting on one channel) can still help the network. With very strong augmentation, training loss may be higher than validation loss, since the training images are harder than the clean validation ones. Augmentation is applied **only to the training set**, implemented in a custom `Dataset` that applies the transformations when an item is loaded:

Listing 11.1: On-the-fly augmentation in a custom dataset.

```
train_tf = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomErasing(p=0.5, scale=(0.02, 0.2)),
])

class AugmentedDataset(Dataset):
    def __init__(self, X, y, transform=None):
        self.X, self.y, self.transform = X, y, transform
    def __len__(self):
        return len(self.X)
    def __getitem__(self, i):
        x = self.X[i]
        if self.transform is not None:
            x = self.transform(x)
        return x, self.y[i]
```

With the same network and only horizontal flip and random erasing, the model struggles more to reach F1 = 1 on training (training is noisier) and validation F1 rises to about 86.6%: a big improvement without changing the model.

Test-time augmentation in practice. Predict the test images, then predict their flipped versions, average the logits (or probabilities) and take the argmax. On the test set this improved the score from about 73% to 76% essentially for free (3 points are a lot in a competition). Guidelines:

- use (a subset of) the transformations used at training time; erasing parts of the image at test time makes little sense;
- use deterministic transformations with fixed parameters (a horizontal flip with probability 1; a rotation of exactly 2° or 4°?), and *tune* the choice and the parameters on the *validation* set after training, never on the test set;
- TTA is an ensemble and reduces variance on average, but it may also hurt; it is a final push, not a game changer. Augmentation at training time is what really matters.

11.6.2 Image retrieval

An image search engine compares images through their *semantics*, not their pixels. Cut the network before the classifier (typically at the flattening, where the vector contains all the information the classifier needs): this is the **embedding model**. Compute the embeddings of all the images of a database; given a query image, compute its embedding and rank the database by a similarity measure (mean squared error, cosine similarity, mean absolute error...). With the network trained above, the nearest neighbors of an animal are mostly animals, though the network may have learned features describing "what is not in the other class" rather than the object itself (explainability tools, Chapter 12, can tell).

No training is needed if one uses a pre-trained network. Taking *EfficientNet-B0* (the lightest of the EfficientNet family, about 4 million parameters, built by stacking a repeated block many times) and replacing its classifier with `nn.Identity()` gives an embedding model for free. A pre-trained network expects the **same preprocessing** used during its training: torchvision provides the transforms associated with the weights, typically resizing and normalization with the ImageNet channel means and standard deviations. The input size need not match exactly, but there is a lower bound (after enough poolings/strides the activation reaches 1×1), while larger inputs are fine. With ImageNet features, the nearest neighbors of a dog are dogs (ImageNet has many dog classes).

11.6.3 Transfer learning and fine-tuning on Oxford-IIIT Pets

A metaphor: you live on an island where there are only oranges and you are a master at peeling them. One day someone asks you to peel an apple: you apply the same skill (transfer learning), and then, having understood that the fruit is different, you adapt your technique more finely (fine-tuning).

The *Oxford-IIIT Pets* dataset has 37 breeds of cats and dogs, sometimes very similar, with heterogeneous resolutions, cameras and exposures, and fairly balanced classes (so accuracy is usable). The test split is divided into validation and a new test set. When using a pre-trained (frozen) backbone, the inputs must be normalized with ImageNet statistics, both in training and in validation/test.

The model is an EfficientNet-B0 backbone followed by global average pooling (PyTorch's `AdaptiveAvgPool2d(1)`: one average per channel), dropout and a linear classifier; augmentation: horizontal flip, random affine (small translations) and random erasing.

Listing 11.2: Transfer learning with a torchvision backbone.

```
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights

weights = EfficientNet_B0_Weights.IMAGENET1K_V1      # None = from scratch
model = efficientnet_b0(weights=weights)
model.classifier = nn.Sequential(nn.Dropout(0.2),
                                 nn.Linear(1280, num_classes))
for p in model.features.parameters():                 # freeze backbone
    p.requires_grad = False
preprocess = weights.transforms()                     # ImageNet normalization
```

- **From scratch** (`weights=None`): about 57% accuracy, not trivial on this dataset.
- **Transfer learning** (ImageNet weights, frozen backbone, only about 47k trainable parameters): about 61% in much less time. Not a large gain: ImageNet knows nothing of 37 breeds of cats and dogs.

- **Fine-tuning**: start from the transfer-learning model, unfreeze the last layers of the backbone (here the last 20) and retrain everything with a lower learning rate: almost 10 points of accuracy gained.

Practical advice on fine-tuning:

- Which and how many layers to unfreeze depends on the problem and on memory (47k trainable parameters vs. millions, or billions for large language models).
- Keep the **batch normalization** layers (Section 6.4) frozen: their running statistics were estimated on huge batches of a huge dataset and are usually better than what can be re-estimated on a small dataset. Unfreeze the chosen layers, then re-freeze the batch-norm ones among them.
- Variants: hierarchical (gradual) unfreezing, adapting one portion at a time; or training the unfrozen backbone and the new head simultaneously with different learning rates (smaller for the backbone), possibly different optimizers. (Parameter-efficient techniques such as low-rank adaptation are more suited to attention layers.)

Comparing the learning curves, the pre-trained models start from a much better point: "you train less and you train better". When building a new application, **start from a pre-trained model**. Finally, predicting images downloaded from the web (cropped and resized as in training) gives correct classes with high confidence.

Remark 11.3 (Using techniques not seen in the course) *In a report or a project, any technique is allowed, but each decision must be justified ("an LLM suggested it" is not a justification), and the report should present an analysis, not a single brute-force model: e.g., a 20 GB ensemble of models is not a sign of a good work.*

Chapter 12

Famous CNN Architectures and What CNNs Learn

This chapter follows the timeline of the major CNN architectures. Besides the historical perspective, it is an opportunity to introduce building blocks proposed after the basic CNN (global average pooling, separable convolutions) and different ways of arranging convolutional layers. Between architectures, we also investigate what trained CNNs actually compute.

12.1 LeNet (1998)

Everything started in 1998 with the paper *Gradient-based learning applied to document recognition* (LeCun, Bottou, Bengio, Haffner), in the Proceedings of the IEEE. The task was handwritten digit recognition: classifying 28×28 images of digits into 10 classes. The network, called **LeNet** after its first author Yann LeCun, was used to read the amounts written on bank checks and postal codes, with a large impact in automating such processes; its confusion matrix is almost perfectly diagonal.

LeNet-5 is a standard CNN with only two convolutional layers, each followed by subsampling, and dense layers for the classification. In PyTorch:

Listing 12.1: LeNet-5.

```
lenet = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5), nn.Tanh(),     # 32x32x1 -> 28x28x6
    nn.AvgPool2d(2),                               # -> 14x14x6
    nn.Conv2d(6, 16, kernel_size=5), nn.Tanh(),    # -> 10x10x16
    nn.AvgPool2d(2),                               # -> 5x5x16
    nn.Flatten(),                                  # -> 400
    nn.Linear(400, 120), nn.Tanh(),
    nn.Linear(120, 84), nn.Tanh(),
    nn.Linear(84, 10))
```

Remarks:

- The activation is the hyperbolic tangent and the subsampling is *average* pooling.
- The number of filters can be read from the number of channels: 6 in the first layer, 16 in the second.
- The filter size cannot always be read from the shapes: with "same" padding the output has the size of the input whatever the filter size. Here, though, the size drops from 32×32 to 28×28 without stride: with "valid" padding a 5×5 filter removes two rows at the top and bottom and two columns on each side, so the filters are 5×5.
- Parameters of the first layer: 6 filters $5 \times 5 \times 1$ plus 6 biases, $150 + 6 = 156$. The total is about 61k, and the vast majority is in the dense part: the layer $400 \to 120$ alone has $400 \cdot 120 + 120 = 48\,120$ parameters.

LeNet is a very shallow CNN extracting a 400-dimensional feature vector with very simple learnable features. The convolutional part has few parameters but is computationally demanding: the operations are repeated over large volumes. This is probably why more layers could not be trained at the time.

Valid padding as a design choice. The input is 32×32 (the 28×28 digits zero-padded), and valid convolutions progressively discard the periphery ($32 \to 28$, $14 \to 10$). This makes sense: the periphery of the digit images is black and irrelevant for classification. With "same" padding the last volume would be $8 \times 8 \times 16$, i.e., a feature vector more than twice as large, and the dense part would explode. When resources are scarce and the relevant content lies at the center of the image, valid padding saves a lot of dimensions in the latent representation.

CNN vs. MLP parameters. A multilayer perceptron with the same dense layers applied to the flattened 32×32 image would have a comparable number of parameters (about 86k), but none of the translation-equivariance properties. Now consider RGB input. For the MLP, the input vector triples and so does the first, largest layer. For the CNN, only the depth of the first-layer filters changes from 1 to 3: $5{\cdot}5{\cdot}3{\cdot}6{+}6 = 456$ instead of 156, and the rest is unchanged. Handling multiple input channels has almost no impact on a CNN.

Remark 12.1 (Two perspectives on choosing a CNN) *One may see the advantage of CNNs from a purely machine-learning perspective (a model with a number of parameters suitable for the training set size), or from an imaging perspective: what is relevant in images are local patterns that may appear anywhere, so we want to make the* convolution *trainable, rather than an operation designed for vectors. Both lead to CNNs.*

12.2 AlexNet (2012)

Between 1998 and 2012 neural networks went through a "winter": researchers working on them almost felt ashamed, since nobody cared about them anymore. In 2012 **AlexNet** (Krizhevsky, Sutskever, Hinton, University of Toronto) showed that a CNN could solve very sophisticated visual recognition problems, winning ImageNet by a large margin; this marks the beginning of the deep learning revolution.

AlexNet is a standard CNN, essentially a boosted LeNet: **5 convolutional layers and 3 dense layers**, about **60 million parameters** (1 000 times LeNet), input images $224 \times 224 \times 3$, 1 000 output classes. As in LeNet, the vast majority of parameters (94%) is in the dense part. Details:

- The well-known illustration splits the network into two parallel streams because the activations did not fit into a single GPU (a GTX 580 with 1.5 GB of memory; today's high-end GPUs have about 100 times as much). Training took 5–6 days, with everything programmed by hand. The convolutional part, the most memory demanding because all activations must be stored, can be split because convolutions are local: the upper part of the image went to one GPU, the lower to the other, with some mixing at certain stages and full mixing in the dense part.

- The first layer has large 11×11 filters (those of the salamander example, Chapter 10) with **stride 4** to reduce the spatial size quickly. A large stride is possible with large filters; with 3×3 filters a stride of 4 would skip parts of the image. The trend afterwards is towards smaller filters.

- ReLU activations, dropout in the dense layers.

- The reported result uses an **ensemble** of 7 networks: the same architecture trained 7 times on the same data. Random initialization and random batches lead stochastic gradient descent to different local minima, hence different models; their predictions on a test image are aggregated (majority voting, averaging posteriors), reducing the top-5 error from 18.2% to 15.4%.

Bengio, Hinton and LeCun received the 2018 Turing Award (considered the Nobel Prize of computer science), and Hinton the 2024 Nobel Prize in Physics.

12.3 VGG (2014)

VGG comes from the Visual Geometry Group of Oxford (a traditional computer vision group, authors of the reference book on multiple-view geometry) and won the 2014 ImageNet localization task (second in classification). The goal was to go *deeper*: more layers mean larger receptive fields and more sophisticated nonlinear features. VGG-16 has **13 convolutional layers** and 3 dense layers. Simply stacking layers did not work (vanishing gradients, training difficulties); the key observation was to obtain depth with **smaller filters**.

Example 12.1 (Three 3×3 layers vs. one 7×7) *Consider one-channel input and output. A single* 7×7 *layer has receptive field* 7×7*,* $49+1$ *parameters and one nonlinearity. Three stacked* 3×3 *layers have receptive fields* $3\times3 \to 5\times5 \to 7\times7$*: the same receptive field, but only* $3\cdot9+3 = 30$ *parameters and* three *nonlinearities. With* C *channels in and out (ignoring biases):* $49C^2$ *vs.* $27C^2$*. The stack is cheaper and more expressive.*

Convolutional layers are therefore organized in **blocks** of 2–3 consecutive 3×3 convolutions followed by max pooling. The last activation is $7 \times 7 \times 512$, i.e., a latent vector of about 25 000 components; attaching dense layers of 4 096 neurons gives the bulk of the about 138 million parameters (more than 120 million in the dense part). Moreover, all the activations must be stored during training (and computed at inference): about 100 MB of memory per 224×224 image. The next architectures focus on *reducing* the number of parameters, especially in the classification part, with smarter designs.

12.4 Visualizing what CNNs learn

First-layer filters. The filters of the first layer of AlexNet ($11 \times 11 \times 3$) can be displayed as images. Their correlation with the image is maximal where the content looks like them, so they detect simple patterns. Most of them are *grayscale*: they ignore color and respond to high-frequency patterns (edges at various orientations, alternating stripes, textures), similar to DCT/Fourier basis functions or to the principal components of small image patches. The others are *color* filters at low frequency (e.g., a smooth green region next to a smooth pink/red one), triggered by contrasted colors side by side. First layers extract very low-level visual patterns; successfully trained architectures learn similar first-layer filters, consistently with the template interpretation of Chapter 9. Deeper filters cannot be visualized this way: they have many channels and are correlated not with images but with stacks of activations.

Activations. One can instead look at the activations. For LeNet classifying a "2", some channels of the second convolutional layer get "hot" while for a "1" others do; after flattening, different parts of the feature vector are active, and at the end the neuron of the predicted class has the largest value.

Maximally activating patches. Take a trained CNN and a channel of a deep layer (e.g., the ninth convolutional layer). Feed the whole training set and, for each neuron of that channel, store the input image that activates it the most, cropping the *receptive field* of that neuron (a deep neuron is influenced only by its receptive field). Displaying in a row the patches of neurons of the same channel:

- different channels respond to completely different content;
- neurons of the *same* channel respond to very similar content: eyes and noses of white dogs; yellow text on a dark background (even the font is the same); smiling faces; wheels...

This is not by chance, but *by design*: all the neurons of a channel compute the same function (a cascade of convolutions and nonlinearities) applied to different receptive fields. The convolutional part of a CNN is a **huge learnable nonlinear filter**, sliding over the image and returning scores related to the semantics of the content. To minimize the classification error, training makes channels respond to specific semantic content, more specific and larger the deeper the layer. (These are real training images, and the visualization shows the image giving the *largest* value: the neuron does not respond only to that input.) This view is essential for semantic segmentation (Chapter 14).

The dreams of the network. Since gradients can be computed anywhere, we can treat the *input image* as the parameter and maximize the activation of a chosen neuron by *gradient ascent*: start from a random image, compute the activation and its gradient with respect to the input, take a step in the gradient direction, iterate. The images maximizing neurons of the first layers show simple textures, those of deeper layers show increasingly complex structures. Maximizing the score (before the softmax) of an output class, with a regularization term forcing the optimized input to look like a natural image, gives striking pictures: a billiard table seen diagonally (probably the most common view in the training data), flamingos, ground beetles, the patterns of an Indian cobra.

All this evidence confirms the interpretation: the first layers contain neurons maximally activated by tiny, individually irrelevant patterns; deeper layers respond to higher-level visual patterns over larger regions; the result is a feature vector classified by the dense part.

12.5 Network in Network and Global Average Pooling

The *Network in Network* paper (ICLR 2014) proposed two ingredients.

MLPconv layers. A convolutional layer takes a small region, computes a linear combination followed by a nonlinearity, and slides this operation over the input. Instead of stacking many layers of such simple operations, one may use fewer layers with a more sophisticated operation on each region: flatten the region and feed it to a small multilayer perceptron (a nonlinear trainable function), applied in the same sliding way. MLPconv layers were not very successful (and they are not convolutions, being cascades of linear operations and nonlinearities), but they are worth knowing.

Global Average Pooling (GAP). The second contribution solves the problem of huge latent vectors after flattening. Given the last activation f of size $H \times W \times C$, instead of flattening it into an HWC vector, GAP outputs a C-dimensional vector with the average of each channel:

$$F_k = \frac{1}{HW} \sum_{x,y} f(x, y, k), \qquad k = 1, \ldots, C. \tag{12.1}$$

In the original paper a softmax was attached directly to the GAP. Then the number of channels of the last convolutional layer must equal the number of classes (e.g., 1 000 filters for ImageNet), and all the trainable parameters are in the convolutional part. Why does this work? Each channel is the response of the same semantic filter over the whole image; averaging it measures how much the semantic content of that channel ("smiling man", "wheel", "dog") is present in the image, and normalizing these scores with a softmax suffices. One can also put dense layers after the GAP (e.g., in transfer learning, when the number of channels differs from the number of classes).

Seen as a matrix, the GAP is a sparse fixed matrix with entries $1/(HW)$: nothing to learn. A dense layer after flattening connects HWC inputs; after GAP it connects only C inputs, saving a factor HW. Removing the huge fully connected layers dramatically reduces the parameters and the risk of overfitting: GAP acts as a **structural regularizer**. In the paper's experiments, MLPconv + flattening + dense layers overfit; adding dropout reduced the test error; replacing the dense part with GAP + softmax reduced it further. In PyTorch it is `nn.AdaptiveAvgPool2d(1)` (the output size is 1×1; nothing to learn) followed by `nn.Flatten()` (just reshaping).

Further benefits of GAP.

- **Arbitrary input size.** A larger input yields larger activations; after flattening the vector would be longer and incompatible with the dense layer, while after GAP its length is still C. Note that the receptive fields do not change: the network still looks for patterns of the size seen in training, so an image zoomed in ten times would not work.
- **Translation invariance.** Convolutions are translation equivariant, but after flattening different locations are multiplied by different weights. With GAP, a translation of the content moves the activations,

and their average does not change. Experiment: train two networks (six convolutional layers with max pooling) on MNIST zero-padded to 64×64, one with flattening + dense head (almost 400k parameters), one with GAP (about 100k). Both reach the same test accuracy on centered digits. Shifting the test digits (upwards) only at inference, the GAP network's predictions are unchanged, while the flattening network fails terribly (it predicts "6" almost always).

12.6 MobileNet and separable convolutions

The first architectures were increasingly deep with many parameters; the next trend is to make them simpler and cheaper. Note also that the authors of the following works move from academia to industry: **MobileNet** is from Google, interested in running CNNs on embedded, low-power devices, where the memory and the number of operations of `Conv2d` layers are prohibitive.

Consider an input activation with M channels and spatial size $D_F \times D_F$, and a standard convolution with N filters of size $D_K \times D_K \times M$. For each of the D_F^2 output locations and each filter, one computes $D_K^2 M$ products, so the cost is

$$D_K^2 \cdot M \cdot N \cdot D_F^2.$$

A **depthwise separable convolution** splits the operation in two:

1. a **depthwise** convolution: M single-channel filters $D_K \times D_K \times 1$, the m-th applied only to the m-th input channel (like classic image filtering, no channel mixing): cost $D_K^2 \cdot M \cdot D_F^2$;
2. a **pointwise** convolution: N filters $1 \times 1 \times M$, mixing the channels at each location: cost $M \cdot N \cdot D_F^2$.

The ratio of costs is

$$\frac{D_K^2 M D_F^2 + M N D_F^2}{D_K^2 M N D_F^2} = \frac{1}{N} + \frac{1}{D_K^2},$$

i.e., a reduction by a factor of about $D_K^2 = 9$ for 3×3 filters (and N may be in the hundreds). Parameters are reduced similarly. The two operations are *not* equivalent to a standard convolution: it is a different (approximating) operation. MobileNets include many other architectural variants that we do not detail; they can be trained on Colab.

Accuracy vs. cost. A 2016 comparison of the CNNs of the time plots top-1 ImageNet accuracy (the correct class is the top-scoring one) against giga-operations per image, with circle size proportional to parameters. AlexNet: about 55% top-1, 60 million parameters. VGG-16/19: more layers, about 140 million parameters and about 30 giga-operations per image. The ideal is the top-left corner (high accuracy at no cost); MobileNets have few operations and parameters. Inception, ResNet, DenseNet (Chapter 13) are deeper and more computationally demanding than AlexNet, but more accurate and with *fewer* parameters, since these are spent mostly in extracting features (convolutions) rather than classifying them.

Chapter 13

Normalization, Explainability and Modern CNN Architectures

This chapter introduces batch normalization in the context of CNNs, shows how a CNN with global average pooling can tell *where* it looked to make its prediction (class activation maps), and concludes the tour of famous architectures with networks that are no longer purely sequential: Inception and ResNet.

13.1 Data preprocessing

Gradient-based optimization works better when data are centered around the origin and on a standard scale (even in classical computer vision, algorithms fitting transformations normalize the data to avoid numerical instabilities). Here each point is an input (an image), not a pixel. Standard options:

- **zero-centering**: subtract the mean;
- **standardization** (z-score): subtract the mean and divide by the standard deviation, obtaining zero mean and unit variance;
- **decorrelation** and **whitening** via PCA: rotate the data onto the principal components (diagonal covariance), then rescale (identity covariance).

PCA-based transformations are common for tabular data but not with CNNs, whose inputs live in a very high dimensional space; there one sticks to some form of centering. For CIFAR-10 ($32 \times 32 \times 3$):

- AlexNet-style: subtract the *mean image* computed on the training set ($32 \times 32 \times 3$ preprocessing parameters);
- VGG-style: subtract the *per-channel mean* (three numbers: the average red, green and blue levels);
- ResNet-style: subtract the per-channel mean and divide by the per-channel standard deviation.

These preprocessing parameters are *learned from data*, although not by gradient descent: they are statistics of the training set. They must be computed on the **training set only** (at every fold of a cross-validation) and then applied unchanged to validation and test data: in practice no test image is available when the model is built. When loading a pre-trained model (for inference or transfer learning) one must also load its preprocessing function.

13.2 Batch normalization in CNNs

Batch normalization (introduced in Section 6.4) applies the same idea *inside* the network: a layer, placed after convolutional or dense layers, that standardizes the activations. The point to clarify for CNNs is *over*

what the statistics are computed. During training, the activations form a four-dimensional tensor: batch $\times$ H $\times$ W $\times$ channels. Batch normalization takes, **for each channel**, all the values over all the spatial locations and all the images of the mini-batch, and computes their mean μ_c and variance σ_c^2. Then

$$\hat{x} = \frac{x - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}}, \qquad y = \gamma_c \, \hat{x} + \beta_c.$$

- μ_c, σ_c^2 are **non-trainable** parameters: they are given by a fixed formula, not optimized. A layer with 128 channels has 128×2 of them.
- Is standardization (zero mean, unit variance) really what we want? It may be convenient to let the network decide where to place the distribution: by default the layer applies a per-channel affine rescaling with **trainable** parameters γ_c, β_c (128×2 more parameters). In PyTorch: `nn.BatchNorm2d(num_features=C)`; with `affine=False` the trainable rescaling is removed.
- At inference we do not process batches but single images. The non-trainable statistics are therefore estimated over the whole training set in a *running* manner (running averages accumulated during training, not storing all activations), and then kept fixed. The layer thus **behaves differently at training and at inference time**.
- After training, batch normalization is a fixed affine map per channel, which can be merged into the adjacent linear (convolutional) layer: *no overhead at test time*.

Benefits: activations do not collapse to zero or explode, the gradient flow improves, higher learning rates and faster convergence are possible, the network is more robust to (unlucky) initializations, and it acts as a regularizer. Variants normalize over different groups of the tensor: *layer norm* (over channels and space, per sample; standard in transformers), *instance norm*, *group norm*; in CNNs batch normalization is by far the most widely used.

13.3 Multi-label classification and localization

Multi-label classification. An image often contains many things (a person holding a beer, sausages, a restaurant…). *Multi-class* classification (more than two classes) still assigns a single label per image: a softmax output is a probability distribution, so even if nothing related to the classes is in the image it returns the most likely one, and if several classes are present it returns only one. For **multi-label** classification it suffices to change the output activation: replace the softmax with a **sigmoid** on each output neuron. Each output is then an independent probability, so the network can predict several labels, or none (e.g., both "cat" and "dog"). The training labels are no longer one-hot but binary vectors with a 1 for every category present (e.g., $1, 0, 1, 0, \ldots$), and the loss is the sum of binary cross-entropies over the outputs.

Localization. If an image contains multiple objects, one would rather draw a box around each one: this is object detection (Chapter 15). Localizing a *single* object is just regression: predict four numbers, e.g., the upper-left and lower-right corners, or the upper-left corner, width and height, with linear output neurons and, e.g., the mean squared error. It is standard supervised learning (the lab below does it), provided that bounding-box annotations are available. The interesting question is how to localize when the training set contains *only class labels*.

13.4 Class activation maps

Class Activation Mapping (CAM, 2016) applies to networks whose convolutional part is followed by a GAP and a *single* dense output layer. Even if such a network is trained for classification only (categorical cross-entropy, no localization information), we can retrieve which regions of the image mostly influenced the prediction, because the last activation, before the GAP, still preserves spatial information.

The original paper shows action recognition examples. For an image predicted as "brushing teeth" the heat map highlights the toothbrush (two of them when two people brush their teeth). The face is not

highlighted: all images of human actions contain faces, so faces are not distinctive. For "cutting trees", the chainsaw and also the head are highlighted, probably because the head is not facing the camera (unusual for other classes) and the man wears a protective mask.

Derivation. Let $f_k(x, y)$ be the k-th channel of the last convolutional activation, $k = 1, \dots, C$. The GAP gives (dropping the irrelevant constant $1/N$)

$$F_k = \sum_{x,y} f_k(x, y),$$

and the score (before the softmax) of class ℓ is

$$S_\ell = \sum_{k=1}^{C} w_k^\ell F_k + b_\ell = \sum_k w_k^\ell \sum_{x,y} f_k(x, y) + b_\ell = \sum_{x,y} \underbrace{\sum_k w_k^\ell f_k(x, y)}_{M_\ell(x,y)} + b_\ell.$$

The **class activation map** of class ℓ is

$$M_\ell(x, y) = \sum_{k=1}^{C} w_k^\ell \, f_k(x, y), \tag{13.1}$$

and the score is just the sum of the map over all locations (plus the bias). The interpretation: activations are nonnegative (ReLU), so a channel contributes to a large score of class ℓ only through a large weight w_k^ℓ. A large weight means that the presence of the semantic content of channel k is relevant for class ℓ. Recall that the neurons of a channel all respond to the same content (a dog face, a smiling person...). If channel 2 responds to dogs, w_2^{dog} is large, and the weighted sum of the channels produces a heat map located over the dog. Querying the same activations for class "person" uses different weights (the dog channel has a small weight for "person"), and the map moves on the person. Typically one queries the predicted class, but any class can be queried ("cat" vs. "dog" in the same image give different maps).

Remarks:

- There is no need for C to equal the number of classes L: the dense layer adapts the sizes, so one can take any pre-trained backbone, attach GAP and a single dense layer, and train it.
- CAMs are low resolution (the size of the last activation) and look blurry. They are upsampled to the image size by *bilinear interpolation* and overlaid on it. Maps from earlier layers would have higher resolution but less semantics (more on this with segmentation).
- Averaging (GAP) encourages the identification of the whole object; global max pooling would focus on a small discriminative detail.
- The constraint of a GAP followed by a single dense layer may reduce classification performance compared to deeper heads (AlexNet and VGG had several dense layers). Training a simpler replica just for explanations does not explain the actual classifier.

Weakly supervised localization. In standard supervision, the training labels are for the same task we want to solve (class labels for classification, boxes for detection). In **weak supervision** the labels are for a different (weaker) task. CAM gives localization from class labels only: upsample the CAM of the predicted class, take its peak, set to zero everything below 20% of the maximum, and draw the bounding box around the largest remaining connected region. The box is not perfectly accurate (e.g., compared to the ground-truth box of a grasshopper), but no box was ever used in training. This also suggests a design criterion: rather than squeezing activations through many max poolings, keep some spatial resolution before a GAP, provided the receptive field is large enough.

Explainability. CAMs are perhaps the simplest form of *explainability*, a key research topic: networks with millions of parameters perform opaque computations, and in critical domains (e.g., medicine) we want to verify that the input was correctly interpreted, or let an expert confirm it. Examples:

- *Understanding errors*: an image showing a cap is predicted as "buckle"; the CAM shows the network looked at a watch strap's buckle.
- *Discovering dataset biases*: a classifier excellent on horses in training failed on new horse images. The CAM revealed that it looked at a *watermark*: all the horse training images came from a photographer who watermarked his pictures. A classifier of Siberian vs. Alaskan huskies actually discriminated snowy (white) from forest (green) backgrounds. In a past edition of this course's challenge on leaf diseases, different species had been photographed on different tables under different lights: the winning model classified the tables, and on images without the table it was close to random guessing.

Grad-CAM. The most popular saliency method removes the constraint on the head: any differentiable classifier (several dense layers, even flattening instead of GAP) can be explained. Instead of the weights of a linear output layer, the channels A^k of the last convolutional activation are weighted by the *gradient* of the class score y^ℓ with respect to them, averaged over the locations:

$$\alpha_k^\ell = \frac{1}{Z}\sum_{i,j}\frac{\partial y^\ell}{\partial A_{ij}^k}, \qquad L^\ell_{\text{Grad-CAM}} = \text{ReLU}\Big(\sum_k \alpha_k^\ell A^k\Big).$$

For a CAM-compatible network the weights reduce (up to a constant) to the CAM ones. Since derivatives are defined everywhere, Grad-CAM can be computed for earlier activations too: resolution increases (less upsampling) but the semantics becomes less meaningful the farther from the output.

Augmented Grad-CAM. The low resolution can be improved with test-time augmentation: compute Grad-CAMs of several augmented copies of the image (slightly rotated, shifted…), obtaining a set of low-resolution maps related to the original by known transformations. Viewing the chain "augment → network → low-resolution map" as a downsampling operator, a *super-resolution* problem is formulated and solved by gradient descent, yielding a high-resolution map closer to the object and better with multiple objects. It was used, e.g., to locate waste disposal sites in satellite images with a classifier trained on image-level labels only.

13.5 Inception (GoogLeNet, 2014)

So far networks were sequential: each layer feeds the next. The next architectures introduce parallel branches. **GoogLeNet** (authors mostly from Google) introduces a *block*, the **inception module**, motivated by two observations:

- the right filter size is hard to choose: distinctive patterns (e.g., the nose of a cat) may correspond to very different sizes in pixels, i.e., different receptive fields; the network should be flexible to look at several of them;
- deeper networks are better but hard to train and computationally expensive.

In the naive inception module, the input goes in parallel through 1×1, 3×3 and 5×5 convolutions and a 3×3 max pooling. To *concatenate* the outputs along the channels, all branches must preserve the spatial size: "same" padding for the convolutions, and stride 1 (with padding) for the max pooling, which then does not reduce the size but adds a nonlinearity. E.g., with a $28\times28\times256$ input and 128, 192 and 96 filters for the three convolutions, the concatenated output has $128+192+96+256=672$ channels and the module costs more than 850 million operations: stacking several such modules would be a nightmare.

1×1 **convolutions as bottlenecks.** Counterintuitively, cost is reduced by adding 1×1 convolutions. A 1×1 convolution with a single filter on a 64-channel volume outputs a single channel: it mixes all the channels at each location with learnable weights. With fewer filters than input channels it *compresses* the channels by a learnable transformation (not a mere average). Placing 1×1 bottlenecks before the 3×3 and 5×5 convolutions (and after the pooling) reduces the number of output channels and, above all, the operations, since the expensive convolutions act on thinner volumes.

The network. GoogLeNet has 22 layers (with pooling, 27): a few standard convolutional layers to reach a sufficiently large activation, then 9 stacked inception modules, and finally **GAP**, a linear classifier and softmax. It has only about **5 million parameters** (vs. about 140 million for VGG, its contemporary), almost all in the convolutional part: parameters are now spent on computing good features rather than on powerful classifiers of shallow features. It won ImageNet 2014.

Auxiliary classifiers. To fight the vanishing gradient in such a deep network, two extra classification heads (with average pooling and dense layers) are attached to intermediate layers. Training minimizes the sum of the three cross-entropy losses: the gradients of the auxiliary losses reach the early layers through fewer multiplications, so they are less likely to vanish. After training the auxiliary heads are discarded.

Take-home messages: multiple parallel branches; 1×1 convolutions as bottlenecks to reduce computation; intermediate losses to improve training convergence.

13.6 ResNet (2015)

The problem attacked by **ResNet** (again from industry, Microsoft Research) was training *very* deep networks. The authors observed that a 56-layer network had *higher* training and test error than a 20-layer one. This is not overfitting: with overfitting the training error of the deep network would keep decreasing while the test error increases; here both errors were higher throughout training. The deep network learns, yet worse than the shallow one.

This is puzzling: if 20 layers were the best solution, a 56-layer network could reproduce it by learning those 20 layers followed by 36 *identity* mappings. Apparently the identity is hard to learn (it is a very peculiar combination of filters). The idea: if the identity is hard to learn, put it in the architecture as a structural bias.

Residual block. A block of weight layers (convolutions, possibly 1×1) computes $F(x)$; a **skip connection** carries the input x and adds it to the output, before the final ReLU:

$$\text{output} = \text{ReLU}\big(F(x) + x\big).$$

If nothing useful can be done, the block can set its weights to zero and propagate the identity. The sum is differentiable, so training by backpropagation is unchanged. If $H(x)$ is the ideal mapping, the block learns $F(x) = H(x) - x$, i.e., the *residual* to add to the input: hence **residual learning**. Stacking residual blocks, the 56-layer network outperforms the 20-layer one. The intuition is that most layers of deep networks are close to the identity, performing minor modifications to their input. Gradients also flow more easily through the skip connections.

Since input and output are *summed* (not concatenated as in Inception), the block must preserve the spatial size ("same" padding) and the number of channels; otherwise a linear projection (1×1 convolution, possibly strided) adjusts the shortcut. *Bottleneck* blocks use 1×1 convolutions to reduce and then restore the channels around a 3×3 convolution. ResNet reached 152 layers (and over 1 000 in experiments), won ImageNet 2015 and roughly halved the error with respect to 22-layer networks. ResNet-50 is a standard backbone (`torchvision.models.resnet50`).

Variants.

- **Wide ResNet**: many more filters per layer.

- **ResNeXt**: combines parallel paths (as in Inception) with skip connections; implemented with *grouped* convolutions.
- **DenseNet**: each layer receives the activations of all previous layers in the block (dense connectivity by concatenation), so the classifier can exploit low-, mid- and high-level features (CVPR 2017 best paper). Its layers use the order batch norm–ReLU–convolution.
- **EfficientNet** (2019): a family of networks obtained by scaling a base network with a single hyperparameter that trades parameters/computation for accuracy, from B0 (fewer than 5 million parameters) to B7.

With the understanding of CNNs gained so far, these papers can be read and understood.

13.7 Lab: modern blocks, localization and class activation maps

13.7.1 Building blocks

Activations come in two families: the ReLU family (leaky ReLU, ELU, GELU…) and the sigmoid family; SiLU/Swish ($x\,\sigma(x)$) sits in between and is common in modern networks (one more hyperparameter to explore). Convolutions extract information hierarchically (stacking them makes sense only with nonlinearities in between); pooling reduces the spatial size, saving memory and, with max pooling, cutting redundant information (the network adapts to exploit this bottleneck). Max vs. average pooling: no winning strategy, test on your problem. The general recipe of deep learning is: define a block, stack it many times. When reading a model from the literature, at least read the abstract, introduction and conclusions of its paper.

- **VGG-style block**: (conv–ReLU)$\times n$ then max pooling; with flattening, most parameters are in the last dense layer.
- **GAP**: `nn.AdaptiveAvgPool2d((1,1))` + `nn.Flatten()`. On the same small network, the head shrinks from about 163 000 to a few hundred parameters, with equal or better performance and translation invariance. No free lunch: sometimes flattening, or several dense layers after GAP, work better.
- **Inception block**: four parallel branches applied to the same input and concatenated along the channel axis (`torch.cat(dim=1)`); the only care is that spatial sizes match (for sums or products, channels must match too). A common rule of thumb: double the channels every time the spatial size is halved, to keep the number of parameters under control.
- **Batch normalization**, introduced with Inception-v2 to let gradients flow through many layers, placed before the nonlinearity (conv–BN–ReLU). Today layer normalization is the standard in many architectures.
- **Residual block**: conv–BN–ReLU–conv–BN, add the shortcut, ReLU; if shapes differ, the shortcut is a 1×1 convolution + BN. Summing instead of concatenating saves channels, at the cost of forcing information through a bottleneck.
- **Depthwise separable convolution** (MobileNet-v1): a depthwise `nn.Conv2d(C, C, 3, padding=1, groups=C)` followed by a pointwise `nn.Conv2d(C, C_out, 1)`; parameters drop drastically. (MobileNet v1, v2, v3 are different families; v3 comes from automated architecture search.)
- Inverted bottlenecks (expanding the channels by a factor $K > 4$ inside a residual block), **ResNeXt** (parallel grouped paths summed), **DenseNet** (concatenation with a *growth rate* of channels), and **EfficientNet**'s compound scaling of depth, width and input resolution, the seed of automated deep learning (neural architecture search over a search space of sub-networks of a trained super-network).

Knowing what is inside these blocks, rather than just importing a model from `torchvision`, is what allows one to judge, and manipulate, the solutions proposed by an LLM.

13.7.2 Bounding-box regression

The dataset contains 1 200 images of cats and dogs (balanced) with bounding boxes, plus a test set of 10 images, and another with several animals per image. Images are center-cropped to a square, resized, converted from BGR to RGB, and normalized with ImageNet statistics (the backbone is pre-trained); box coordinates are normalized to $[0, 1]$ so that they do not depend on the input size. (Some images are tricky: a collage of several cats whose box is the whole image, a cat in front of a mirror.)

The regressor is an EfficientNet-B0 backbone (frozen, transfer learning) + dropout + a linear layer with 4 outputs (the box coordinates; two corners would suffice), trained with the MSE loss. As validation metric, rather than the unbounded MSE or MAE, the *Spearman* rank correlation is used (bounded and interpretable; it rewards the correct ordering of coordinates). It reaches about 0.86; the predicted boxes are reasonable though not perfect.

13.7.3 Classification, CAM and Grad-CAM

The classifier (same backbone, GAP, a single linear layer) is trained by transfer learning and then fine-tuned unfreezing the last three blocks, reaching 99.6% validation accuracy. Can we localize without the box regressor? The CAM (13.1) is computed by taking the weights of the linear layer for the target class (by default the predicted one), weighting and summing the channels of the last convolutional activation (all of them, or the top-k by weight), upsampling the map and overlaying it on the image.

Listing 13.1: Class activation map.

```
@torch.no_grad()
def compute_cam(model, x, target=None):
    feats = model.features(x)                       # (1, C, h, w)
    logits = model.classifier(feats.mean(dim=(2, 3)))
    target = logits.argmax(1).item() if target is None else target
    w = model.classifier[-1].weight[target]         # (C,)
    cam = torch.einsum('c,chw->hw', w, feats[0])
    cam = F.interpolate(cam[None, None], size=x.shape[-2:],
                        mode='bilinear', align_corners=False)[0, 0]
    cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
    return cam, target
```

The maps focus mostly on the animals' faces. Thresholding the map and taking the bounding box of the largest region above threshold gives boxes as good as, or better than, the regressor's (more focused on the most discriminative part, rather than the whole animal). Displaying the individual channels with their weights shows that the CAM is their weighted sum, with many channels of small weight.

These are *approximations* of what the network does: a perfect explanation would require accounting for all the parameters (other approaches exist, e.g., Shapley values from game theory). With several animals in an image, the CAM of the target class focuses on the largest one and ignores the others: the network was trained to classify the most probable class, not to count instances. Querying "dog" in an image with a cat and a dog works; with several dogs and cats it does not.

Grad-CAM, computed with gradients (via hooks on the chosen layer), gives similar but sharper maps and better boxes, and applies to any differentiable architecture. Computing it from earlier layers moves away from the "focus" of the network: layers closer to the classifier build the features relevant for the decision, while earlier ones mostly filter out unnecessary information, and their maps are much less informative. Such visualizations are also a valuable addition to a project report.

Chapter 14

Semantic Segmentation

This chapter and the next deal with *higher-level* visual recognition problems: tasks that go beyond associating a single quantity (a class, or a regression value) with an image.

14.1 The segmentation problem

Definition 14.1 (Segmentation) *Let $\mathcal{X}$ be the image domain, i.e., the set of pixel coordinates. Segmentation consists in identifying a set of regions $\{R_i\}$ forming a* partition *of $\mathcal{X}$:*

$$\bigcup_i R_i = \mathcal{X}, \qquad R_i \cap R_j = \emptyset \quad \forall i \neq j.$$

Until about 2012, segmentation methods were essentially **unsupervised**: a notion of similarity among pixels is defined (e.g., similar colors and spatial proximity) and pixels are clustered accordingly (*superpixel* algorithms, close to k-means in the image domain). This ignores semantics: the pixels of a person end up in several segments.

Definition 14.2 (Semantic segmentation) *Given a label set Λ, semantic segmentation maps an image $I \in \mathbb{R}^{R \times C \times 3}$ to a* semantic map *$S \in \Lambda^{R \times C}$ with the same spatial size: each pixel is assigned a label. Regions contain pixels with the same semantic content.*

In a street scene, all the pixels of persons are blue, of cars gray, of bicycles another color, and so on. It is pixel-wise classification, but each pixel is classified by looking also at its neighborhood: the same dark color may belong to a person in one place and to a bicycle in another. Semantic segmentation is *not* detection: there is no notion of individual instances, so three adjacent persons form a single connected blue region and one cannot count them (instance-level tasks are in Chapter 15). It is nonetheless very useful: in medical imaging one wants the area covered by a tissue, in remote sensing the area covered by crops, vegetation or roads, where the notion of instance makes no sense.

Annotations. Training requires pairs (image, ground-truth map) in which *every* pixel is labeled with one of the classes of interest (typically 20–80): even small clouds, fences, stones. This is extremely tedious. For natural images it can be outsourced to non-experts (e.g., crowdsourcing platforms); for medical tissues affected by rare pathologies, or aerial images of landslides and avalanches, experts are needed, which makes annotations expensive and scarce. Annotating higher-level tasks is much harder than annotating classes. For natural images, **COCO** (Common Objects in Context) provides about 80 categories and well over 100 000 annotated images; its annotations are per instance (each bird segmented separately) so as to serve several tasks; for semantic segmentation, instances of the same class share the same label.

14.2 From classification to dense prediction

A naive solution: patch classification. With the tools seen so far, one can train a single multi-class CNN that classifies *patches*. How to label a training patch? Not by majority voting over the ground truth (a patch centered on the thin stem of a flower would be labeled "sky", and thin structures would always be overwhelmed by the background), but with the label of its **central pixel**. To segment an image one then classifies a patch centered at *every* pixel, sliding by one pixel at a time (classifying tiles would only give predictions at their centers). This gives a coarse solution but is computationally a nightmare: overlapping patches share most of their content, which is processed again and again. (The CAM-based alternative of Chapter 13 uses only image-level labels; when pixel-level labels are available, it is better to use them.)

A fully convolutional stack. Computation can be shared by avoiding pooling altogether: a stack of convolutions (with "same" padding) and nonlinearities maps an $N \times N$ image to an $N \times N$ output. The last convolutional layer has L filters, one per class, and a **softmax along the channels** turns the L values at each pixel into a probability distribution; the prediction is the argmax per pixel. Training uses one-hot labels and the categorical cross-entropy at each pixel, summed (or averaged) over pixels. This is the standard output of a segmentation network. Without downsampling, however, all the activations are as large as the image with many channels: very demanding in computation and memory; and the receptive field grows slowly (by the filter size at each layer), so the network must be very deep to see context.

The tension between semantics and location. Introducing pooling increases the receptive field and the neurons of deep layers capture semantics (we know classifiers are good at it), but spatial detail is lost. In the words of one of the first papers on the subject, semantic segmentation faces an inherent tension between semantics and location: *global information resolves the "what", local information resolves the "where"*. The solution is an **encoder–decoder** architecture:

- a **contracting path** (encoder), like the convolutional backbone of a classifier: convolutions and poolings down to a latent representation that still has spatial dimensions, but much smaller ($M \times M$, $M \ll N$), with large receptive fields and many channels (thousands at the bottom of U-Net);
- an **expanding path** (decoder), which recovers a high-resolution output from this deep, low-resolution, semantically rich representation.

14.3 Upsampling

The new ingredient is an operation increasing the spatial size of activations (channel by channel):

- **nearest neighbor**: each value is replicated (four times to double each dimension);
- **bed of nails**: each value is placed in the top-left position of its block and the rest is filled with zeros;
- **max unpooling**: like bed of nails, but each value is placed where the maximum was in the corresponding max-pooling layer of the encoder (whose locations must be stored).

These operations have no learnable parameters, which is not good for accurate predictions. Cascading them with a convolution + ReLU, which mixes the upsampled channels, makes the upsampling stage trainable; by setting the number of filters, the number of channels can also be reduced.

Transposed convolution. It combines upsampling and learnable convolution in one layer. The filter acts as a *building block* of the output: each input value multiplies the whole filter, the scaled copy is placed in the output at a position determined by the stride, and overlapping contributions are summed. The input values play the role of coefficients and the filter the role of the image used to build the output (hence "transposed"). In 1D, with input (i_1, i_2), a filter (f_1, f_2, f_3) and stride 1, the output is $(i_1 f_1,\ i_1 f_2 + i_2 f_1,\ i_1 f_3 + i_2 f_2,\ i_2 f_3)$; with stride 2 the copies are displaced by two positions, giving a larger output. With multiple channels, as in `Conv2d`, each filter has as many channels as the input and produces one output channel; the number of output channels equals the number of filters.

Training. The loss is the categorical cross-entropy computed at each pixel against the one-hot label, summed over all pixels.

Remark 14.1 (Can the decoder recover lost details?) *If the stem of a flower disappears after pooling, how can upsampling bring it back? Skip connections help (below). But note that downsampling is not plain resizing: a network trained to segment stems will have channels responding to stems, and max pooling propagates the strongest activations. Relevant structures are thus encoded in the low-resolution activations, if not as pixels then as active features.*

14.4 U-Net

U-Net (2015) was proposed for biomedical image segmentation and is still a reference architecture (along with ResNet-based alternatives). In its well-known U-shaped scheme the blocks are seen "into the screen" (height and number of channels are shown).

Contracting path. Blocks of two 3×3 convolutions + ReLU without padding, followed by a 2×2 max pooling that halves the size; the number of channels doubles at each level (1 for the grayscale input $\to$ 64 $\to$ 128 $\to \ldots \to$ 1 024). From 572×572 the spatial size goes down to about 30×30: large receptive fields.

Expanding path. At each level: an up-convolution (transposed convolution) doubles the size and halves the channels; the activation of the contracting path at the same level (cropped, because unpadded convolutions shrank it) is **concatenated** through a **skip connection**, bringing high-resolution (less semantic but spatially accurate) information; two 3×3 convolutions mix the two. The output, 388×388, is obtained with a 1×1 convolution with one filter per class (two here: cell vs. background, with softmax; or one with a sigmoid). There are no fully connected layers.

Why is the output smaller than the input? Pixels near the border do not have their full receptive field available. Using valid convolutions, the border pixels serve only as *context* for the central ones, and predictions are given only for pixels with full context. To segment a large image, it is processed in tiles: to predict a (green) tile, a larger (blue) region is fed, with the missing context at the image border obtained by mirroring.

Remark 14.2 (Always predict the background) *Segmentation networks give dense predictions: every pixel needs a class, hence a* background *class is always needed. For the sea lions this means 5 types of sea lions plus background: 6 output channels.*

Fully convolutional networks. A network made only of convolutions (FCN) can be trained on patches or crops of a given size and applied to images of arbitrary size, possibly processed in tiles if very large.

Training details. U-Net was trained end-to-end, from images to segmentation maps, with large tiles, so that a batch contained a single image. The training sets were tiny (some datasets had about 30 annotated images, as is common in specific medical domains), so data augmentation was essential. Besides the usual transformations, **elastic deformations** (a grid is randomly perturbed and the image is interpolated from the original to the displaced grid) are realistic for tissues, though they would not be for natural images. Geometric transformations must be applied identically to the image *and* to its label map.

Weighted loss. With class imbalance in classification one can augment the rare classes more. In segmentation this is not possible: small details cannot be resampled independently of the rest of the image. One resorts to a pixel-wise *weighted* cross-entropy:

$$w(\mathbf{x}) = w_c(\mathbf{x}) + w_0 \exp\left(-\frac{(d_1(\mathbf{x}) + d_2(\mathbf{x}))^2}{2\sigma^2}\right),$$

where w_c balances the class frequencies and d_1, d_2 are the distances to the nearest and second-nearest cell. The second term is large for background pixels squeezed between two touching cells (both distances are small) and vanishes far from cells, forcing the network to separate adjacent cells (a task for which instance segmentation would be more suitable, but here it was tackled by segmentation only).

In PyTorch, a single block (conv–BN–ReLU–conv–BN–ReLU) can be used both in the contracting and in the expanding path: what changes is how the blocks are connected.

Example 14.1 (Kidney histopathology) *In a collaboration with a medical research institute, kidney tissue images had to be segmented into four classes: interstitium, tubules, glomeruli, other. The area of the interstitium is a biomarker of the progression of some diseases. The challenge is that tissues change a lot across patients and pathologies, and dense annotations are terribly expensive: a single network trained on a single dataset would be impractical. The solution was to train on healthy subjects and then* fine-tune *on very sparse annotations of the new patient, in the form of* scribbles *drawn by the pathologist (five minutes instead of two hours of annotation).*

14.5 From a CNN classifier to a fully convolutional network

There is a direct way of turning a classifier trained on image-level labels into a segmentation network (not via saliency maps). The classifier may have been trained on patches labeled from the annotations: for the sea lions, the annotations were colored dots on each animal, so 90×90 patches centered on the dots (and background patches) give a patch classification dataset.

What happens if we feed the trained classifier a *larger* image? Convolutions are perfectly defined on larger inputs: a trained filter can be slid over any image, and the convolutional part simply returns a larger activation. Everything breaks at the dense layers, which expect an input of exactly the size seen in training: for the extra activations there are no weights.

Dense layers as 1×1 convolutions. Suppose the convolutional part maps an $N \times N$ image to a $1 \times 1 \times C$ activation $\mathbf{x}$, followed by a dense layer with L outputs:

$$s_\ell = \sum_{j=1}^{C} w_{\ell j} x_j + b_\ell = \mathbf{w}_\ell^\top \mathbf{x} + b_\ell.$$

The inner product $\mathbf{w}_\ell^\top \mathbf{x}$ is the correlation (convolution) of the $1 \times 1 \times C$ volume $\mathbf{x}$ with a $1 \times 1 \times C$ filter whose entries are $\mathbf{w}_\ell$, evaluated at the single location. So the ℓ-th output neuron is the ℓ-th filter of a convolutional layer with 1×1 filters and bias b_ℓ: copying the trained weights and biases of all L neurons into a `Conv2d` with L filters of size 1×1 gives *exactly* the same predictions on images of the training size. On a larger image, however, the 1×1 convolution now works on a larger activation and returns L output channels with spatial extent. Every dense layer can be converted this way (a dense layer $L \to N$ becomes N filters of size $1 \times 1 \times L$): the classifier becomes a **fully convolutional network** without any segmentation annotation, just by copying weights.

GAP and flattening. If the network has a GAP, it is also a linear operation: its k-th output corresponds to a filter of size $H \times W \times C$ equal to $1/(HW)$ on channel k and zero elsewhere, so it can be written as a convolution too. If the network has a flattening on an $H \times W \times C$ activation, each weight vector $\mathbf{w}_\ell$ of the next dense layer has HWC entries: reshaping it into an $H \times W \times C$ filter, the neuron computes the convolution of the activation with that filter, evaluated at a single location. This becomes a convolutional layer with $H \times W$ filters and **valid padding** (the filter was trained to be applied to that region only, giving a single output); on larger inputs it slides and produces a spatial output.

Heat maps. The output of the FCN on a larger image is a volume with L channels, one per class; applying the softmax along the channels, each channel is a *heat map* of the probability of that class over the image. In the original paper, a classifier trained on ImageNet gives, for an image with a cat and a dog, a "tabby cat" channel active on the cat and a dog channel active on the dog: a coarse segmentation without any

segmentation annotation. For the sea lions, a classifier trained on 90×90 patches was applied in a single pass to 5k $\times$ 4k (20-megapixel) images: no patches to crop and no repeated computations. (The heat maps were not excellent, because the classifier itself was rather poor.)

Increasing resolution. The heat map is downsampled by the overall factor F of the network (pooling and strides). Options:

- simple upsampling (replicating values) gives a coarse map;
- **shift-and-stitch**: run inference F^2 times on the image shifted by $0, \dots, F-1$ pixels in each direction and interleave the outputs into a full-resolution map. For $F = 2$: the original, shifted by one row, by one column, and by both, assembled into a map twice as large;
- if the network has only strided convolutions (no max pooling), the filters can be dilated by padding them with zeros so that a single pass gives full resolution;
- **learnable upsampling** (as in the decoders above), which gives much better segmentation maps, as shown in the same paper.

14.6 Lab: fully convolutional networks and U-Net

14.6.1 From a patch classifier to an FCN (sea lions)

The dataset contains 96×96 patches cut from one very large aerial image of sea lions, with six classes: adult males, subadult males, adult females, juveniles, pups, background. From domain knowledge: males and subadult males are two to three times larger than females, while pups differ from adults by the color of the fur; we expect confusion between adult and subadult males, and between adult females, juveniles and subadult males (for size reasons). The predefined split has 3 600 training and 1 200 validation and test patches. A custom `Dataset` (implementing `__len__` and `__getitem__`) converts arrays to tensors and applies simple augmentation (random horizontal and vertical flips, color jitter); data loaders use several workers and pinned memory.

The CNN has four conv–ReLU blocks (32 to 256 filters) and three max poolings ($96 \to 12$), then flatten, dropout, `Linear`$(12{\cdot}12{\cdot}256, 256)$, ReLU, `Linear`$(256, 6)$. The first dense layer alone has $(12{\cdot}12{\cdot}256{+}1){\cdot}256 \approx 9.4$ million parameters. It reaches about 64.6% validation accuracy (six probably unbalanced classes); errors are those expected (adult vs. subadult males; females vs. juveniles).

Conversion: the first dense layer becomes `Conv2d(256, 256, kernel_size=12, padding=0)`: on the $12 \times 12 \times 256$ activation, a $12 \times 12 \times 256$ filter computes a single value, and 256 filters give $1 \times 1 \times 256$; parameters: $(12 \cdot 12 \cdot 256 + 1) \cdot 256$, exactly as the dense layer. The second becomes `Conv2d(256, 6, kernel_size=1)`. Weights are copied (with `.clone()`, so that the two models do not share memory), reshaping the dense weights with `.view` into kernel shape:

Listing 14.1: Copying dense weights into convolutional layers.

```
fcn.load_state_dict(cnn.state_dict(), strict=False)   # conv blocks
with torch.no_grad():
    fcn.fc1.weight.copy_(cnn.fc1.weight.view(256, 256, 12, 12).clone())
    fcn.fc1.bias.copy_(cnn.fc1.bias.clone())
    fcn.fc2.weight.copy_(cnn.fc2.weight.view(6, 256, 1, 1).clone())
    fcn.fc2.bias.copy_(cnn.fc2.bias.clone())
```

(The view of the dense weights must match the order in which `Flatten` unrolled channels and positions, which it does for (channels, height, width) tensors.) Always verify such manipulations: on a random patch, the maximum absolute difference between the outputs of the CNN and of the FCN (squeezed) is zero. Feeding the whole large image gives a heat map 8 times smaller (the overall downsampling, 96/12), rescaled to the image size and plotted class by class: background (sea, rocks) and adult females look convincing, juveniles less so. This is a form of weakly supervised segmentation: classification labels used for a more complex task. Compared to looping over patches in Python, the whole computation is a single pass on the GPU,

with exactly the same result. Such FCNs are no longer used for segmentation, but the trick of turning dense layers into convolutions to accept any input size is a useful tool, like residual learning.

Limitation. If the image is very large compared to the patch (a huge cat in a big image, with 96×96 patches), each patch sees only a tiny part of the scene: the classifier has no idea of what is in the rest of the image. Segmentation needs both local information and *global* context.

14.6.2 U-Net on Cityscapes

Cityscapes is an autonomous-driving dataset: images taken from a car in cities, with pixel-level labels. Its 30 classes are grouped into 8 macro classes: void/unlabeled (static, ground), road-like (flat human constructions), construction (buildings, walls, fences), traffic objects (signs, lights), nature (vegetation, terrain), sky, humans (pedestrians, riders), vehicles. Images are landscape-shaped and downscaled to run on Colab (pedestrians become a few pixels). Stratified splitting is impossible (labels are per pixel), but imbalance matters: sky and road dominate, while pedestrians are small and rare. The bottom of every image always shows the hood of the car, which could be ignored altogether.

Remark 14.3 (Augmentation in segmentation) Photometric *augmentations (color) can be applied to the image alone.* Geometric *augmentations (flips, rotations, crops...) must be applied identically to the image and to its label map.*

Encoder–decoder and autoencoders. U-Net has the structure of an *autoencoder* (encoder, bottleneck, decoder; Chapter 8). Autoencoders learn compressed representations; aligning the embeddings of different modalities (as CLIP does for images and text) yields multimodal models; decoders can generate data from compressed representations (as in generative models). In U-Net, the encoder–decoder path processes the whole image and provides *global* information (the bottleneck enlarges the receptive field, which was limited to 96×96 in the FCN above), while the skip connections copy *local*, geometric information from the encoder to the decoder.

Listing 14.2: A small U-Net.

```
class UNetBlock(nn.Module):
    def __init__(self, c_in, c_out, n=2):
        super().__init__()
        layers = []
        for i in range(n):
            layers += [nn.Conv2d(c_in if i == 0 else c_out, c_out, 3, padding=1),
                       nn.BatchNorm2d(c_out), nn.ReLU()]
        self.block = nn.Sequential(*layers)
    def forward(self, x):
        return self.block(x)

class UNet(nn.Module):
    def __init__(self, c_in=3, n_classes=8):
        super().__init__()
        self.d1, self.d2 = UNetBlock(c_in, 32), UNetBlock(32, 64)
        self.pool = nn.MaxPool2d(2)
        self.bottleneck = UNetBlock(64, 128)
        self.up = nn.Upsample(scale_factor=2, mode='nearest')
        self.u1 = UNetBlock(128 + 64, 64)
        self.u2 = UNetBlock(64 + 32, 32)
        self.out = nn.Conv2d(32, n_classes, kernel_size=1)
    def forward(self, x):
        d1 = self.d1(x)
        d2 = self.d2(self.pool(d1))
        b = self.bottleneck(self.pool(d2))
        u1 = self.u1(torch.cat([self.up(b), d2], dim=1))   # skip connection
```

```
        u2 = self.u2(torch.cat([self.up(u1), d1], dim=1))  # skip connection
        return self.out(u2)                                 # (B, classes, H, W)
```

The upsampling layer (here nearest neighbor; bilinear interpolation or others are possible) is deterministic and not trainable; the "intelligent" part is the convolutional block that follows, whose input channels account for the concatenation of decoder and encoder activations. A final 1×1 convolution outputs one channel per class.

Metric: mean Intersection over Union. The loss is the pixel-wise cross-entropy. The metric is the **Jaccard index**, i.e., the *intersection over union*: for each class, the number of pixels in the intersection of predicted and ground-truth regions divided by the number of pixels in their union,

$$\mathrm{IoU}_c = \frac{|P_c \cap G_c|}{|P_c \cup G_c|},$$

averaged over classes (mIoU). It is computed with the `torchmetrics` library (multi-class Jaccard index), ignoring the void class and using a weighted average to account for differently represented classes (so that the rare pedestrians count).

Results. Already after one epoch the predictions are reasonable; the mIoU grows from about 0.58 to 0.73 and reaches about 0.88 at the end (a high value for this metric, which is usually hard to maximize), with early stopping when the model starts overfitting. Per-class IoU shows weaknesses on persons and traffic infrastructure: not a model to deploy on a real car. Plotting the per-class probability maps (rather than only the argmax) shows that the network captured more than the hard prediction reveals.

Lessons for large images. In the second challenge of the course (whole-slide images, very high-resolution scans of tissues, with small cells to classify), downsampling the images to fit them into memory destroyed the details of small objects and led to low performance; working on *patches* (possibly selected using the available segmentation masks) worked better.

Foundation models. The state of the art for segmentation includes foundation models such as Meta's *Segment Anything* (SAM), which take both images and text prompts: asked for "zebra", it segments every zebra and also separates the individuals, on images, videos and even 3D data. In practice, rather than reinventing the wheel, one starts from existing (pre-trained) models, e.g., U-Nets trained on related tasks, and fine-tunes them.

Chapter 15

Object Detection and Instance Segmentation

Object detection is a high-level visual recognition problem: it involves spotting several portions of an image (and possibly relating them), rather than associating a single quantity with the whole image. We build it up from simpler tasks.

15.1 Localization and pose estimation

Localization. Given an image containing a single main object, draw a bounding box around it. The box is described by four numbers, e.g., the coordinates of the upper-left corner plus height and width (x, y, h, w), or the two opposite corners. This is a **regression** problem: keep any CNN backbone and replace the output layer with **four** neurons. Which activation? Not a softmax: we are not estimating probabilities, and the four numbers need not sum to one. Use a *linear* activation (no activation); a ReLU if negative values must be excluded (but then boxes cannot extend beyond the image on one side); a sigmoid if coordinates are expressed relative to the image size, in $[0, 1]$. In PyTorch it is a `nn.Linear(features, 4)`, trained with a regression loss (MSE, MAE), not cross-entropy. Relating the output layer (number of neurons, activation, loss) to the task is a fundamental skill.

Human pose estimation. Estimate the positions of K body joints (e.g., 14, including head and neck, which are not joints strictly speaking): $2K = 28$ numbers, since joints are independently located (unlike the corners of a box, constrained by geometry). It is again regression with $2K$ linear outputs, typically obtained by transfer learning from a classification backbone, and requires annotated joints for training. The network always outputs all joints, even *invisible* ones (a leg hidden behind a desk): it infers them from the relative positions of the visible body parts, which is reasonable. Networks such as OpenPose estimate many more keypoints. With *several* people, a regression network is not enough: it would output a single right hand, taken from anywhere. Multi-person pose estimation requires producing multiple instances, as detection does.

A multiple-output problem. In object detection the number of outputs depends on the input content: a picture of a crowded classroom yields many boxes for the class "person", a picture of an empty room none. This is unusual in machine learning, where a model maps an input to a single output.

15.2 Multitask learning

The simplest multitask problem is **classification + localization** of an image containing a single object: the output is a categorical variable plus four real numbers. The network has a shared backbone and **two heads**: a *classification head* with one neuron per class and softmax, and a *regression head* with four linear neurons. There are now two losses, the categorical cross-entropy $\mathcal{L}_S$ and the regression loss $\mathcal{L}_R$ (e.g., MSE).

Gradient descent minimizes a *scalar* function (with two objectives one would face Pareto optimality, with no single minimum), so the two losses are combined, typically convexly:

$$\mathcal{L} = \alpha\,\mathcal{L}_S + (1-\alpha)\,\mathcal{L}_R, \qquad \alpha \in [0,1].$$

How to set α? It is not a hyperparameter like the others: models trained with different α minimize different losses, so their losses cannot be compared to select the best one. The practical rule is to make the two terms of comparable magnitude. If the box coordinates are measured in pixels (integers on a 200×200 grid), the MSE becomes huge and dominates: the network optimizes only localization. Normalize the coordinates or choose α so that the losses are in the same range (e.g., if one loss is 100 times the other, compensate); when reusing an architecture from the literature, reuse its weights α too. (One loss may be a sum of logarithms, the cross-entropy, and the other not, so they may change differently during training; what matters is that their typical values and ranges are comparable.) In PyTorch the backbone features are fed to both heads, the forward returns two outputs, and the total loss is `lambda_cls * loss_cls + lambda_box * loss_box`.

15.3 Object detection

Definition 15.1 (Object detection) *Given a fixed set of categories Λ and an image, detect all the instances of those categories: the output is a variable-length list of tuples (x, y, h, w, ℓ), one per object, with its bounding box and label.*

Training requires images annotated with all the bounding boxes and their classes.

Intersection over Union. The quality of a detection is measured by comparing the predicted box B_p with the ground truth B_{gt}:

$$\mathrm{IoU}(B_p, B_{gt}) = \frac{\mathrm{area}(B_p \cap B_{gt})}{\mathrm{area}(B_p \cup B_{gt})}$$

(the Jaccard index of pattern recognition). A detection that fully contains a much smaller ground truth has a poor IoU, since the union is large; so does a detection covering only part of a large object. IoU is mainly used for evaluation (a detection is considered correct if its IoU exceeds a threshold, e.g., 0.5), while training uses classification and box-regression losses.

Sliding window. The straightforward solution with a pre-trained classifier (say AlexNet, input 224×224, plus a *background* class, as in segmentation): crop all the 224×224 windows of the image and classify each. Computationally a nightmare (convolutions over overlapping windows are recomputed many times), and objects larger than the window (the car in the foreground) or of different proportions are missed. Using windows of many sizes and aspect ratios, resized or fed to a CNN accepting variable sizes, is not viable either. The only advantage: no training is needed.

15.3.1 R-CNN (2014)

R-CNN (Region-based CNN, Girshick et al.; Ross Girshick is a major contributor to the whole family of detectors that follows) avoids testing all possible windows by using an external **region proposal algorithm**. Such algorithms (e.g., selective search) are not trainable: they are heuristics finding regions with homogeneous texture or color, isolated blobs, uniform areas, and return about 2 000 candidate boxes per image. They have very high recall (any object is most likely inside some proposal) and very low precision. Then:

1. each proposal is cropped and *warped* to the input size of a pre-trained CNN (AlexNet or VGG at the time), losing its proportions;

2. the CNN computes a latent vector for each region; these (large) vectors are stored for the whole training set, each labeled with the class of the ground-truth object it overlaps, or *background* if the overlap is small or null (PASCAL VOC: 20 classes + background);

3. class-specific **SVMs** are trained on the latent vectors (at the time, SVMs on top of CNN features were considered better classifiers; a neural network would allow end-to-end training);

4. a **bounding-box regressor** is trained to adjust the proposals for better localization.

Every detector of the family keeps the pair *classifier + box regressor*. Limitations: no end-to-end training (several losses: hinge loss for SVMs, regression loss, possibly fine-tuning of the backbone); proposals are not learnable (many regions on irrelevant objects); training is slow and needs huge disk space for the stored features; inference takes about a minute per image, while applications (tracking people, counting cars in a video) need real time.

15.3.2 Fast R-CNN (2015)

Six months later, **Fast R-CNN** (single author, Girshick, now at Microsoft) removes the main inefficiency: instead of cropping and warping each region and running the CNN on it, run the convolutional backbone *once* on the whole image, and **project the proposals onto the last convolutional activation** (the latent representation, which still has a spatial extent). The mapping of coordinates follows from the downsampling (strides, poolings). Overlapping regions no longer recompute convolutions.

RoI pooling. The projected regions have different sizes in the latent space, so they cannot be flattened and fed to the same dense layers. The *Region of Interest pooling* layer fixes an output size (e.g., 3×3, or 7×7), divides each region, channel by channel, into a grid of that size, and max-pools each cell. A 27×32 region and a 5×6 region both become 3×3 per channel: the larger the region, the coarser the grid. The fixed-size result is flattened, transformed by a few dense layers into a feature vector, and fed to two heads: a softmax classifier (classes + background) and a box regressor.

Only the RoI pooling and the heads are repeated for each region; the whole network is trained *end to end* with a multitask loss and the backbone runs once. Inference drops to about 2 seconds per image, now dominated by the external region proposal algorithm.

15.3.3 Faster R-CNN (2015)

Faster R-CNN ("towards real-time object detection with region proposal networks") replaces the region proposal algorithm with a **Region Proposal Network** (RPN), making proposals trainable: the detector learns to propose only regions likely to contain objects of the classes of interest, not any blob or uniform area. The RPN outputs $N_{\text{prop}} \times 4$ boxes, with N_{prop} smaller than the 2 000 of the heuristics because proposals are more focused; the rest is as in Fast R-CNN.

How can a network output a variable number of regions? Like region proposal algorithms, the RPN is *class agnostic*: it says "there might be an object here", without saying which. The trick is to associate with *every location* of the backbone's last activation K predefined boxes, the **anchors** (e.g., $K = 9$: squares, horizontal and vertical rectangles, at three scales), centered at the corresponding image location. Then:

- a **classification branch** (a few trainable convolutional layers, then a 1×1 convolution with $2K$ filters and a pairwise softmax) outputs at each location, for each anchor, the probability that the anchor contains / does not contain an annotated object: the *objectness score*;

- a **regression branch** (a 1×1 convolution with $4K$ filters) outputs at each location four corrections for each anchor, to better match the object.

No new architectural element is needed: just convolutions (in particular 1×1) wisely used, and a suitable *loss*. That a channel corresponds to an anchor is purely a matter of how the loss is defined: during training, each anchor at each location is compared (via IoU) with the annotated boxes to decide whether it is positive or negative, and the regression targets are the offsets between the anchor and its matched box.

At inference, the objectness scores are thresholded, and where several overlapping anchors have high scores only the best is kept by **non-maximum suppression** (NMS); about 300 proposals remain. They are projected onto the latent representation, RoI-pooled, classified and refined by the detection head. With

L classes (background included), the regressor outputs boxes for the $L-1$ object classes only: no box is predicted for background.

Flexibility of anchors. The architecture does not depend on the anchor shapes, only on their number K. To detect boats at sea, use very elongated horizontal anchors: only the loss (the matching of anchors with annotations) changes. Other output shapes are possible: rotated rectangles, oriented line segments (lane detection with anchors corresponding to orientations), ellipses (but there is no closed form for the overlap of two ellipses, so nobody does it), or **3D bounding boxes** (center and three sizes: six numbers for an axis-aligned box, plus orientation): with RGB-D or LiDAR data annotated in 3D and mapped to the image by camera calibration, the network can be trained to predict 3D boxes, even from a single image, since real objects have typical sizes (a toy car would be predicted as five meters long). Many detection problems fit this framework.

Training. Four losses are involved: objectness and box regression for the RPN, classification and box regression for the detection head. A common procedure: start from a pre-trained backbone; train the RPN to detect all objects ignoring their class; freeze it and train the detection head (possibly fine-tuning the backbone, pre-trained on 1 000 classes while detection usually involves 20–80); since the backbone changed, fine-tune the RPN again; finally a joint pass over the whole network. Inference reaches about 5 frames per second. Faster R-CNN is a **two-stage detector**: one stage proposes regions, the other classifies and refines them. (It was used, in a 1D version, to detect and classify spikes in time series in an industrial project, and in medical imaging.)

15.3.4 YOLO

YOLO ("You Only Look Once", Redmon, Girshick et al.) is a **single-shot** detector: it frames detection as a single regression from pixels to boxes and class probabilities. The image is divided into a coarse grid (e.g., 7×7, the cell size depending on the image size); each cell has B anchor boxes; for each cell and anchor the network predicts the box adjustments (4 numbers), an objectness score, and the class probabilities (C numbers): an output tensor of size $7 \times 7 \times B \times (5 + C)$. It is like the RPN principle implemented within a single stack of convolutions. YOLO became the reference when high frame rates are needed, with many later versions (YOLOv11, v12…, from companies such as Ultralytics) addressing other tasks besides detection. Two-stage detectors (Faster R-CNN, available, e.g., in Detectron, developed by Girshick at Meta) are preferred for more accurate localization.

15.4 Instance segmentation

For an image with two dogs and a cat: object detection returns three boxes with classes; semantic segmentation returns one region for the cat and one single region for both dogs (pixel-wise classification); **instance segmentation** assigns a different label to each instance, so each dog gets its own mask. The output is a list as in detection, where each tuple contains the box, the class and a binary mask of the object.

Mask R-CNN (2017) (again with Girshick) cascades object detection and semantic segmentation: on top of Faster R-CNN, for each detected box the corresponding region of the latent representation is cropped (RoI pooling, in an improved version called RoIAlign) and fed to a small fully convolutional, U-Net-like *decoder* that predicts a binary mask of the object inside the box (the class may also be re-estimated). The results are impressive in both detection and instance segmentation: tiny details such as a partially visible handbag, elephants, a dining table, many sheep. Adding another output branch predicting **keypoints** combines instance segmentation with multi-person pose estimation, including the positions of non-visible joints.

Feature Pyramid Networks (FPN). To detect objects at different scales, an FPN backbone has a contracting path and an expanding path (as in U-Net), but predictions (RPN proposals and detections) are

made at *every* resolution of the expanding path: large objects from the coarse levels, small objects from the fine ones. FPN backbones are common in Faster/Mask R-CNN.

Example 15.1 (Counting cells) *In an industrial project, a Mask R-CNN with FPN was trained to segment and count cells. The company had its own hand-crafted image-analysis algorithm; separating touching cells and handling strong intensity gradients (dark and bright regions) make such algorithms hard to design, and they must be redesigned whenever the cell preparation or the cell type changes. A trained "black box" network adapts by retraining, which is why companies and medical research institutes often prefer it.*

Beyond this course. Research directions building on these tools include: 3D object detection for autonomous driving fusing camera images and LiDAR point clouds (with depth possibly estimated from a single camera), trained with self-supervision since 3D annotation is a nightmare; segmentation without segmentation labels (e.g., learning from what a human operator removes from a waste-sorting conveyor belt); segmentation in extreme settings (camouflaged animals, 3D medical imaging); synthetic generation of annotated 3D medical data with style transfer; 6-degrees-of-freedom pose estimation (object and camera pose); human pose estimation from drones in sports; networks processing raw photon streams of emerging single-photon sensors; self-supervised, contrastive and federated learning. Advanced topics such as unsupervised and self-supervised learning, anomaly detection, transformers and multimodal learning, generative models (GANs, diffusion), deep learning on graphs and 3D data, adversarial attacks and explainability go beyond this part of the book.

15.5 Lab: multitask learning and two-stage detectors

15.5.1 Classification and localization with two heads

The cats-and-dogs dataset with one box per image (Chapter 13) is now tackled *directly*: the label contains both class and box. Compared to CAM-based localization this needs box labels, but gives direct control and a direct measure of localization quality. Boxes are normalized so that the network predicts fractions of the image size. The data contain difficulties (whole bodies, faces only, other subjects partially visible), so performance below 100% is expected; classes are balanced.

Augmentation with boxes. As in segmentation, photometric augmentations apply to the image only, while geometric ones must be applied to the box too: a horizontal flip of the image must flip the box. In practice coordinates are mapped back to pixel space, transformed together with the image, and normalized again.

Metric. Box quality is monitored with the *Spearman* rank correlation $\rho \in [-1, 1]$: values are converted to ranks and compared (+1: same ordering, −1: inverted, 0: unrelated). It captures monotonic rather than linear dependence.

Listing 15.1: A two-headed network for classification and localization.

```
class ClsLocNet(nn.Module):
    def __init__(self, backbone, n_feat, n_classes=2, p=0.2):
        super().__init__()
        self.backbone = backbone                        # e.g. EfficientNet features + GAP
        self.cls_head = nn.Sequential(nn.Dropout(p), nn.Linear(n_feat, n_classes))
        self.box_head = nn.Sequential(nn.Dropout(p), nn.Linear(n_feat, 4))
    def forward(self, x):
        f = self.backbone(x)
        return self.cls_head(f), self.box_head(f)      # (logits, box)

# one training step
logits, boxes = model(images)
loss = lambda_cls * F.cross_entropy(logits, labels) \
     + lambda_box * F.mse_loss(boxes, target_boxes)
```

```
loss.backward()
```

The box head predicts the top-left and bottom-right corners (top-left corner + width and height would be equivalent). Even though there are two tasks, the optimization is single-objective, on the weighted sum; here equal weights work, but in general they must be tuned with care, like a regularization coefficient. Results: accuracy rises from 74% to 97% and Spearman's ρ from 0.7 to 0.82; fine-tuning the last three blocks gives 99% accuracy (only two errors) and a slightly better ρ. Direct regression is more sensitive to noise than one might hope: some boxes cut part of the animal or include extra regions, but the animals are localized correctly.

15.5.2 Faster R-CNN and Mask R-CNN on Penn-Fudan

Remark 15.1 (Detection vs. semantic vs. instance segmentation) *Semantic segmentation does not count objects; object detection identifies (and so counts) instances with boxes; instance segmentation is the intersection of the two: a mask for each instance.*

The *Penn-Fudan* pedestrian dataset has 170 images, two classes (pedestrian, background), and for each image boxes, labels, instance masks, instance ids, areas and the `iscrowd` flag (marking annotations of groups of objects). Some visible people are *not* annotated: a good model may detect them and be penalized by the evaluation. Ground truth is not always correct, so look at the data. Geometric transformations are applied consistently to the image and all its targets (boxes, masks).

The pre-trained model. `torchvision.models.detection` provides Faster R-CNN with a ResNet-50 FPN backbone (about 41 million parameters). Used as is (pre-trained on COCO), it shows the expected behaviour: very high recall, low precision, many false positives with high confidence (e.g., people detected in printed patterns), since it is designed to over-generate regions so that no object is missed. Visualizing proposals by FPN level shows that the deepest levels are the most informative, and that proposals are extremely dense around objects.

Non-maximum suppression. Among heavily overlapping boxes keep only one, according to a policy: discard boxes whose IoU with a higher-confidence box exceeds a threshold, or discard boxes contained in others. By default a light NMS is applied; detections are also filtered by a confidence threshold (both thresholds default to 0.5).

Fine-tuning. The box predictor is replaced for two classes and the whole model is fine-tuned with the original detection losses (classification, and a smooth-L_1 distance for boxes), a small weight decay and cosine-annealing learning-rate scheduling. In few epochs the detection F1 reaches about 95% (largely by *forgetting* false positives: the general model already knows people). The confidence threshold is a hyperparameter that can be tuned on validation (like test-time augmentation choices); here tuning does not improve, and a high threshold is preferred for precision since recall is already very high. Results transfer to the test set.

Instance segmentation. Mask R-CNN = Faster R-CNN + a small fully convolutional mask head predicting, inside each box, the pixels belonging to the object (as with the sea lions of Chapter 14, but per instance). The pre-trained model has slightly more parameters (the mask head), and before fine-tuning its precision is again low, also because it finds unannotated people (human labeling is always imperfect). After fine-tuning, the mask F1 is about 95%, likely the upper bound given the annotation errors; a confidence threshold of 0.8 chosen on validation slightly improves box detection too, and the test masks are very close to the ground truth.

Zero-shot, text-conditioned models. Recent models, such as the latest version of Meta's Segment Anything, perform *zero-shot*, *text-conditioned* detection and instance segmentation. *Conditioning* means controlling a general model (able to solve many tasks) with information aligned to its internal representation, here natural language; *zero-shot* means no retraining is needed (Mask R-CNN is not zero-shot). Asking for "person" segments every person; changing the prompt changes the target; combined with simple processing,

the masks can be used, e.g., to blur faces. These models run on modest GPUs and can be used for countless tasks, provided one understands what they do.